C Plus Plus Programming MCQs

19 Topics • 1333 Questions

Introduction To C++

Q1. What is the extension of a C++ source file?

A. .cpp ✓
B. .cc
C. .cxx
D. .c
Answer: Option A — .cpp

Q2. Which of the following is not a fundamental data type in C++?

A. char
B. float
C. boolean ✓
D. double
Answer: Option C — boolean

Q3. What does the 'cin' object in C++ represent?

A. File output stream
B. File input stream
C. Standard output stream
D. Standard input stream ✓
Answer: Option D — Standard input stream

Q4. What keyword is used to define a class in C++?

A. struct
B. interface
C. class ✓
D. object
Answer: Option C — class

Q5. In C++, what operator is used to allocate dynamic memory?

A. alloc
B. new ✓
C. allocate
D. None of the above
Answer: Option B — new

Q6. What is the syntax for a single-line comment in C++?

A. --
B. #
C. // ✓
D. /* */
Answer: Option C — //

Q7. What function is used to output data to the console in C++?

A. print
B. display
C. write
D. cout ✓

Answer: Option D — cout

Q8. Which header file is needed to use the 'cin' and 'cout' objects in C++?

A. <iostream> ✓

B. <stdio.h>

C. <conio.h>

D. <fstream>

Answer: Option A — <iostream>

Q9. What does the 'endl' manipulator do in C++?

A. Ends the program

B. Moves the cursor to the start

C. Clears the screen

D. Inserts a new line ✓

Answer: Option D — Inserts a new line

Q10. What is the output of the following code: cout << (5 > 2) ? "True" : "False";

A. TRUE ✓

B. FALSE

C. 5

D. 2

Answer: Option A — TRUE

Q11. Which keyword is used to define a constant in C++?

A. static

B. final

C. const ✓

D. constant

Answer: Option C — const

Q12. What is the output of the following code: int x = 5, y = 2; cout << x / y;

A. 2.5

B. 2 ✓

C. Compilation error

D. 2.2

Answer: Option B — 2

Q13. What is the correct way to declare a pointer in C++?

A. int ptr;

B. pointer int;

C. int* ptr; ✓

D. ptr int;

Answer: Option C — int* ptr;

Q14. What is the keyword used for inheritance in C++?

A. extends

B. inherits

C. derive

D. : ✓

Answer: Option D — :

Q15. What function is used to return the length of a string in C++?

A. length()

B. size() ✓

C. strlen()

D. length

Answer: Option B — size()

Q16. What is the output of the following code: int x = 10; cout << "Value of x is: " << x++ << endl;?

A. Value of x is: 11

B. Value of x is: 9

C. Compilation error

D. Value of x is: 10 ✓

Answer: Option D — Value of x is: 10

Q17. Which of the following is not a valid C++ identifier?

A. _variableName

B. 2ndVariable

C. variable_name@

D. VariableName

E. Both (B) and (C) ✓

Answer: Option E — Both (B) and (C)

Q18. What is the result of the expression: sizeof(int) in C++?

A. Size of int in bytes ✓

B. Size of int in bits

C. Error: sizeof cannot be applied

D. Implementation-defined

Answer: Option A — Size of int in bytes

Q19. Which C++ feature allows a function to call itself?

A. Polymorphism

B. Inheritance

C. Iteration

D. Recursion ✓

Answer: Option D — Recursion

Q20. What is the correct syntax to dynamically allocate memory for an array of integers in C++?

A. int arr[size];

B. int arr[] = new int[size];

C. int *arr = new int[size]; ✓

D. int arr = new int[size];

Answer: Option C — int *arr = new int[size];

Q21. What does the 'static' keyword mean when used with a global variable in C++?

A. Scope limited to the file ✓

B. Can't be modified

C. Retains its value between calls

D. Global scope

Answer: Option A — Scope limited to the file

Q22. Which of the following is used to access the first element of an array in C++?

A. arr.front()

B. arr[0] ✓

C. arr.begin()

D. None of the above

Answer: Option B — arr[0]

Q23. What is the output of the following code: cout << (5 == 5.0); in C++?

A. FALSE

B. TRUE

C. 1 ✓

D. 0

Answer: Option C — 1

Q24. What is the purpose of the 'break' statement in a switch-case construct in C++?

A. Skip the next case

B. Start the next iteration

C. Terminate the program

D. Exit the switch block ✓

Answer: Option D — Exit the switch block

Q25. What is the correct way to include a C++ standard library header file named 'vector' in a C++ program?

A. #include <vector> ✓

B. #include "vector.h"

C. #include <vector.h>

D. #include "vector"

Answer: Option A — #include <vector>

Q26. Which operator is used to access the member functions and variables of an object in C++?

A. :

B. ::

C. ->

D. . ✓

Answer: Option D — .

Q27. What is the output of the following code: cout << (10 / 4); in C++?

A. 2.5

B. 3

C. 2 ✓

D. Compiler error

Answer: Option C — 2

Q28. What is the correct syntax to define a function named 'add' that takes two integers as parameters and returns their sum in C++?

A. int add(int a, b) { }

B. int add(int a, int b) { } ✓

C. int add(a, b) { }

D. None of the above

Answer: Option B — int add(int a, int b) { }

Q29. Which of the following is not a valid data type in C++?

A. long double

B. short float ✓

C. long long int

D. wchar_t

Answer: Option B — short float

Q30. What does the 'new' operator return when memory allocation fails in C++?

A. Garbage value

B. Exception

C. nullptr ✓

D. 0

Answer: Option C — nullptr

Q31. Which of the following is the correct syntax for declaring a pointer to an integer in C++?

A. ptr int;

B. int ptr;

C. pointer int;

D. int* ptr; ✓

Answer: Option D — int* ptr;

Q32. What is the output of the following code: cout << (5 < 2) ? "True" : "False"; in C++?

A. FALSE ✓

B. TRUE

C. 5

D. 2

Answer: Option A — FALSE

Q33. In C++, what is the purpose of the 'endl' manipulator?

A. Inserts a new line ✓

B. Clears the screen

C. Moves the cursor to the start

D. Ends the program

Answer: Option A — Inserts a new line

Q34. What is the result of the expression: sizeof(char) in C++?

A. Implementation-defined

B. Error: sizeof cannot be applied

C. Size of char in bits

D. Size of char in bytes ✓

Answer: Option D — Size of char in bytes

Q35. What does the 'const' keyword signify when used with a member function in a C++ class declaration?

A. This function is static

B. This function can't be called

C. This function doesn't modify the object it's called on ✓

D. This function is pure virtual

Answer: Option C — This function doesn't modify the object it's called on

---

Q36. Which operator is used to dereference a pointer in C++?

A. ::

B. * ✓

C. .

D. :

Answer: Option B — *

Q37. What is the correct way to include the C++ standard input/output library header file in a C++ program?

A. #include "stdio"

B. #include <stdio.h>

C. #include <iostream> ✓

D. #include "iostream.h"

Answer: Option C — #include <iostream>

Q38. What is the output of the following code: int x = 5, y = 2; cout << x / y; in C++?

A. 2.5

B. 2.6

C. Compilation error

D. 2 ✓

Answer: Option D — 2

Q39. What is the correct syntax to define a class named 'Rectangle' with private member variables 'length' and 'width' in C++?

A. class Rectangle { private: int length; private: int width; };

B. class Rectangle { int length, width; private: };

C. class Rectangle { length, width; private: int; };

D. class Rectangle { private: int length, width; }; ✓

Answer: Option D — class Rectangle { private: int length, width; };

Q40. What is the output of the following code: cout << "Hello\nWorld"; in C++?

A. Hello World ✓

B. Hello

C. World

D. Hello\nWorld

Answer: Option A — Hello World

Q41. What is the purpose of the 'break' statement in a loop in C++?

A. Skip the next iteration

B. Start the next iteration

C. Exit the loop ✓

D. Terminate the program

Answer: Option C — Exit the loop

Q42. What is the correct way to define a constant named 'PI' with a value of 3.14159 in C++?

A. const PI = 3.14159;

B. const double PI = 3.14159; ✓

C. constant PI = 3.14159;

D. None of the above

Answer: Option B — const double PI = 3.14159;

Q43. What is the output of the following code: cout << (10 % 3); in C++?

A. Compilation error

B. 0

C. 1 ✓

D. 3

Answer: Option C — 1

Q44. What will be the output of the following program in both C and C++? #include<stdio.h> int main(int argc, char const *argv[]) { printf("%d\n", (int)sizeof('a')); return 0; }

A. Output in C is 1 and in C++ is 4

B. Output in C is 4 and in C++ is 1 ✓

C. Output in C is 1 and in C++ is 1

D. Output in C is 4 and in C++ is 4

Answer: Option B — Output in C is 4 and in C++ is 1

Q45. What happens if a class does not have a name?

A. It will not have a constructor

B. It will not have a destructor ✓

C. It is not allowed

D. It will neither have a constructor or destructor

Answer: Option B — It will not have a destructor

Q46. Which of the following escape sequence represents carriage return?

A. \r ✓

B. \n

C. \n\r

D. \c

Answer: Option A — \r

Q47. What are the formal parameters in C++?

A. Parameters with which functions are called

B. Parameters which are used in the definition of the function ✓

C. Variables other than passed parameters in a function

D. Variables that are never used in the function

Answer: Option B — Parameters which are used in the definition of the function

Q48. Which of the following is a static polymorphism mechanism?

A. Function overloading

B. Operator overloading

C. Templates

D. All of the mentioned ✓

Answer: Option D — All of the mentioned

Q49. What if we define the below structure in C and C++?

A. Error in C but not in C++ ✓

B. Error in C++ but not in C

C. No error in both C and C++

D. Error in both C and C++

Answer: Option A — Error in C but not in C++

Q50. Which of the following is true? I. All operators in C++ can be overloaded. II. The basic meaning of an operator can be changed.

A. I only

B. II only

C. Both I and II

D. Neither I nor II ✓

Answer: Option D — Neither I nor II

Q51. What is std in C++?

A. std is a standard class in C++

B. std is a standard namespace in C++ ✓

C. std is a standard header file in C++

D. std is a standard file reading header in C++

Answer: Option B — std is a standard namespace in C++

Q52. What is dynamic binding?

A. The process of linking the actual code with a procedural call during run-time ✓

B. The process of linking the actual code with a procedural call during compile-time

C. The process of linking the actual code with a procedural call at any-time

D. All of the mentioned

Answer: Option A — The process of linking the actual code with a procedural call during run-time

Q53. What happens if the following line is executed in C and C++? const int a;

A. Error in both C and C++

B. Warning in both C and C++

C. Error in C and successful execution in C++

D. Error in C++ and successful execution in C ✓

Answer: Option D — Error in C++ and successful execution in C

Q54. What happens if we run the following code in both C and C++? #include<stdio.h> struct STRUCT { int a = 5; int func() { printf("%d\n", a); } }; int main() { struct STRUCT s; s.func(); return 0; }

A. The program runs fine and both prints output "HELLO THIS IS STRUCTURE"

B. The program gives an error in case of C but runs perfectly in case of C++ ✓

C. The program gives an error in case of C++ but runs perfectly in case of C

D. The program gives an error in case of both C and C++

Answer: Option B — The program gives an error in case of C but runs perfectly in case of C++

Q55. What will be the output of the following C++ code? #include<iostream> using namespace std; class A { ~A(){ cout<<"Destructor called\n"; } }; int main() { A *a1 = new A(); A *a2 = new A(); return 0; }

A. Destructor called

B. Destructor called Destructor called

C. Error

D. Nothing is printed ✓

Answer: Option D — Nothing is printed

Q56. Which of the following is called extraction/get from operator?

A. <<

B. >> ✓

C. >

D. <

Answer: Option B — >>

Q57. Which of the following is called address operator?

A. *

B. & ✓

C. _

D. %

Answer: Option B — &

Q58. What is name mangling in C++?

A. The process of adding more information to a function name so that it can be distinguished from other functions by the compiler ✓

B. The process of making common names for all the function of C++ program for better use

C. The process of changing the names of variable

D. The process of declaring variables of different types

Answer: Option A — The process of adding more information to a function name so that it can be distinguished from other functions by the compiler

Q59. What is the size of a boolean variable in C++?

A. 1 bit ✓

B. 1 byte

C. 4 bytes

D. 2 bytes

Answer: Option A — 1 bit

Q60. What is the size of a character literal in C and C++?

A. 4 and 1 ✓

B. 1 and 4

C. 1 and 1

D. 4 and 4

Answer: Option A — 4 and 1

Q61. Which of the following is correct?

A. struct cannot have member function in C but it can in C++ ✓

B. struct cannot have member function in C++ but it can in C

C. struct cannot have member function in both C and C++

D. struct can have member function in both C and C++

Answer: Option A — struct cannot have member function in C but it can in C++

Q62. What will be the output of the following C++ code? #include<iostream> using namespace std; class A { ~A(){ cout<<"Destructor called\n"; } }; int main() { A a; return 0; }

A. Destructor called

B. Nothing will be printed

C. Error ✓

D. Segmentation fault

Answer: Option C — Error

Q63. Why this pointer is used?

A. To access the members of a class which have the same name as local variables in that scope ✓

B. To access all the data stored under that class

C. To access objects of other class

D. To access objects of other variables

Answer: Option A — To access the members of a class which have the same name as local variables in that scope

Q64. Which of the following is correct about static variables?

A. Static functions do not support polymorphism

B. Static data members cannot be accessed by non-static member functions

C. Static data members functions can access only static data members ✓

D. Static data members functions can access both static and non-static data members

Answer: Option C — Static data members functions can access only static data members

Q65. Which of the following syntax can be used to use a member of a namespace without including that namespace?

A. namespace::member ✓

B. namespace->member

C. namespace.member

D. namespace~member

Answer: Option A — namespace::member

Q66. Who created C++?

A. Bjarne Stroustrup ✓

B. Dennis Ritchie

C. Ken Thompson

D. Brian Kernighan

Answer: Option A — Bjarne Stroustrup

Q67. Which of the following is correct about static polymorphism?

A. In static polymorphism, the conflict between the function call is resolved during the compile time ✓

B. In static polymorphism, the conflict between the function call is resolved during the run time

C. In static polymorphism, the conflict between the function call is never resolved during the execution of a program

D. In static polymorphism, the conflict between the function call is resolved only if it required

Answer: Option A — In static polymorphism, the conflict between the function call is resolved during the compile time

Q68. Which of the following is the scope resolution operator?

A. .

B. *

C. :: ✓

D. ~

Answer: Option C — ::

Q69. What happens if the following program is executed in C and C++? #include <stdio.h> void main() { printf("Hello World"); }

A. Error in both C and C++

B. Successful run in both C and C++

C. Error in C and successful execution in C++

D. Error in C++ and successful execution in C ✓

Answer: Option D — Error in C++ and successful execution in C

Q70. Which of the following is an entry-controlled loop?

A. for

B. while

C. do-while

D. both while and for ✓

Answer: Option D — both while and for

Q71. What will be the output of the following C++ code? #include <iostream> int const s=9; int main() { std::cout << s; return 0; }

A. 9 ✓

B. Garbage value

C. Error

D. Segmentation fault

Answer: Option A — 9

Q72. Which concept is used to implement late binding?

A. Virtual functions ✓

B. Operator functions

C. Constant functions

D. Static functions

Answer: Option A — Virtual functions

Q73. What will be the output of the following C++ code? #include <iostream> int main(int argc, char const *argv[]) { cout<<"Hello World"; return 0; }

A. Hello World ✓

B. Compile-time error

C. Run-time error

D. Segmentation fault

Answer: Option A — Hello World

Q74. Which of the following cannot be used with the virtual keyword?

A. Class

B. Member functions

C. Constructors ✓

D. Destructors

Answer: Option C — Constructors

Q75. What happens if the following program is executed in C and C++? #include <stdio.h> int main(void) { int new = 5; printf("%d", new); }

A. Error in both C and C++

B. A successful run in both C and C++

C. Error in C and successful execution in C++

D. Error in C++ and successful execution in C ✓

Answer: Option D — Error in C++ and successful execution in C

Q76. Which of the following is an exit-controlled loop?

A. for

B. while

C. do-while ✓

D. all of the mentioned

Answer: Option C — do-while

Q77. What will be the output of the following C++ code? #include <iostream> using namespace std; class Test { static int x; public: Test() { x++; } static int getX() {return x;} }; int Test::x = 0; int main() { cout << Test::getX() << " "; Test t[5]; cout << Test::getX(); }

A. 0 0

B. 5 0

C. 0 5 ✓

D. 5 5

Answer: Option C — 0 5

Q78. What happens if the following program is executed in C and C++? #include<stdio.h> int main() { foo(); } int foo() { printf("Hello"); return 0; }

A. Error in both C and C++

B. Warning in both C and C++

C. Error in C++ but Warning in C ✓

D. Error in C but Warning in C++

Answer: Option C — Error in C++ but Warning in C

Q79. Which of the following is C++ equivalent for scanf()?

A. cin ✓

B. cout

C. print

D. input

Answer: Option A — cin

Q80. What will be the output of the following C++ code? #include<iostream> using namespace std; int x = 1; int main() { int x = 2; { int x = 3; cout << ::x << endl; } return 0; }

A. 1 ✓

B. 2

C. 3

D. 123

Answer: Option A — 1

Q81. What is static binding?

A. The process of linking the actual code with a procedural call during run-time

B. The process of linking the actual code with a procedural call during compile-time ✓

C. The process of linking the actual code with a procedural call at any-time

D. All of the mentioned

Answer: Option B — The process of linking the actual code with a procedural call during compile-time

Q82. What will be the output of the following C++ code? #include<iostream> using namespace std; int x[100]; int main() { cout << x[99] << endl; }

A. Garbage value

B. 0 ✓

C. 99

D. Error

Answer: Option B — 0

Q83. Which of the following is accessed by a member function of a class?

A. The object of that class

B. All members of a class ✓

C. The public part of a class

D. The private part of a class

Answer: Option B — All members of a class

Q84. Which of the following type is provided by C++ but not C?

A. int

B. bool ✓

C. float

D. double

Answer: Option B — bool

Q85. What will be the output of the following C++ code? #include <iostream> using namespace std; class A { private: int x; public: A(int _x) { x = _x; } int get() { return x; } }; class B { static A a; public: static int get() { return a.get(); } }; int main(void) { B b; cout << b.get(); return 0; }

A. Garbage value

B. Compile-time Error ✓

C. Run-time Error

D. Nothing is printed

Answer: Option B — Compile-time Error

Q86. What is the other name of compile-time polymorphism?

A. Static polymorphism ✓

B. Dynamic polymorphism

C. Executing polymorphism

D. Non-executing polymorphism

Answer: Option A — Static polymorphism

Q87. What is the size of a character type in C and C++?

A. 4 and 1

B. 1 and 4

C. 1 and 1 ✓

D. 4 and 4

Answer: Option C — 1 and 1

Q88. Which of the following is the correct difference between cin and scanf()?

A. both are the same

B. cin is a stream object whereas scanf() is a function ✓

C. scanf() is a stream object whereas cin is a function

D. cin is used for printing whereas scanf() is used for reading input

Answer: Option B — cin is a stream object whereas scanf() is a function

Q89. Which of the following is not a type of inheritance?

A. Multiple

B. Multilevel

C. Distributive ✓

D. Hierarchical

Answer: Option C — Distributive

Q90. Which of the following is used for comments in C++?

A. // comment

B. /* comment */

C. both // comment or /* comment */ ✓

D. // comment */

Answer: Option C — both // comment or /* comment */

Q91. Which of the following operator(s) can be used with pointers? i. – only ii. +, * iii. +, – iv. +, -, * v. / vi. + only

A. i only ✓

B. vi only

C. ii and v

D. iv

Answer: Option A — i only

Q92. Which of the following is correct about this pointer in C++?

A. this pointer is passed as a hidden argument in all the functions of a class

B. this pointer is passed as a hidden argument in all non-static functions of a class ✓

C. this pointer is passed as a hidden argument in all static functions of a class

D. this pointer is passed as a hidden argument in all static variables of a class

Answer: Option B — this pointer is passed as a hidden argument in all non-static functions of a class

Q93. Which of the following syntax for declaring a variable of struct STRUCT can be used in both C and C++?

A. struct STRUCT S; ✓

B. STRUCT S;

C. Both struct STRUCT S; and STRUCT S;

D. Both C and C++ have different syntax

Answer: Option A — struct STRUCT S;

Q94. What happens if the following program is executed in C and C++? #include <stdio.h> void func() { printf("Hello"); } void main() { func(); func(2); }

A. Error in both C and C++

B. Outputs Hello twice in both C and C++

C. Error in C and Outputs Hello twice in C++

D. Error in C++ and Outputs Hello twice in C ✓

Answer: Option D — Error in C++ and Outputs Hello twice in C

Q95. Which of the following is correct?

A. A class is an instance of its objects

B. An object is an instance of its class ✓

C. A class is an instance of the data type that the class have

D. An object is an instance of the data type of the class

Answer: Option B — An object is an instance of its class

Q96. Which of the following is correct in C++?

A. Classes cannot have protected data members

B. Structures can have member functions ✓

C. Class members are public by default

D. Structure members are private by default

Answer: Option B — Structures can have member functions

Q97. What will be the output of the following C++ code? #include <iostream> using namespace std; class Point { int x, y; public: Point(int i = 0, int j =0) { x = i; y = j; } int getX() const { return x; } int getY() {return y;} }; int main() { const Point t; cout << t.getX() << " "; cout << t.gety(); return 0; }

A. 0 0

B. Garbage values

C. Compile error ✓

D. Segmentation fault

Answer: Option C — Compile error

Q98. Which of the following escape sequence represents tab?

A. \t ✓

B. \t\r

C. \b

D. \a

Answer: Option A — \t

Q99. What is the other name of run-time polymorphism?

A. Static polymorphism

B. Dynamic polymorphism ✓

C. Executing polymorphism

D. Non-executing polymorphism

Answer: Option B — Dynamic polymorphism

Q100. Which function is used to write a single character to console in C++?

A. cout.put(ch) ✓

B. cout.putline(ch)

C. write(ch)

D. printf(ch)

Answer: Option A — cout.put(ch)

Variables And Data Types In C++

Q101. What is the correct keyword to declare a variable in C++?

A. var

B. variable

C. int ✓

D. declare

Answer: Option C — int

Q102. Which of the following is a valid variable name in C++?

A. myVariable ✓

B. 123variable

C. variable&

D. float

Answer: Option A — myVariable

Q103. What is the data type of the variable 'x' in the following declaration: double x = 3.14;?

A. int

B. double ✓

C. float

D. char

Answer: Option B — double

Q104. Which of the following is not a primitive data type in C++?

A. bool

B. char

C. string ✓

D. float

Answer: Option C — string

Q105. What is the maximum value that can be stored in an 'int' data type in C++?

A. 32767

B. 2147483647 ✓

C. 65535

D. 4294967295

Answer: Option B — 2147483647

Q106. What is the size of a 'char' data type in C++?

A. 2 bytes

B. 4 bytes

C. 1 byte ✓

D. 8 bytes

Answer: Option C — 1 byte

Q107. What is the result of the following expression in C++: 5 / 2?

A. Compiler error

B. 2.6

C. 2.5

D. 2 ✓

Answer: Option D — 2

Q108. Which operator is used to find the size of a data type in C++?

A. size

B. sizeofof

C. sizeof ✓

D. lengthof

Answer: Option C — sizeof

Q109. What is the correct way to declare a constant in C++?

A. final

B. const ✓

C. #define

D. None of the above

Answer: Option B — const

Q110. What will be the output of the following code: cout << (true && false); in C++?

A. TRUE

B. FALSE ✓

C. 1

D. 0

Answer: Option B — FALSE

Q111. What is the correct syntax to declare and initialize a variable 'age' of type 'int' with a value of 25 in C++?

A. age = int(25);

B. int = age 25;

C. int age = 25; ✓

D. age int = 25;

Answer: Option C — int age = 25;

Q112. What is the range of values for a 'short int' data type in C++?

A. -2147483648 to 2147483647

B. -128 to 127

C. 0 to 65535

D. -32768 to 32767 ✓

Answer: Option D — -32768 to 32767

Q113. What does the 'auto' keyword do in C++?

A. Declares a variable with an inferred type ✓

B. Declares a global variable

C. Declares an automatic variable

D. Declares a static variable

Answer: Option A — Declares a variable with an inferred type

Q114. Which of the following is an example of a floating-point data type in C++?

A. int

B. float ✓

C. char

D. bool

Answer: Option B — float

Q115. What is the correct way to represent hexadecimal values in C++?

A. Suffix with H

B. Prefix with H

C. Prefix with 0h

D. Prefix with 0x ✓

Answer: Option D — Prefix with 0x

Q116. What is the correct data type for storing a single character in C++?

A. int

B. float

C. char ✓

D. double

Answer: Option C — char

Q117. What will be the output of the following code: cout << sizeof(char); in C++?

A. 4

B. 1 ✓

C. Compiler error

D. 2

Answer: Option B — 1

Q118. Which of the following is a valid way to initialize a variable in C++?

A. int x(5);

B. x = 5;

C. int x = 5; ✓

D. int x; x = 5;

Answer: Option C — int x = 5;

Q119. What is the maximum value that can be stored in an 'unsigned int' data type in C++?

A. 32767

B. 2147483647

C. 65535

D. 4294967295 ✓

Answer: Option D — 4294967295

Q120. Which of the following is the correct way to declare a constant 'PI' with a value of 3.14159 in C++?

A. #define PI 3.14159;

B. constant PI = 3.14159;

C. double PI = 3.14159;

D. const double PI = 3.14159; ✓

Answer: Option D — const double PI = 3.14159;

Q121. What will be the output of the following code: cout << (5 != 2); in C++?

A. 5

B. 2

C. 1 ✓

D. Compiler error

Answer: Option C — 1

Q122. What is the size of a 'long double' data type in C++?

A. 2 bytes

B. 4 bytes

C. 8 bytes ✓

D. 16 bytes

Answer: Option C — 8 bytes

Q123. Which of the following is not a valid variable name in C++?

A. _variableName

B. variableName@

C. myVariable123

D. 123variable ✓

Answer: Option D — 123variable

Q124. What is the correct way to represent octal values in C++?

A. Prefix with 0o

B. Prefix with 0x

C. Prefix with 0 ✓

D. Suffix with O

Answer: Option C — Prefix with 0

Q125. What does the 'volatile' keyword do in C++?

A. Indicates that a variable may be changed unexpectedly ✓

B. Indicates that a variable cannot be changed

C. Declares a constant variable

D. Declares a global variable

Answer: Option A — Indicates that a variable may be changed unexpectedly

---

Q126. What is the result of the following expression in C++: 7 % 3?

A. Compiler error

B. 3

C. 2

D. 1 ✓

Answer: Option D — 1

Q127. What is the correct way to declare a pointer to an integer in C++?

A. ptr int;

B. pointer int;

C. int *ptr; ✓

D. int ptr;

Answer: Option C — int *ptr;

Q128. What is the data type of the expression: 5.2f in C++?

A. long double

B. float ✓

C. int

D. None of the above

Answer: Option B — float

Q129. What is the output of the following code: `cout << (true || false);` in C++?

A. 0

B. 1

C. TRUE ✓

D. FALSE

Answer: Option C — TRUE

Q130. What is the range of values for a 'signed char' data type in C++?

A. -128 to 127 ✓

B. 0 to 255

C. -32768 to 32767

D. -2147483648 to 2147483647

Answer: Option A — -128 to 127

Q131. What is the size of the 'bool' data type in C++?

A. 2 bytes

B. 4 bytes

C. Compiler-dependent

D. 1 byte ✓

Answer: Option D — 1 byte

Q132. Which of the following is not a valid data type in C++?

A. long float

B. unsigned long long int

C. wchar_t

D. byte ✓

Answer: Option D — byte

Q133. What will be the result of the expression: 10 / 3 in C++?

A. 3 ✓

B. 3.3333

C. 3.34

D. Compiler error

Answer: Option A — 3

Q134. What is the correct way to declare a constant variable named 'PI' with a value of 3.14159 in C++?

A. PI = 3.14159;

B. constant PI = 3.14159;

C. #define PI 3.14159

D. const double PI = 3.14159; ✓

Answer: Option D — const double PI = 3.14159;

Q135. Which keyword is used to define a constant in C++?

A. constant

B. static const

C. const ✓

D. define

Answer: Option C — const

Q136. What is the data type of the expression: 3.14f in C++?

A. long double

B. float ✓

C. int

D. None of the above

Answer: Option B — float

Q137. What is the maximum value that can be stored in an 'unsigned short int' data type in C++?

A. 2147483647

B. 4294967295

C. 65535 ✓

D. 32767

Answer: Option C — 65535

Q138. What is the output of the following code: cout << (7 >= 7); in C++?

A. TRUE ✓

B. FALSE

C. 1

D. 0

Answer: Option A — TRUE

Q139. What is the data type of the expression: sizeof('A') in C++?

A. long

B. double

C. char

D. int ✓

Answer: Option D — int

Q140. What is the result of the expression: 5.0 + 3 in C++?

A. 3.5

B. 5.3

C. 8 ✓

D. Compiler error

Answer: Option C — 8

Q141. What is the range of values for an 'unsigned long int' data type in C++?

A. 0 to 65535

B. -32768 to 32767

C. 0 to 4294967295 ✓

D. -2147483648 to 2147483647

Answer: Option C — 0 to 4294967295

Q142. What is the correct way to declare a pointer to a character in C++?

A. pointer char;

B. char *ptr; ✓

C. char ptr;

D. None of the above

Answer: Option B — char *ptr;

Q143. What does the 'volatile' keyword indicate in C++?

A. The variable is thread-safe

B. The variable is not accessible

C. The value of the variable may change at any time by external factors ✓

D. The value of the variable remains constant

Answer: Option C — The value of the variable may change at any time by external factors

Q144. What is the output of the following code: cout << (true && true); in C++?

A. FALSE

B. 1

C. 0

D. TRUE ✓

Answer: Option D — TRUE

Q145. Which type is best suited to represent the logical values?

A. integer

B. boolean ✓

C. character

D. float

Answer: Option B — boolean

Q146. What will be the output of the following C++ function? int main() { register int i = 1; int *ptr = &i; cout << *ptr; return 0; }

A. 0

B. 1

C. Compiler error may be possible ✓

D. Runtime error may be possible

Answer: Option C — Compiler error may be possible

Q147. What is the value of the bool? bool is_int(789.54)

A. True

B. False ✓

C. 1

D. 2

Answer: Option B — False

Q148. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { char c = 74; cout << c; return 0; }

A. A

B. N

C. J ✓

D. I

Answer: Option C — J

Q149. Evaluate the following. (false && true) || false || true

A. 0

B. 1 ✓

C. false

D. 2

Answer: Option B — 1

Q150. The size_t integer type in C++ is?

A. Unsigned integer of at least 64 bits

B. Signed integer of at least 16 bits

C. Unsigned integer of at least 16 bits ✓

D. Signed integer of at least 64 bits

Answer: Option C — Unsigned integer of at least 16 bits

Q151. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int a = 10; if (a < 10) { for (i = 0; i < 10; i++) cout << i; } else { cout << i; } return 0; }

A. 0123456789

B. 123456789

C. 0

D. error ✓

Answer: Option D — error

Q152. What happens when a null pointer is converted into bool?

A. an error is flagged

B. bool value evaluates to true

C. bool value evaluates to false ✓

D. the statement is ignored

Answer: Option C — bool value evaluates to false

Q153. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { float i = 123.0f; cout << i << endl; return 0; }

A. 123.00

B. 1.23

C. 123 ✓

D. compile time error

Answer: Option C — 123

Q154. Pick the odd one out.

A. array type ✓

B. character type

C. boolean type

D. integer type

Answer: Option A — array type

Q155. How many characters are specified in the ASCII scheme?

A. 64

B. 128 ✓

C. 256

D. 24

Answer: Option B — 128

Q156. Choose the correct option. extern int i; int i;

A. both 1 and 2 declare i

B. 1 declares the variable i and 2 defines i

C. 1 declares and defines i, 2 declares i

D. 1 declares i,2 declares and defines i ✓

Answer: Option D — 1 declares i,2 declares and defines i

Q157. Which is used to indicate single precision value?

A. F or f ✓

B. L or l

C. Either F or for L or l

D. Neither F or for L or l

Answer: Option A — F or f

Q158. 0946, 786427373824, 'x' and 0X2f are . . . . . and . . . . . literals respectively.

A. decimal, character, octal, hexadecimal

B. octal, hexadecimal, character, decimal

C. hexadecimal, octal, decimal, character

D. octal, decimal, character, hexadecimal ✓

Answer: Option D — octal, decimal, character, hexadecimal

Q159. In which type do the enumerators are stored by the compiler?

A. string

B. integer ✓

C. float

D. string & float

Answer: Option B — integer

Q160. The value 132.54 can be represented using which data type?

A. double ✓

B. void

C. int

D. bool

Answer: Option A — double

Q161. What is the size of wchar_t in C++?

A. 2

B. 4

C. 2 or 4

D. Based on the number of bits in the system ✓

Answer: Option D — Based on the number of bits in the system

Q162. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int x = -1; unsigned int y = 2; if(x > y) { cout << "x is greater"; } else { cout << "y is greater"; } }

A. x is greater ✓

B. y is greater

C. implementation defined

D. arbitrary

Answer: Option A — x is greater

Q163. What will be the output of the following C++ code? #include <iostream> using namespace std; void addprint() { static int s = 1; s++; cout << s; } int main() { addprint(); addprint(); addprint(); return 0; }

A. 234 ✓

B. 111

C. 123

D. 235

Answer: Option A — 234

Q164. What is the value of p in the following C++ code? #include <iostream> using namespace std; int main() { int p; bool a = true; bool b = false; int x = 10; int y = 5; p = ((x | y) + (a + b)); cout << p; return 0; }

A. 0

B. 16 ✓

C. 12

D. 2

Answer: Option B — 16

Q165. What will be the output of the following C++ code? #include <iostream> using namespace std; int f(int p, int q) { if (p > q) return p; else return q; } main() { int a = 5, b = 10; int k; bool x = true; bool y = f(a, b); k =((a * b) + (x + y)); cout << k; }

A. 55

B. 62

C. 52 ✓

D. 75

Answer: Option C — 52

Q166. Which of these expressions will return true if the input integer v is a power of two?

A. (v | (v + 1)) == 0;

B. (~v & (v – 1)) == 0;

C. (v | (v – 1)) == 0;

D. (v & (v – 1)) == 0; ✓

Answer: Option D — (v & (v – 1)) == 0;

Q167. What constant defined in <climits> header returns the number of bits in a char?

A. CHAR_SIZE

B. SIZE_CHAR

C. BIT_CHAR

D. CHAR_BIT ✓

Answer: Option D — CHAR_BIT

Q168. What will be the output of the following C++ code? #include <stdio.h> int main() { char a = '\012'; printf("%d", a); return 0; }

A. Compiler error

B. 12

C. 10 ✓

D. Empty

Answer: Option C — 10

Q169. What will happen when defining the enumerated type?

A. it will not allocate memory ✓

B. it will allocate memory

C. it will not allocate memory to its variables

D. allocate memory to objects

Answer: Option A — it will not allocate memory

Q170. When a language has the capability to produce new data type mean, it can be called as

A. overloaded

B. extensible ✓

C. encapsulated

D. reprehensible

Answer: Option B — extensible

Q171. Which of the given statements are false i. extern int func; ii. extern int func2(int,int); iii. int func2(int,int); iv. extern class foo;

A. iii and iv only

B. ii and iii only

C. only iv ✓

D. ii, iii and iv

Answer: Option C — only iv

Q172. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int i = 3; int l = i / -2; int k = i % -2; cout << l << k; return 0; }

A. compile time error

B. -1 1 ✓

C. 1 -1

D. implementation defined

Answer: Option B — -1 1

Q173. What does the following statement mean? void a;

A. variable a is of type void

B. a is an object of type void

C. declares a variable with value a

D. flags an error ✓

Answer: Option D — flags an error

Q174. What is the value of the following 8-bit integer after all statements are executed? int x = 1; x = x << 7; x = x >> 7;

A. 1

B. -1

C. 127

D. Implementation defined ✓

Answer: Option D — Implementation defined

Q175. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int num = 0x20 + 020 + 20; cout << sizeof(num)<<'\n'; return 0; }

A. 2

B. 4

C. Depends on compiler ✓

D. Garbage

Answer: Option C — Depends on compiler

Q176. Which of the following will not return a value?

A. null

B. void ✓

C. empty

D. free

Answer: Option B — void

Q177. Suppose in a hypothetical machine, the size of char is 32 bits. What would sizeof(char) return?

A. 4

B. 1 ✓

C. Implementation dependent

D. Machine dependent

Answer: Option B — 1

Q178. Is bool a fundamental data type in C++?

A. Yes ✓

B. No, it is a typedef of unsigned char

C. No, it is an enum of {false, true}

D. No, it is expanded from macros

Answer: Option A — Yes

Q179. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int a = 8; cout << "ANDing integer 'a' with 'true' :" << (a && true); return 0; }

A. ANDing integer 'a' with 'true' :8 ✓

B. ANDing integer 'a' with 'true' :0

C. ANDing integer 'a' with 'true' :1

D. ANDing integer 'a' with 'true' :9

Answer: Option A — ANDing integer 'a' with 'true' :8

Q180. Which data type is used to represent the absence of parameters?

A. int

B. short

C. void ✓

D. float

Answer: Option C — void

Q181. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { enum channel {star, sony, zee}; enum symbol {hash, star}; int i = 0; for (i = star; i <= zee; i++) { printf("%d ", i); } return 0; }

A. 012

B. 123

C. compile time error ✓

D. runtime error

Answer: Option C — compile time error

Q182. What does '\a' escape code represent?

A. alert ✓

B. backslash

C. tab

D. form feed

Answer: Option A — alert

Q183. What will be the output of the following C++ code? #include <iomanip> #include <iostream> using namespace std; int main() { cout << setprecision(17); double d = 0.1; cout << d << endl; return 0; }

A. 0.11

B. 0.10000000000000001 ✓

C. 0.100001

D. compile time error

Answer: Option B — 0.10000000000000001

Q184. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { float num1 = 1.1; double num2 = 1.1; if (num1 == num2) cout << "stanford"; else cout << "harvard"; return 0; }

A. harvard ✓

B. stanford

C. compile time error

D. runtime error

Answer: Option A — harvard

Q185. How do we represent a wide character of the form wchar_t?

A. L'a' ✓

B. l'a'

C. L[a]

D. la

Answer: Option A — L'a'

Q186. Which of the following statements are true? int f(float)

A. f is a function taking an argument of type int and returning a floating point number

B. f is a function taking an argument of type float and returning an integer ✓

C. f is a function of type float

D. f is a function of type int

Answer: Option B — f is a function taking an argument of type float and returning an integer

Q187. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int a = 5; float b; cout << sizeof(++a + b); cout << a; return 0; }

A. 2 6

B. 4 6

C. 2 5

D. 4 5 ✓

Answer: Option D — 4 5

Q188. Which of the following statements are false?

A. bool can have two values and can be used to express logical expressions

B. bool cannot be used as the type of the result of the function ✓

C. bool can be converted into integers implicitly

D. a bool value can be used in arithmetic expressions

Answer: Option B — bool cannot be used as the type of the result of the function

Q189. In C++, what is the sign of character data type by default?

A. Signed

B. Unsigned

C. Implementation dependent ✓

D. Unsigned Implementation

Answer: Option C — Implementation dependent

Q190. Which of these expressions will isolate the rightmost set bit?

A. x = x & (~x)

B. x = x ^ (~x)

C. x = x & (-x) ✓

D. x = x ^ (-x)

Answer: Option C — x = x & (-x)

Q191. What will be the output of the following C++ code? #include <iostream> using namespace std; int main ( ) { static double i; i = 20; cout << sizeof(i); return 0; }

A. 4

B. 2

C. 8 ✓

D. garbage

Answer: Option C — 8

Q192. Identify the incorrect option.

A. 1 <= sizeof(bool) <= sizeof(long)

B. sizeof(float) <= sizeof(double) <= sizeof(long double)

C. sizeof(char) <= sizeof(long) <=sizeof(wchar_t) ✓

D. sizeof(N) = sizeof(signed N) = sizeof(unsigned N)

Answer: Option C — sizeof(char) <= sizeof(long) <=sizeof(wchar_t)

Q193. Choose the incorrect option.

A. void is used when the function does not return a value

B. void is also used when the value of a pointer is null ✓

C. void is used as the base type for pointers to objects of unknown type

D. void is a special fundamental type

Answer: Option B — void is also used when the value of a pointer is null

Q194. Find the odd one out.

A. std::vector<int>

B. std::vector<short>

C. std::vector<long>

D. std::vector<bool> ✓

Answer: Option D — std::vector<bool>

Q195. Which variable does equals in size with enum variable?

A. int variable ✓

B. float variable

C. string variable

D. float & string variable

Answer: Option A — int variable

Q196. What will be the output of the following C++ code? #include <iostream> using namespace std; enum colour { green, red, blue, white, yellow, pink }; int main() { cout << green<< red<< blue<< white<< yellow<< pink; return 0; }

A. 012345 ✓

B. 123456

C. compile time error

D. runtime error

Answer: Option A — 012345

Q197. Which of the two operators ++ and - work for the bool data type in C++?

A. None

B. ++ ✓

C. -

D. ++ & -

Answer: Option B — ++

Q198. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int i; enum month { JAN = 1, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC }; for (i = MAR; i <= NOV; i++) cout << i; return 0; }

A. 01234567891011

B. 123456789101112

C. 34567891011 ✓

D. 123456789

Answer: Option C — 34567891011

Q199. What is the range of the floating point numbers?

A. -3.4E+38 to +3.4E+38 ✓

B. -3.4E+38 to +3.4E+34

C. -3.4E+38 to +3.4E+36

D. -3.4E+38 to +3.4E+32

Answer: Option A — -3.4E+38 to +3.4E+38

Q200. Which of the following belongs to the set of character types?

A. char

B. wchar_t

C. only a

D. both wchar_t and char ✓

Answer: Option D — both wchar_t and char

Operators And Expressions In C++

Q201. What will be the result of the expression: 5 + 3 * 2 in C++?

A. 16

B. 11 ✓

C. 10

D. 2

Answer: Option B — 11

Q202. What is the operator precedence of the multiplication operator (*) in C++?

A. Highest ✓

B. Lowest

C. Equal to addition

D. Equal to subtraction

Answer: Option A — Highest

Q203. What will be the output of the following code: cout << (5 == 5); in C++?

A. TRUE ✓

B. FALSE

C. 1

D. 0

Answer: Option A — TRUE

Q204. What is the result of the expression: 10 / 3 in C++?

A. 3 ✓

B. 3.333

C. 3.34

D. Compiler error

Answer: Option A — 3

Q205. Which of the following operators is used to access the address of a variable in C++?

A. & ✓

B. *

C. -

D. +

Answer: Option A — &

Q206. What will be the value of 'x' after executing the code: int x = 5; x += 3; in C++?

A. 8 ✓

B. 5

C. 15

D. Compiler error

Answer: Option A — 8

Q207. Which of the following is a unary operator in C++?

A. +

B. - ✓

C. *

D. /

Answer: Option B — -

Q208. What does the modulus operator (%) do in C++?

A. Returns the remainder ✓

B. Returns the quotient

C. Returns the sum

D. Returns the product

Answer: Option A — Returns the remainder

Q209. What is the output of the following code: cout << (10 != 10); in C++?

A. TRUE

B. FALSE ✓

C. 1

D. 0

Answer: Option B — FALSE

Q210. Which of the following is not a relational operator in C++?

A. ==

B. !=

C. % ✓

D. <=

Answer: Option C — %

Q211. What will be the value of 'x' after executing the code: int x = 10; x--; in C++?

A. 10

B. 9 ✓

C. 11

D. Compiler error

Answer: Option B — 9

Q212. What is the result of the expression: 2 * 3 + 5 / 2 in C++?

A. 8 ✓

B. 8.5

C. 9

D. Compiler error

Answer: Option A — 8

Q213. What is the output of the following code: cout << (true && false); in C++?

A. TRUE

B. FALSE ✓

C. 1

D. 0

Answer: Option B — FALSE

Q214. What is the operator associativity of the assignment operator (=) in C++?

A. Right to left

B. Left to right ✓

C. Equal to addition

D. Equal to subtraction

Answer: Option B — Left to right

Q215. What is the result of the expression: 5 + 3 * 2 in C++?

A. 16 ✓
B. 11
C. 10
D. Compiler error
Answer: Option A — 16

Q216. What is the purpose of the modulo operator (%) in C++?

A. Division remainder ✓
B. Multiplication
C. Exponentiation
D. Bitwise AND
Answer: Option A — Division remainder

Q217. What will be the output of the following code: cout << (5 == 5.0); in C++?

A. TRUE ✓
B. FALSE
C. 1
D. 0
Answer: Option A — TRUE

Q218. Which operator is used to access the memory address of a variable in C++?

A. *
B. & ✓
C. ->
D. ::
Answer: Option B — &

Q219. What is the result of the expression: 3 * (4 + 2) in C++?

A. 18 ✓
B. 20
C. 12
D. Compiler error
Answer: Option A — 18

Q220. What does the '++' operator do when used as a postfix operator?

A. Increments after use ✓
B. Increments before use
C. Decrements after use
D. Decrements before use
Answer: Option A — Increments after use

Q221. What will be the value of 'x' after executing the following code: int x = 5; x += 3; in C++?

A. 5
B. 8 ✓
C. 3
D. Compiler error

Answer: Option B — 8

Q222. What is the purpose of the ternary operator (?:) in C++?

A. Conditional operator ✓

B. Arithmetic operator

C. Bitwise operator

D. Relational operator

Answer: Option A — Conditional operator

Q223. What will be the output of the following code: cout << (4 > 3 ? "Yes" : "No"); in C++?

A. Yes ✓

B. No

C. 4

D. 3

Answer: Option A — Yes

Q224. What is the result of the expression: 10 % 3 in C++?

A. 1 ✓

B. 3

C. 0

D. Compiler error

Answer: Option A — 1

Q225. What does the 'sizeof' operator return in C++?

A. Size in bytes ✓

B. Size in bits

C. Memory address

D. Value of the variable

Answer: Option A — Size in bytes

Q226. What is the output of the following code: cout << (5 != 5); in C++?

A. TRUE

B. FALSE

C. 1

D. 0 ✓

Answer: Option D — 0

Q227. Which operator is having the highest precedence?

A. postfix ✓

B. unary

C. shift

D. equality

Answer: Option A — postfix

Q228. What will be the output of the following C++ code? #include <iostream> using namespace std; int main () { int x, y; x = 5; y = ++x * ++x; cout << x << y; x = 5; y = x++ * ++x; cout << x << y; return 0; }

A. 749735 ✓

B. 736749

C. 367497

D. 367597

Answer: Option A — 749735

Q229. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int a = 5, b = 6, c, d; c = a, b; d = (a, b); cout << c << ' ' << d; return 0; }

A. 5 6 ✓

B. 6 5

C. 6 7

D. 6 8

Answer: Option A — 5 6

Q230. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int a = 5, b = 6, c; c = (a > b) ? a : b; cout << c; return 0; }

A. 6 ✓

B. 5

C. 4

D. 7

Answer: Option A — 6

Q231. What is this operator called ?:?

A. conditional ✓

B. relational

C. casting operator

D. unrelational

Answer: Option A — conditional

Q232. Which operator is having the right to left associativity in the following?

A. Array subscripting

B. Function call

C. Addition and subtraction

D. Type cast ✓

Answer: Option D — Type cast

Q233. What is the use of dynamic_cast operator?

A. it converts virtual base class to derived class ✓

B. it converts the virtual base object to derived objects

C. it will convert the operator based on precedence

D. it converts the virtual base object to derived class

Answer: Option A — it converts virtual base class to derived class

Q234. What will be the output of the following C++ code? #include <iostream> using namespace std; main() { double a = 21.09399; float b = 10.20; int c ,d; c = (int) a; d = (int) b; cout << c <<' '<< d; return 0; }

A. 20 10

B. 10 21

C. 21 10 ✓

D. 10 20

Answer: Option C — 21 10

Q235. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int i, j; j = 10; i = (j++, j + 100, 999 + j); cout << i; return 0; }

A. 1000

B. 11

C. 1010 ✓

D. 1001

Answer: Option C — 1010

Q236. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int a; a = 5 + 3 * 5; cout << a; return 0; }

A. 35

B. 20 ✓

C. 25

D. 30

Answer: Option B — 20

Control Flow Statements In C++

Q237. What is the syntax for the 'if' statement in C++?

A. if (condition) then statement;

B. if (condition) then { statement; }

C. if { condition } then { statement; }

D. if (condition) { statement; } ✓

Answer: Option D — if (condition) { statement; }

Q238. What is the purpose of the 'else' statement in C++?

A. Executes after the 'if' block

B. Executes before the 'if' block

C. Executes when the 'if' condition is false ✓

D. Executes regardless of the 'if' block

Answer: Option C — Executes when the 'if' condition is false

Q239. What is the correct syntax for the 'switch' statement in C++?

A. switch (expression) { case constant1: statement1; case constant2: statement2; }

B. switch (expression) { case constant1: statement1; break; case constant2: statement2; break; default: defaultStatement; } ✓

C. switch (expression) { case constant1: statement1; default: defaultStatement; }

D. None of the above

Answer: Option B — switch (expression) { case constant1: statement1; break; case constant2: statement2; break; default: defaultStatement; }

Q240. What will be the output of the following code: int x = 10; if (x > 5) cout << "Hello"; else cout << "Goodbye"; in C++?

A. Compilation error

B. HelloGoodbye

C. Hello ✓

D. Goodbye

Answer: Option C — Hello

Q241. What does the 'break' statement do in a loop in C++?

A. Skips the current iteration

B. Skips the next iteration

C. Starts the next iteration

D. Terminates the loop ✓

Answer: Option D — Terminates the loop

Q242. What is the correct syntax for the 'while' loop in C++?

A. while (condition) { statement; } ✓

B. while (condition) then { statement; }

C. while { condition } { statement; }

D. while { statement; } condition;

Answer: Option A — while (condition) { statement; }

Q243. What will be the output of the following code: int i = 0; while (i < 5) { cout << i << " "; i++; } in C++?

A. Compilation error

B. 1 2 3 4

C. 0 1 2 3

D. 0 1 2 3 4 ✓

Answer: Option D — 0 1 2 3 4

Q244. What is the purpose of the 'default' case in a 'switch' statement in C++?

A. Executes before any other cases

B. Executes after all other cases

C. Executes when no case matches ✓

D. Executes instead of the 'case' cases

Answer: Option C — Executes when no case matches

Q245. What is the syntax for the 'do-while' loop in C++?

A. do { condition; } while (statement);

B. do { statement; } while (condition); ✓

C. do (condition) { statement; } while;

D. None of the above

Answer: Option B — do { statement; } while (condition);

Q246. What will be the output of the following code: for (int i = 5; i >= 0; --i) { cout << i << " "; if (i == 3) break; } in C++?

A. Compilation error

B. 5 4 3 ✓

C. 5 4 3 2 1 0

D. 5 4 3 0 1

Answer: Option B — 5 4 3

Q247. In C++, what is the difference between the 'while' loop and the 'do-while' loop?

A. 'while' loop always executes at least once, 'do-while' loop may not

B. 'while' loop always executes at least once, 'do-while' loop always does

C. 'while' loop checks condition before execution, 'do-while' loop checks after execution ✓

D. None of the above

Answer: Option C — 'while' loop checks condition before execution, 'do-while' loop checks after execution

Q248. What is the purpose of the 'if-else if-else' statement in C++?

A. Provides multiple conditions to be tested ✓

B. Executes multiple statements

C. Executes only one statement

D. Terminates the program

Answer: Option A — Provides multiple conditions to be tested

Q249. What will be the output of the following code: for (int i = 0; i < 3; ++i) { for (int j = 0; j < 2; ++j) { cout << i << j << " "; } }?

A. Compilation error

B. 00 10 20 01 11

C. 00 11 22 33 44

D. 00 01 10 11 20 21 ✓

Answer: Option D — 00 01 10 11 20 21

Q250. What is the syntax for the 'if-else' statement in C++?

A. if (condition) then { statement; } else { statement; }

B. if { condition } { statement; } else { statement; }

C. if (condition) { statement; } else { statement; } ✓

D. if (condition) { statement; } else statement;

Answer: Option C — if (condition) { statement; } else { statement; }

Q251. What does the 'else if' statement allow in C++?

A. Skips to the next iteration

B. Additional condition checks ✓

C. Terminate the program

D. None of the above

Answer: Option B — Additional condition checks

Q252. What is the output of the following code: int x = 10; if (x == 10) { cout << "x is 10"; } else { cout << "x is not 10"; } in C++?

A. No output

B. Compilation error

C. x is 10 ✓

D. x is not 10

Answer: Option C — x is 10

Q253. What is the purpose of the 'continue' statement in a loop in C++?

A. Terminates the loop

B. Skips the remaining code in the loop

C. Restarts the loop

D. Skips the current iteration ✓

Answer: Option D — Skips the current iteration

Q254. What will be the output of the following code: int i = 0; while (i < 3) { cout << i << " "; i++; } in C++?

A. Compilation error

B. 1 2 3

C. 0 1 2 3

D. 0 1 2 ✓

Answer: Option D — 0 1 2

Q255. What is the purpose of the 'break' statement in a loop in C++?

A. Skips the next iteration

B. Skips to the next condition

C. Terminates the loop ✓

D. Restarts the loop

Answer: Option C — Terminates the loop

Q256. What will be the output of the following code: for (int i = 0; i < 5; ++i) { if (i == 2) continue; cout << i << " "; } in C++?

A. 0 1 2 3 4

B. 0 1 3 4 ✓

C. Compilation error

D. 1 2 3 4 5

Answer: Option B — 0 1 3 4

Q257. What is the output of the following code: int x = 5; switch (x) { case 1: cout << "One"; break; case 5: cout << "Five"; break; } in C++?

A. One

B. Compilation error

C. No output

D. Five ✓

Answer: Option D — Five

Q258. What is the syntax for the 'for' loop in C++?

A. for (statement; initialization; condition; update)

B. for { initialization; condition; update } { statement; }

C. for (condition; update; initialization) { statement; }

D. for (initialization; condition; update) { statement; } ✓

Answer: Option D — for (initialization; condition; update) { statement; }

Q259. What is the output of the following code: for (int i = 0; i < 3; ++i) { for (int j = 0; j < 2; ++j) { cout << i << j << " "; } } in C++?

A. 00 11 22 33 21 01

B. 00 10 20 01 11 22

C. 00 01 10 11 20 21 ✓

D. Compilation error

Answer: Option C — 00 01 10 11 20 21

Q260. What does the 'do' keyword represent in the 'do-while' loop syntax in C++?

A. Represents the loop initialization

B. Represents the loop body ✓

C. Represents the loop incrementation

D. None of the above

Answer: Option B — Represents the loop body

Q261. What is the syntax for the 'if' statement with multiple conditions in C++?

A. if { condition1 } && { condition2 } { statement; }

B. if { condition1 && condition2 } { statement; }

C. if (condition1 && condition2) { statement; } ✓

D. if (condition1) && (condition2) { statement; }

Answer: Option C — if (condition1 && condition2) { statement; }

Q262. What will be the output of the following code: int x = 5; x > 2 ? cout << "Yes" : cout << "No"; in C++?

A. No

B. 5

C. 2

D. Yes ✓

Answer: Option D — Yes

Q263. In C++, what is the purpose of the 'do-while' loop?

A. Terminates the loop

B. Executes a block of code only once

C. Executes a block of code repeatedly until a condition becomes false ✓

D. Executes a block of code repeatedly until a condition becomes true

Answer: Option C — Executes a block of code repeatedly until a condition becomes false

Q264. What will be the output of the following code: for (int i = 0; i < 5; ++i) { if (i == 3) continue; cout << i << " "; } in C++?

A. 0 1 2 3 4

B. 0 1 2 3

C. Compilation error

D. 0 1 2 4 ✓

Answer: Option D — 0 1 2 4

Q265. What is the syntax for the 'else if' statement in C++?

A. else if (condition) { statement; } ✓

B. if else (condition) { statement; }

C. elseif (condition) { statement; }

D. else (condition) { statement; }

Answer: Option A — else if (condition) { statement; }

Q266. What does the 'default' case represent in a 'switch' statement in C++?

A. Terminates the switch block

B. Executes only when all other cases match

C. Executes after every other case

D. Executes when no other case matches ✓

Answer: Option D — Executes when no other case matches

Q267. What is the correct syntax for the 'do-while' loop in C++?

A. while { condition } { statement; }

B. do { condition; } while (statement);

C. do { statement; } while (condition); ✓

D. do (condition) { statement; } while;

Answer: Option C — do { statement; } while (condition);

Q268. What will be the output of the following code: int i = 0; while (i < 5) { cout << i << " "; ++i; } in C++?

A. Compilation error

B. 1 2 3 4

C. 0 1 2 3 4 ✓

D. 0 1 2 3

Answer: Option C — 0 1 2 3 4

Q269. What is the syntax for the 'switch' statement in C++?

A. switch (expression) { case constant1: statement1; case constant2: statement2; default: defaultStatement; }

B. switch (expression) { case constant1: statement1; case constant2: statement2; }

C. switch (expression) { case constant1: statement1; default: defaultStatement; }

D. switch (expression) { case constant1: statement1; break; case constant2: statement2; break; default: defaultStatement; } ✓

Answer: Option D — switch (expression) { case constant1: statement1; break; case constant2: statement2; break; default: defaultStatement; }

Q270. In C++, what is the purpose of the 'if-else' statement?

A. Allows execution of different blocks of code based on a condition ✓

B. Terminates the program

C. Executes the same block of code repeatedly

D. Skips the current iteration in a loop

Answer: Option A — Allows execution of different blocks of code based on a condition

---

Q271. What will be the output of the following code: int i = 5; if (i != 5) cout << "Not equal"; else cout << "Equal"; in C++?

A. Compilation error

B. 5

C. Not equal

D. Equal ✓

Answer: Option D — Equal

Q272. What is the purpose of the 'if' statement in C++?

A. Executes a block of code repeatedly until a condition becomes true

B. Executes a block of code only once

C. Executes a block of code if a condition is true ✓

D. Terminates the program

Answer: Option C — Executes a block of code if a condition is true

Q273. What is the output of the following code: int i = 0; do { cout << i << " "; i++; } while (i < 3); in C++?

A. 0 1 2 3

B. 0 1 2 ✓

C. Compilation error

D. 1 2 3

Answer: Option B — 0 1 2

Q274. What does the 'continue' statement do in a loop in C++?

A. Restarts the loop

B. Skips the remaining code in the loop

C. Terminates the loop

D. Skips the current iteration and continues with the next iteration ✓

Answer: Option D — Skips the current iteration and continues with the next iteration

Q275. What is the output of the following code: for (int i = 0; i < 5; ++i) { if (i == 3) continue; cout << i << " "; } in C++?

A. 0 1 2 3

B. 0 1 2 3 4

C. 0 1 2 4 ✓

D. Compilation error

Answer: Option C — 0 1 2 4

Q276. What will be the output of the following code: for (int i = 0; i < 3; ++i) { for (int j = 0; j < 2; ++j) { cout << i << j << " "; } } in C++?

A. 00 11 22 33

B. 00 10 20 01 11

C. Compilation error

D. 00 01 10 11 ✓

Answer: Option D — 00 01 10 11

Q277. In C++, what is the syntax for the 'while' loop?

A. while (condition) { statement; } condition;

B. while { statement; } condition;

C. while (condition) { statement; } ✓

D. while { condition } { statement; }

Answer: Option C — while (condition) { statement; }

Q278. What will be the output of the following code: int i = 5; if (i != 5) cout << "Not equal"; else cout << "Equal"; in C++?

A. Equal ✓

B. Not equal

C. 5

D. Compilation error

Answer: Option A — Equal

Q279. What is the output of the following code: int i = 0; while (i < 5) { cout << i << " "; ++i; } in C++?

A. Compilation error

B. 1 2 3 4

C. 0 1 2 3

D. 0 1 2 3 4 ✓

Answer: Option D — 0 1 2 3 4

Q280. How many times will the following 'for' loop iterate: for (int i = 0; i < 5; ++i) in C++?

A. 4

B. 5 ✓

C. 6

D. Infinite loop

Answer: Option B — 5

Q281. In C++, what is the purpose of the 'continue' statement in a loop?

A. Skips the remaining code in the loop

B. Skips the current iteration ✓

C. Terminates the loop

D. Restarts the loop

Answer: Option B — Skips the current iteration

Q282. What will be the output of the following code: int i = 0; while (i < 3) { cout << ++i << " "; } in C++?

A. 1 2 3

B. 0 1 2 3 ✓

C. 1 2 3 0

D. Compilation error

Answer: Option B — 0 1 2 3

Q283. What will be the output of the following code: int x = 10; if (x < 5) cout << "Less"; else if (x == 5) cout << "Equal"; else cout << "More"; in C++?

A. More ✓

B. Less

C. Equal

D. Compilation error

Answer: Option A — More

Functions And Procedures In C++

Q284. What is the correct syntax for defining a function in C++?

A. returnType functionName(parameters) { body; } ✓

B. functionName(parameters) { returnType body; }

C. functionName(returnType, parameters) { body; }

D. returnType functionName(parameters, body) { }

Answer: Option A — returnType functionName(parameters) { body; }

Q285. What is the purpose of a function prototype in C++?

A. Determines the parameters of the function

B. Specifies the return type of the function

C. Provides the implementation of the function

D. Declares the function before it is defined ✓

Answer: Option D — Declares the function before it is defined

---

Q286. Which keyword is used to define a function in C++ that does not return any value?

A. return

B. null

C. void ✓

D. None of the above

Answer: Option C — void

Q287. What is the correct way to call a function named "add" that takes two parameters in C++?

A. add(a, b);

B. add(int a, int b); ✓

C. functionName = add(a, b);

D. add.functionName(a, b);

Answer: Option B — add(int a, int b);

Q288. What does the 'return' statement do in a function in C++?

A. Skips the remaining code in the function

B. Returns a value from the function ✓

C. Restarts the function execution

D. None of the above

Answer: Option B — Returns a value from the function

Q289. What is the output of the following code: int sum(int a, int b) { return a + b; } cout << sum(3, 5); in C++?

A. 35

B. Compilation error

C. 8 ✓

D. sum(3, 5)

Answer: Option C — 8

Q290. Which of the following is true about function overloading in C++?

A. Only one function can have a specific name

B. A function cannot be defined with the same name twice

C. Function overloading is not supported in C++

D. Multiple functions can have the same name with different parameters ✓

Answer: Option D — Multiple functions can have the same name with different parameters

---

Q291. What is the purpose of default arguments in C++ functions?

A. Allows a function to be called without providing all arguments ✓

B. Sets a default return value for the function

C. Defines default behavior for the function

D. Specifies the default data type for parameters

Answer: Option A — Allows a function to be called without providing all arguments

Q292. What is the correct syntax for declaring a function prototype in C++?

A. returnType functionName(parameters); ✓

B. functionName(parameters) returnType;

C. functionName returnType(parameters);

D. None of the above

Answer: Option A — returnType functionName(parameters);

Q293. What is the purpose of passing arguments by reference in C++ functions?

A. Ensures the function receives correct arguments

B. Reduces the memory consumption of the function

C. Improves performance of the function

D. Allows changes made to the arguments to reflect outside the function ✓

Answer: Option D — Allows changes made to the arguments to reflect outside the function

Q294. What is the output of the following code: int square(int x) { return x * x; } int main() { cout << square(3) + square(2); } in C++?

A. 13 ✓

B. 25

C. 12

D. Compilation error due to mismatched return types

Answer: Option A — 13

Q295. What is the correct syntax for defining a recursive function in C++?

A. returnType functionName(parameters) { functionName(parameters); } ✓

B. returnType functionName(parameters) { functionName(); }

C. functionName(parameters) { return functionName(parameters); }

D. None of the above

Answer: Option A — returnType functionName(parameters) { functionName(parameters); }

Q296. What is the purpose of a function header in C++?

A. Contains the actual code of the function

B. Specifies the function's location in memory

C. Declares the name, return type, and parameters of the function ✓

D. Provides default arguments for the function

Answer: Option C — Declares the name, return type, and parameters of the function

Q297. What will be the output of the following code: int max(int a, int b) { return (a > b) ? a : b; } cout << max(5, 3); in C++?

A. max(5, 3) ✓

B. 3

C. Compilation error due to missing function definition

D. max(3, 5)

Answer: Option A — max(5, 3)

Q298. Which of the following is true about function recursion in C++?

A. Recursion can only be achieved through function pointers

B. Recursion is limited to predefined standard functions

C. A function calling itself directly or indirectly ✓

D. Recursion is not allowed in C++

Answer: Option C — A function calling itself directly or indirectly

Q299. What is the purpose of a function in C++?

A. To declare variables

B. To print output

C. To initialize memory

D. To perform a specific task ✓

Answer: Option D — To perform a specific task

Q300. Which of the following is true about function parameters in C++?

A. They are variables used to pass data into functions ✓

B. They are placeholders for function names

C. They are used to return values from functions

D. They are only used in main functions

Answer: Option A — They are variables used to pass data into functions

---

Q301. What is the correct way to declare a function in C++ that takes no parameters and returns an integer?

A. void functionName();

B. functionName(int);

C. functionName() int;

D. int functionName(); ✓

Answer: Option D — int functionName();

Q302. What is the purpose of the 'void' keyword in a function declaration in C++?

A. Indicates that the function accepts no parameters

B. Indicates that the function returns a void pointer

C. Indicates that the function does not return a value ✓

D. Indicates that the function is a member function

Answer: Option C — Indicates that the function does not return a value

Q303. What is the output of the following code: int multiply(int x, int y) { return x * y; } int main() { cout << multiply(3, 4); } in C++?

A. multiply(3, 4)

B. 12 ✓

C. Compilation error due to missing return statement

D. multiply(4, 3)

Answer: Option B — 12

Q304. Which of the following is a valid function declaration in C++?

A. sum(int a, int b) { return a + b; }

B. sum(int, int) { return a + b; }

C. sum(a, b) { return a + b; }

D. int sum(int a, int b); ✓

Answer: Option D — int sum(int a, int b);

Q305. What is the purpose of function overloading in C++?

A. Allows multiple functions with the same name but different parameters ✓

B. Improves the efficiency of the functions

C. Ensures that functions can be called from anywhere

D. Facilitates the declaration of global variables

Answer: Option A — Allows multiple functions with the same name but different parameters

---

Q306. What does the 'inline' keyword do in C++ function declaration?

A. Indicates that the function should be ignored

B. Declares that the function should be inlined

C. Specifies that the function should be called inline

D. Suggests the compiler to replace the function call with its definition ✓

Answer: Option D — Suggests the compiler to replace the function call with its definition

Q307. What is the output of the following code: int max(int a, int b) { return (a > b) ? a : b; } int main() { cout << max(5, 5); } in C++?

A. 10

B. 0

C. 5 ✓

D. Compilation error due to ambiguous function call

Answer: Option C — 5

Q308. In C++, how are function arguments passed by default?

A. By pointer

B. By value ✓

C. By address

D. None of the above

Answer: Option B — By value

Q309. What is the purpose of the 'static' keyword in a function declaration in C++?

A. It allows a function to be accessed from other files

B. It restricts the scope of the function ✓

C. It indicates that the function is a member function

D. It allocates memory for the function

Answer: Option B — It restricts the scope of the function

Q310. What is the syntax for defining a recursive function in C++?

A. returnType functionName(parameters) { if (condition) functionName(parameters); }

B. functionName(parameters) { returnType if (condition) functionName(parameters); }

C. functionName(parameters) { functionName(parameters); }

D. returnType functionName(parameters) { functionName(parameters); } ✓

Answer: Option D — returnType functionName(parameters) { functionName(parameters); }

Q311. What will be the output of the following code: int add(int x, int y) { return x + y; } int main() { cout << add(2); } in C++?

A. Compilation error due to missing return statement

B. 0

C. Compilation error due to missing argument ✓

D. 2

Answer: Option C — Compilation error due to missing argument

Q312. What is the purpose of the 'extern' keyword in a function declaration in C++?

A. It declares the function as an external function

B. It ensures that the function is not inlined

C. It makes the function visible only within the file

D. It specifies that the function is defined elsewhere ✓

Answer: Option D — It specifies that the function is defined elsewhere

Q313. What is the correct way to pass an array to a function in C++?

A. By declaring the array globally and accessing it directly

B. By passing the array index as a parameter

C. By passing the array name and size as parameters ✓

D. By passing each array element as a parameter

Answer: Option C — By passing the array name and size as parameters

Q314. What is the purpose of the 'static' keyword in a C++ function?

A. Declares the function as a member of a class

B. Prevents the function from being called recursively

C. Defines the function to be inline

D. Retains the variable's value between function calls ✓

Answer: Option D — Retains the variable's value between function calls

Q315. What does the 'void' keyword represent in the function declaration in C++?

A. Indicates that the function does not return any value ✓

B. Indicates that the function returns a pointer

C. Indicates that the function accepts no arguments

D. Indicates that the function can be called from anywhere

Answer: Option A — Indicates that the function does not return any value

---

Q316. Which of the following is true about inline functions in C++?

A. Inline functions must have a return type declared

B. Inline functions cannot have return statements

C. Inline functions cannot be recursive

D. The compiler replaces the function call with the function code wherever it is called ✓

Answer: Option D — The compiler replaces the function call with the function code wherever it is called

Q317. What is the output of the following code: int add(int a, int b) { return a + b; } int sum = add(3, 5); cout << sum; in C++?

A. 3

B. 5

C. 8 ✓

D. Compilation error

Answer: Option C — 8

Q318. In C++, what is the significance of the function return type 'auto'?

A. It allows the function to return any data type

B. The compiler automatically deduces the return type based on the return statement ✓

C. It specifies the function to be executed automatically

D. None of the above

Answer: Option B — The compiler automatically deduces the return type based on the return statement

Q319. What is the purpose of the 'friend' keyword in a C++ function declaration?

A. It specifies that the function should be inlined

B. It allows the function to call itself recursively

C. It grants access to private and protected members of a class to the function ✓

D. It declares the function as a friend function

Answer: Option C — It grants access to private and protected members of a class to the function

Q320. What is the syntax for declaring a function that takes a variable number of arguments in C++?

A. returnType functionName(var_args...);

B. returnType functionName(var_args...); { body; }

C. returnType functionName(..., var_args);

D. returnType functionName(...); ✓

Answer: Option D — returnType functionName(...);

Q321. What is the output of the following code: int x = 5; cout << "Before: " << x; changeValue(x); cout << "After: " << x; in C++?

A. Before: 5 After: 5 ✓

B. Before: 5 After: 10

C. Before: 5 After: 6

D. Compilation error

Answer: Option A — Before: 5 After: 5

Q322. What is the correct syntax for defining a function template in C++?

A. returnType functionName(T parameter) { template body; }

B. template returnType functionName(T parameter) { body; } ✓

C. returnType functionName(template T parameter) { body; }

D. None of the above

Answer: Option B — template returnType functionName(T parameter) { body; }

Q323. What is the purpose of the 'const' keyword in a function parameter list in C++?

A. Declares the parameter as constant within the function

B. Prevents the parameter from being passed by reference

C. Indicates that the parameter value cannot be changed within the function ✓

D. Specifies that the parameter should be initialized

Answer: Option C — Indicates that the parameter value cannot be changed within the function

Q324. In C++, what is a lambda function?

A. A function that takes no arguments and returns nothing

B. An anonymous function that can be used as an argument to other functions ✓

C. A function that automatically adjusts its return type

D. None of the above

Answer: Option B — An anonymous function that can be used as an argument to other functions

Q325. What is the purpose of the 'decltype' keyword in C++?

A. Determines the type of an expression or variable ✓

B. Declares the type of a function parameter

C. Specifies the type of a function's return value

D. Returns the size of a data type

Answer: Option A — Determines the type of an expression or variable

---

Q326. What is the correct way to declare a function that takes a pointer as an argument in C++?

A. returnType functionName(*type parameter)

B. returnType functionName(type parameter*)

C. returnType functionName(type ¶meter)

D. returnType functionName(type* parameter) ✓

Answer: Option D — returnType functionName(type* parameter)

Q327. What will be the output of the following code: void display() { cout << "Hello"; } int main() { display(); return 0; } in C++?

A. Compilation error

B. 0

C. Hello ✓

D. Undefined behavior

Answer: Option C — Hello

Q328. What will be the output of the following C++ code? #include <iostream> using namespace std; int func (int a, int b) { cout << a; cout << b; return 0; } int main(void) { int(*ptr)(char, int); ptr = func; func(2, 3); ptr(2, 3); return 0; }

A. 2323

B. 232

C. 23

D. compile time error ✓

Answer: Option D — compile time error

Q329. If we start our function call with default arguments means, what will be proceeding arguments?

A. user argument

B. empty arguments

C. default arguments ✓

D. user & empty arguments

Answer: Option C — default arguments

Q330. Which of the following is the default return value of functions in C++?

A. int ✓

B. char

C. float

D. void

Answer: Option A — int

Q331. What is an inline function?

A. A function that is expanded at each call during execution ✓

B. A function that is called during compile time

C. A function that is not checked for syntax errors

D. A function that is not checked for semantic analysis

Answer: Option A — A function that is expanded at each call during execution

Q332. What will be the output of the following C++ code? include <iostream> using namespace std; long factorial (long a) { if (a > 1) return (a * factorial (a + 1)); else return (1); } int main () { long num = 3; cout << num << "! = " << factorial (num); return 0; }

A. 6

B. 24

C. segmentation fault ✓

D. compile time error

Answer: Option C — segmentation fault

Q333. Identify the correct statement.

A. Namespace is used to group class, objects and functions ✓

B. Namespace is used to mark the beginning of the program

C. A namespace is used to separate the class, objects

D. Namespace is used to mark the beginning & end of the program

Answer: Option A — Namespace is used to group class, objects and functions

Q334. What is the scope of the variable declared in the user defined function?

A. whole program

B. only inside the {} block ✓

C. the main function

D. header section

Answer: Option B — only inside the {} block

Q335. If an argument from the parameter list of a function is defined constant then . . . . . .

A. It can be modified inside the function

B. It cannot be modified inside the function ✓

C. Error occurs

D. Segmentation fault

Answer: Option B — It cannot be modified inside the function

Q336. What will be the output of the following C++ code? #include <iostream> using namespace std; void Funct(); int main() { try { Funct(); } catch(double) { cerr << "caught a double type..." << endl; } return 0; } void Funct() { throw 3; }

A. caught a double type

B. compile time error

C. abnormal program termination ✓

D. runtime error

Answer: Option C — abnormal program termination

Q337. What will be the output of the following C++ code? #include <iostream> using namespace std; void Sum(int a, int b, int & c) { a = b + c; b = a + c; c = a + b; } int main() { int x = 2, y =3; Sum(x, y, y); cout << x << " " << y; return 0; }

A. 2 3

B. 6 9

C. 2 15 ✓

D. compile time error

Answer: Option C — 2 15

Q338. Identify the correct statement.

A. c++ does not have built-in interfaces ✓

B. c++ does have built-in interfaces

C. c++ have no concept of interfaces

D. c++ does have built-in interfaces & classes

Answer: Option A — c++ does not have built-in interfaces

Q339. which of the following can be passed in function pointers?

A. variables

B. data types

C. functions ✓

D. objects

Answer: Option C — functions

Q340. Where should default parameters appear in a function prototype?

A. To the rightmost side of the parameter list ✓

B. To the leftmost side of the parameter list

C. Anywhere inside the parameter list

D. Middle of the parameter list

Answer: Option A — To the rightmost side of the parameter list

---

Q341. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; namespace A{ int var = 10; } namespace B{ int cout = 5; } int main() { using namespace B; cout<<A::var; }

A. 10

B. 5

C. Error ✓

D. 105

Answer: Option C — Error

Q342. Which is more effective while calling the functions?

A. call by value

B. call by reference ✓

C. call by pointer

D. call by object

Answer: Option B — call by reference

Q343. What is similar to the interface in c++?

A. methods

B. instance of a class

C. pure abstract class ✓

D. methods & instance of a class

Answer: Option C — pure abstract class

Q344. What will be the output of the following C++ code? #include <iostream> using namespace std; void print(int i) { cout << i; } void print(double f) { cout << f; } int main(void) { print(5); print(500.263); return 0; }

A. 5500.263 ✓

B. 500.2635

C. 500.263

D. 500.266

Answer: Option A — 5500.263

Q345. What will be the output of the following C++ code? #include <iostream> using namespace std; #define SquareOf(x) x * x int main() { int x; cout << SquareOf(x + 4); return 0; }

A. 16

B. 64

C. compile time error

D. 75386824 ✓

Answer: Option D — 75386824

Q346. Pick the incorrect statement for namespaces in C++.

A. Namespace declarations are always global scope

B. Keyword namespace is used at the starting of a namespace definition

C. Namespace has access specifiers like private or public ✓

D. Namespace definitions can be nested

Answer: Option C — Namespace has access specifiers like private or public

Q347. What will be the output of the following C++ code? #include <iostream> #include <stdarg.h> using namespace std; int add (int num, ...) { int sum = 0; va_list args; va_start (args,num); for (int i = 0; i < num; i++) { int num = va_arg (args,int); sum += num; } va_end (args); return sum; } int main (void) { int total = add(8, 1, 2, -1, 4, 12, -2, 9, 7); cout << "The result is " << total; return 0; }

A. 32 ✓

B. 23

C. 48

D. compile time error

Answer: Option A — 32

Q348. What will happen when the exception is not caught in the program?

A. error ✓

B. program will execute

C. block of that code will not execute

D. program will execute & displays wrong output

Answer: Option A — error

Q349. What should be passed in parameters when function does not require any parameters?

A. void

B. blank space ✓

C. both void & blank space

D. tab space

Answer: Option B — blank space

Q350. How many types do functions fall depends on modularization?

A. 1

B. 2 ✓

C. 3

D. 4

Answer: Option B — 2

Q351. What will be the output of the following C++ code? #include <iostream> #include <stdarg.h> using namespace std; void dumplist(int, ...); int main() { dumplist(2, 4, 8); dumplist(3, 6, 9, 7); return 0; } void dumplist(int n, ...) { va_list p; int i; va_start(p, n); while (n-->0) { i = va_arg(p, int); cout << i; } va_end(p); }

A. 2436

B. 48697 ✓

C. 1111111

D. compile time error

Answer: Option B — 48697

Q352. Which keyword is used to access the variable in the namespace?

A. using ✓

B. dynamic

C. const

D. static

Answer: Option A — using

Q353. Where can the default parameter be placed by the user?

A. leftmost

B. rightmost ✓

C. both leftmost & rightmost

D. topmost

Answer: Option B — rightmost

Q354. What will happen while using pass by reference?

A. The values of those variables are passed to the function so that it can manipulate them

B. The location of variable in memory is passed to the function so that it can use the same memory area for its processing ✓

C. The function declaration should contain ampersand (& in its type declaration)

D. The function declaration should contain $

Answer: Option B — The location of variable in memory is passed to the function so that it can use the same memory area for its processing

Q355. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; namespace { int var = 10; } int main() { cout<<var; }

A. 10 ✓

B. Error

C. Some garbage value

D. Nothing but program runs perfectly

Answer: Option A — 10

Q356. Which operator is used to signify the namespace?

A. conditional operator

B. ternary operator

C. scope operator ✓

D. bitwise operator

Answer: Option C — scope operator

Q357. which keyword is used to define the macros in c++?

A. macro

B. define

C. #define ✓

D. #macro

Answer: Option C — #define

Q358. What will be the output of the following C++ code? #include <iostream> using namespace std; int operate (int a, int b) { return (a * b); } float operate (float a, float b) { return (a / b); } int main() { int x = 5, y = 2; float n = 5.0, m = 2.0; cout << operate(x, y) <<"\t"; cout << operate (n, m); return 0; }

A. 10.0 5.0

B. 5.0 2.5

C. 10.0 5

D. 10 2.5 ✓

Answer: Option D — 10 2.5

Q359. What will be the output of the following C++ code? #include <iostream> using namespace std; #define PR(id) cout << "The value of " #id " is "<<id int main() { int i = 10; PR(i); return 0; }

A. 10 ✓

B. 15

C. 20

D. 12

Answer: Option A — 10

Q360. In which of the following we cannot overload the function?

A. return function ✓

B. caller

C. called function

D. main function

Answer: Option A — return function

Q361. What will be the output of the following C++ code? #include <iostream> using namespace std; int fun(int=0, int = 0); int main() { cout << fun(5); return 0; } int fun(int x, int y) { return (x+y); }

A. -5

B. 0

C. 10

D. 5 ✓

Answer: Option D — 5

Q362. Which keyword is used to check exception in the block of code?

A. catch

B. throw

C. try ✓

D. handlers

Answer: Option C — try

Q363. In which of the following cases inline functions may not word? i. If the function has static variables. ii. If the function has global and register variables. iii. If the function contains loops iv. If the function is recursive

A. i, iv

B. iii, iv

C. ii, iii, iv

D. i, iii, iv ✓

Answer: Option D — i, iii, iv

Q364. Which operator is used for accessing a member of namespace?

A. :

B. :: ✓

C. ->

D. .

Answer: Option B — ::

Q365. What will be the output of the following C++ code? #include <iostream> using namespace std; namespace extra { int i; } void i() { using namespace extra; int i; i = 9; cout << i; } int main() { enum letter { i, j}; class i { letter j; }; ::i(); return 0; }

A. 9 ✓

B. 10

C. compile time error

D. 11

Answer: Option A — 9

Q366. To which does the function pointer point to?

A. variable

B. constants

C. function ✓

D. absolute variables

Answer: Option C — function

Q367. What will happen when the handler is not found for an exception?

A. calls the standard library function terminate() ✓

B. raise an error

C. executes the remaining block

D. raise an error and executes the remaining block

Answer: Option A — calls the standard library function terminate()

Q368. What will be the output of the following C++ code? #include <iostream> using namespace std; int func(int m = 10, int n) { int c; c = m + n; return c; } int main() { cout << func(5); return 0; }

A. 15

B. 10

C. compile time error ✓

D. 30

Answer: Option C — compile time error

Q369. Which of the following permits function overloading on c++?

A. type

B. number of arguments

C. type & number of arguments ✓

D. number of objects

Answer: Option C — type & number of arguments

Q370. What happens to a function defined inside a class without any complex operations (like looping, a large number of lines, etc)?

A. It becomes a virtual function of the class

B. It becomes a default calling function of the class

C. It becomes an inline function of the class ✓

D. The program gives an error

Answer: Option C — It becomes an inline function of the class

---

Q371. Which of the following feature is used in function overloading and function with default argument?

A. Encapsulation

B. Polymorphism ✓

C. Abstraction

D. Modularity

Answer: Option B — Polymorphism

Q372. What will be the output of the following C++ code? #include <iostream> using namespace std; int mult (int x, int y) { int result; result = 0; while (y != 0) { result = result + x; y = y - 1; } return(result); } int main () { int x = 5, y = 5; cout << mult(x, y) ; return(0); }

A. 20

B. 25 ✓

C. 30

D. 35

Answer: Option B — 25

Q373. An inline function is expanded during . . . . . .

A. compile-time ✓

B. run-time

C. never expanded

D. end of the program

Answer: Option A — compile-time

Q374. What is the mandatory part to present in function pointers?

A. &

B. return values

C. data types ✓

D. $

Answer: Option C — data types

Q375. Where does the return statement returns the execution of the program?

A. main function

B. caller function ✓

C. same function

D. block function

Answer: Option B — caller function

Q376. What is the other name of the macro?

A. scripted directive ✓

B. executed directive

C. link directive

D. executed & link directive

Answer: Option A — scripted directive

Q377. What will be the output of the following C++ code? #include <iostream> #include <stdarg.h> using namespace std; int flue(char c,...); int main() { int x, y; x = flue('A', 1, 2, 3); y = flue('1', 1.0,1, '1', 1.0f, 1l); cout << x << y; return 0; } int flue(char c,...) { return c; }

A. 6549 ✓

B. 4965

C. 6646

D. compile time error

Answer: Option A — 6549

Q378. What is the general syntax for accessing the namespace variable?

A. namespace::operator ✓

B. namespace,operator

C. namespace#operator

D. namespace$operator

Answer: Option A — namespace::operator

Q379. What will you use if you are not intended to get a return value?

A. static

B. const

C. volatile

D. void ✓

Answer: Option D — void

Q380. From which function the execution of a C++ program starts?

A. start() function

B. main() function ✓

C. new() function

D. end() function

Answer: Option B — main() function

Q381. Which value will it take when both user and default values are given?

A. user value ✓

B. default value

C. custom value

D. defined value

Answer: Option A — user value

Q382. which of the following is used to terminate the function declaration?

A. :

B.)

C. ; ✓

D.]

Answer: Option C — ;

Q383. What will be the output of the following C++ code? #include <iostream> using namespace std; int Add(int X, int Y, int Z) { return X + Y; } double Add(double X, double Y, double Z) { return X + Y; } int main() { cout << Add(5, 6); cout << Add(5.5, 6.6); return 0; }

A. 11 12.1

B. 12.1 11

C. 11 12

D. compile time error ✓

Answer: Option D — compile time error

Arrays And Strings In C++

Q384. What is the correct way to declare a one-dimensional array in C++?

A. int* arr = new int[size];

B. int[] arr = new int[size];

C. int arr = new int[size];

D. int arr[size]; ✓

Answer: Option D — int arr[size];

Q385. How do you access the third element of an array named 'arr' in C++?

A. arr(3)

B. arr[3]

C. arr[2] ✓

D. arr(2)

Answer: Option C — arr[2]

Q386. What is the size of the following array declaration in C++: char str[10];?

A. 11

B. 10 ✓

C. Depends on the length of the string assigned to 'str'

D. None of these

Answer: Option B — 10

Q387. What is the purpose of the 'strlen()' function in C++?

A. Concatenates two strings

B. Compares two strings

C. Returns the length of a null-terminated string ✓

D. Copies a string to another string

Answer: Option C — Returns the length of a null-terminated string

Q388. How do you initialize all elements of an integer array named 'arr' to zero in C++?

A. arr[] = {0};

B. arr = {0};

C. arr.fill(0);

D. for (int i = 0; i < size; ++i) { arr[i] = 0; } ✓

Answer: Option D — for (int i = 0; i < size; ++i) { arr[i] = 0; }

Q389. What is the correct syntax for declaring a two-dimensional array in C++?

A. int arr[rowSize][colSize]; ✓

B. int arr = new int[rowSize][colSize];

C. int[] arr[rowSize][colSize];

D. int* arr[rowSize][colSize];

Answer: Option A — int arr[rowSize][colSize];

Q390. Which of the following statements is true about C-style strings in C++?

A. They must be declared using the 'string' keyword

B. They are objects of the 'string' class

C. They can have variable lengths

D. They are represented as arrays of characters terminated by a null character ✓

Answer: Option D — They are represented as arrays of characters terminated by a null character

---

Q391. What is the output of the following code: char str[] = "Hello"; cout << str[5]; in C++?

A. H

B. o

C. Undefined behavior ✓

D. Compilation error

Answer: Option C — Undefined behavior

Q392. How do you determine the length of a C-style string in C++?

A. Subtracting the address of the first character from the last

B. Using the 'strlen()' function ✓

C. Counting the number of elements in the array

D. None of the above

Answer: Option B — Using the 'strlen()' function

Q393. What is the correct way to pass an array as an argument to a function in C++?

A. By declaring the array as global

B. By passing the array itself

C. Using a pointer to the array ✓

D. By passing each element of the array individually

Answer: Option C — Using a pointer to the array

Q394. What does the 'strcpy()' function do in C++?

A. Concatenates two strings

B. Compares two strings

C. Returns the length of a string

D. Copies one string to another ✓

Answer: Option D — Copies one string to another

Q395. What is the output of the following code: char str[] = "Hello"; cout << sizeof(str); in C++?

A. 6 ✓

B. 5

C. 4

D. Compilation error

Answer: Option A — 6

Q396. What is the correct way to initialize a string object named 'str' in C++?

A. str("Hello");

B. str = new string("Hello");

C. str = "Hello";

D. string str = "Hello"; ✓

Answer: Option D — string str = "Hello";

Q397. What is the purpose of the 'strcat()' function in C++?

A. Compares two strings

B. Copies one string to another

C. Concatenates two strings ✓

D. Returns the length of a string

Answer: Option C — Concatenates two strings

Q398. What is the output of the following code: char str[] = "Hello"; cout << str; in C++?

A. Compilation error

B. Hello ✓

C. Undefined behavior

D. None of the above

Answer: Option B — Hello

Q399. Which of the following correctly initializes a two-dimensional array named matrix in C++ with dimensions 3x3?

A. int matrix[3][3] = {[1, 4, 7], [2, 5, 8], [3, 6, 9]};

B. int matrix[3][3] = {[1, 2, 3], [4, 5, 6], [7, 8, 9]};

C. int matrix[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}; ✓

D. int matrix[3][3] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

Answer: Option C — int matrix[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

Q400. What is the output of the following code snippet in C++? int arr[] = {1, 2, 3, 4, 5}; cout << arr[5];

A. 5

B. 0

C. Compilation error

D. Undefined behavior ✓

Answer: Option D — Undefined behavior

Q401. Which of the following correctly defines a string variable named name initialized with the value "John" in C++?

A. string name = "John"; ✓

B. char name[] = "John";

C. char* name = "John";

D. string name("John");

Answer: Option A — string name = "John";

Q402. What is the purpose of the 'strcmp()' function in C++?

A. Returns the length of a C-style string

B. Copies one C-style string to another

C. Concatenates two C-style strings

D. Compares two C-style strings ✓

Answer: Option D — Compares two C-style strings

Q403. Which of the following correctly declares an array of 10 integers in C++ and initializes all elements to zero?

A. int arr[10] = {};

B. int arr[10]; for(int i = 0; i < 10; ++i) arr[i] = 0;

C. int arr[10] = {0}; ✓

D. int arr[10] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0};

Answer: Option C — int arr[10] = {0};

Q404. What is the output of the following code snippet in C++? char str[] = "Hello"; cout << str[4];

A. e

B. o ✓

C. Undefined behavior

D. l

Answer: Option B — o

Q405. In C++, which function is used to read a string with spaces using cin?

A. cin.getline(str);

B. scanf("%s", str);

C. getline(cin, str); ✓

D. cin >> str;

Answer: Option C — getline(cin, str);

Q406. What is the correct way to access the fifth character in a string variable named text in C++?

A. text(5)

B. text[5]

C. text.charAt(5)

D. text[4] ✓

Answer: Option D — text[4]

Q407. What is the purpose of the 'toupper()' function in C++?

A. Converts a character to uppercase ✓

B. Converts a character to lowercase

C. Compares two characters

D. Returns the length of a string

Answer: Option A — Converts a character to uppercase

Q408. What is the output of the following code snippet in C++? char str[] = "Hello"; cout << sizeof(str);

A. Compilation error

B. 4

C. 5

D. 6 ✓

Answer: Option D — 6

Q409. What is the correct way to declare a string object named myString and initialize it with the value "Welcome" in C++?

A. myString("Welcome");

B. string myString = "Welcome"; ✓

C. char myString[] = "Welcome";

D. None of the above

Answer: Option B — string myString = "Welcome";

Q410. What does the 'strncpy()' function do in C++?

A. Returns the length of a string

B. Compares two strings

C. Copies a specified number of characters from one string to another ✓

D. Concatenates two strings

Answer: Option C — Copies a specified number of characters from one string to another

---

Q411. What is the purpose of the 'strtok()' function in C++?

A. Converts a string to lowercase

B. Returns the length of a string

C. Concatenates two strings

D. Tokenizes a string based on a delimiter ✓

Answer: Option D — Tokenizes a string based on a delimiter

Q412. Which of the following statements is true about array indices in C++?

A. Array indices must be of type 'int'

B. Array indices can be negative

C. Array indices start from 0 ✓

D. Array indices start from 1

Answer: Option C — Array indices start from 0

Q413. What is the correct way to access the last element of an array named arr in C++?

A. arr[end]

B. arr[-1]

C. arr[size]

D. arr[size - 1] ✓

Answer: Option D — arr[size - 1]

Q414. How do you declare a multidimensional array with 3 rows and 4 columns in C++?

A. int arr[3][4] = {0};

B. int arr[3, 4];

C. int arr[][] = { {1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12} }

D. int arr[3][4]; ✓

Answer: Option D — int arr[3][4];

Q415. What is the output of the following code: char str[10]; cout << sizeof(str); in C++?

A. 9

B. 11

C. 10 ✓

D. Compilation error

Answer: Option C — 10

Q416. Which function is used to find the first occurrence of a specified substring within a string in C++?

A. strfind()

B. strstr() ✓

C. substr()

D. None of the above

Answer: Option B — strstr()

Q417. What is the output of the following code: char str[] = "Hello"; cout << str[0] << str[4]; in C++?

A. Undefined behavior

B. Compilation error

C. Ho ✓

D. ello

Answer: Option C — Ho

Q418. In C++, what is the purpose of the 'cin.getline()' function?

A. Reads a character from standard input

B. Reads a word from standard input

C. Reads a string from standard input

D. Reads a line of text from standard input ✓

Answer: Option D — Reads a line of text from standard input

Q419. How do you declare a string object named 'text' in C++?

A. string text; ✓

B. string text();

C. string text{};

D. string text = "";

Answer: Option A — string text;

Q420. What is the correct syntax for declaring an array of 5 integers in C++?

A. int arr{} = {1, 2, 3, 4, 5};

B. int[] arr = new int[5];

C. int arr[] = {1, 2, 3, 4, 5};

D. int arr[5]; ✓

Answer: Option D — int arr[5];

Q421. What is the purpose of the 'cin.ignore()' function in C++?

A. Ignores the next character in the input buffer

B. Ignores the previous character in the input buffer

C. Clears the input buffer ✓

D. Reads a character from the input buffer and discards it

Answer: Option C — Clears the input buffer

Q422. What is the output of the following code: char str[] = "Hello"; cout << strlen(str); in C++?

A. Hello

B. 5 ✓

C. Compilation error

D. Runtime error

Answer: Option B — 5

Q423. What is the correct syntax to declare a string literal in C++?

A. const char* str = "Hello"; ✓

B. string str = "Hello";

C. char* str = "Hello";

D. char str[] = "Hello";

Answer: Option A — const char* str = "Hello";

Q424. How do you access the element at the second row and third column of a two-dimensional array named 'matrix' in C++?

A. matrix(3, 2)

B. matrix[3][2]

C. matrix(2, 3)

D. matrix[2][3] ✓

Answer: Option D — matrix[2][3]

Q425. Which of the following is true regarding the declaration of a string in C++ using the char array?

A. It can be initialized using the assignment operator

B. It cannot be modified after initialization

C. It requires a null character at the end ✓

D. It automatically allocates memory for the null character

Answer: Option C — It requires a null character at the end

Q426. Which is an instantiation of the basic_string class template?

A. Character

B. String class ✓

C. Memory

D. Iterator

Answer: Option B — String class

Q427. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main () { string str="Steve Jobs founded the apple"; string str2 = str.substr (6, 4); unsigned pos = str.find("the"); string str3 = str.substr (pos); cout << str2 << ' ' << str3 << '\n'; return 0; }

A. Jobs the apple ✓

B. the apple

C. Steve

D. Jobs

Answer: Option A — Jobs the apple

Q428. What will be the output of the following C++ code? #include <stdio.h> #include<iostream> using namespace std; int main() { int a = 5, b = 10, c = 15; int arr[3] = {&a, &b, &c}; cout << *arr[*arr[1] - 8]; return 0; }

A. 15

B. 18

C. garbage value

D. compile time error ✓

Answer: Option D — compile time error

Q429. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main () { string str ("Steve jobs"); cout << str.length(); return 0; }

A. 8

B. 10 ✓

C. 12

D. 9

Answer: Option B — 10

Q430. What will happen if a string is empty?

A. It can't be created

B. Raises an error

C. It can be used ✓

D. It cannot be used

Answer: Option C — It can be used

Q431. How many parameters can a resize method take?

A. 1

B. 2

C. 1 or 2 ✓

D. None of the above

Answer: Option C — 1 or 2

Q432. What will be the output of the following C++ code? #include <stdio.h> #include <iostream> using namespace std; int main() { int array[] = {10, 20, 30}; cout << -2[array]; return 0; }

A. -15

B. -30 ✓

C. compile time error

D. garbage value

Answer: Option B — -30

Q433. What is the difference between unsigned int length() and unsigned int size()?

A. Returns a different value

B. They are same ✓

C. Returns a different value but they are same

D. Returns a length

Answer: Option B — They are same

Q434. How does the strings are stored in the memory?

A. Contiguous ✓

B. Non-contiguous

C. Null

D. sequence

Answer: Option A — Contiguous

Q435. What will be the output of the following C++ code? #include <stdio.h> #include<iostream> using namespace std; int array1[] = {1200, 200, 2300, 1230, 1543}; int array2[] = {12, 14, 16, 18, 20}; int temp, result = 0; int main() { for (temp = 0; temp < 5; temp++) { result += array1[temp]; } for (temp = 0; temp < 4; temp++) { result += array2[temp]; } cout << result; return 0; }

A. 6553

B. 6533 ✓

C. 6522

D. 12200

Answer: Option B — 6533

Q436. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main () { string str ("Steve jobs"); cout << str.capacity() << "\n"; return 0; }

A. 9

B. 10

C. 11

D. Not Fix ✓

Answer: Option D — Not Fix

Q437. How many maximum number of parameters does a string constructor can take?

A. 1

B. 2

C. 3 ✓

D. 4

Answer: Option C — 3

Q438. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main () { string str; string str2="Steve jobs"; string str3="He founded apple"; str.append(str2); str.append(str3, 6, 3); str.append(str3.begin() + 6, str3.end()); str.append(5,0x2e); cout << str << '\n'; return 0; }

A. Steve jobs

B. He founded apple

C. Steve

D. Steve jobsndended apple..... ✓

Answer: Option D — Steve jobsndended apple.....

Q439. Which of the following accesses the seventh element stored in array?

A. array[6]; ✓

B. array[7];

C. array(7);

D. array;

Answer: Option A — array[6];

Q440. What is the correct definition of an array?

A. An array is a series of elements of the same type in contiguous memory locations ✓

B. An array is a series of element

C. An array is a series of elements of the same type placed in non-contiguous memory locations

D. An array is an element of the different type

Answer: Option A — An array is a series of elements of the same type in contiguous memory locations

---

Q441. Which of the following gives the memory address of the first element in array?

A. array[0];

B. array[1];

C. array(2);

D. array; ✓

Answer: Option D — array;

Q442. Which character is used to terminate the string?

A. $

B. Null ✓

C. Empty

D. @

Answer: Option B — Null

Q443. Where are the strings stored?

A. Stack

B. Heap

C. Both Stack & Heap ✓

D. Queue

Answer: Option C — Both Stack & Heap

Q444. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main () { string str ("I like to code in C"); unsigned sz = str.size(); str.resize (sz + 2, '+'); str.resize (14); cout << str << '\n'; return 0; }

A. I like to code in c

B. I like to code ✓

C. I like to code in c++

D. I like to codeI like to code

Answer: Option B — I like to code

Q445. What will be the output of the following C++ code? #include <iostream> #include <cstring> #include <string> using namespace std; int main () { string str ("Steve jobs"); char * cstr = new char [str.length() + 1]; strcpy (cstr, str.c_str()); char * p = strtok (cstr," "); while (p != 0) { cout << p << '\n'; p = strtok(NULL," "); } delete[] cstr; return 0; }

A. Steve jo

B. Steve jobs

C. Steve jobs ✓

D. Error

Answer: Option C — Steve jobs

Q446. Which constant member functions does not modify the string?

A. bool empty() ✓

B. assign

C. append

D. delete

Answer: Option A — bool empty()

Q447. What will be the output of the following C++ code? #include <stdio.h> #include <iostream> using namespace std; int main() { char str[5] = "ABC"; cout << str[3]; cout << str; return 0; }

A. ABC ✓

B. ABCD

C. AB

D. AC

Answer: Option A — ABC

Q448. What is the index number of the last element of an array with 9 elements?

A. 9

B. 8 ✓

C. 0

D. Programmer-defined

Answer: Option B — 8

Q449. Which header file is used to manipulate the string?

A. iostream

B. iomanip

C. string ✓

D. container

Answer: Option C — string

Q450. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main () { string str ("Test string"); for (string :: iterator it = str.begin(); it != 5; ++it) cout << *it; return 0; }

A. Test

B. string

C. Test string

D. Error ✓

Answer: Option D — Error

Q451. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main () { string str ("Steve jobs founded the apple"); string str2 ("apple"); unsigned found = str.find(str2); if (found != string :: npos) cout << found << '\n'; return 0; }

A. apple

B. 12

C. 23 ✓

D. Steve jobs founded the

Answer: Option C — 23

Q452. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main () { string name ("Jobs"); string family ("Steve"); name += " Apple "; name += family; name += '\n'; cout << name; return 0; }

A. Steve Jobs

B. Apple

C. Jobs Apple Steve ✓

D. Apple Steve

Answer: Option C — Jobs Apple Steve

Q453. Which of the following correctly declares an array?

A. int array[10]; ✓

B. int array;

C. array{10};

D. array array[10];

Answer: Option A — int array[10];

Q454. What will be the output of the following C++ code? #include <stdio.h> #include<iostream> using namespace std; int main () { int array[] = {0, 2, 4, 6, 7, 5, 3}; int n, result = 0; for (n = 0; n < 8;

n++) { result += array[n]; } cout << result; return 0; }

A. 25

B. 26

C. 27 ✓

D. 21

Answer: Option C — 27

Q455. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main () { string str ("Steve jobs"); unsigned long int found = str.find_first_of("aeiou"); while (found != string :: npos) { str[found] = '*'; found = str.find_first_of("aeiou", found + 1); } cout << str << '\n'; return 0; }

A. Steve

B. jobs

C. St*v* j*bs ✓

D. St*v*

Answer: Option C — St*v* j*bs

Pointers And References In C++

Q456. What is a pointer in C++?

A. A variable that stores the size of another variable

B. A variable that stores the address of another variable ✓

C. A variable that stores a reference to another variable

D. None of the above

Answer: Option B — A variable that stores the address of another variable

Q457. How do you declare a pointer variable in C++?

A. ptr *type;

B. type ptr;

C. type *ptr; ✓

D. ptr type;

Answer: Option C — type *ptr;

Q458. What is the output of the following code: int x = 5; int *ptr = &x; cout << *ptr; in C++?

A. Address of x

B. Garbage value

C. Compilation error

D. 5 ✓

Answer: Option D — 5

Q459. What is the correct syntax to dereference a pointer in C++?

A. *ptr ✓

B. ptr*

C. &ptr

D. ptr

Answer: Option A — *ptr

Q460. What is the purpose of the 'nullptr' keyword in C++?

A. Represents a null type

B. Represents a null value

C. Represents a null reference

D. Represents a null pointer ✓

Answer: Option D — Represents a null pointer

Q461. What does the 'nullptr' keyword replace in C++11 and later versions?

A. nullptr pointer

B. Null value

C. NULL pointer ✓

D. Null reference

Answer: Option C — NULL pointer

Q462. What is the purpose of the 'reference' in C++?

A. Value of an existing variable

B. Alias to an existing variable ✓

C. Address of an existing variable

D. None of the above

Answer: Option B — Alias to an existing variable

Q463. What is the output of the following code: int x = 5; int &ref = x; cout << ref; in C++?

A. Compilation error

B. Garbage value

C. 5 ✓

D. Address of x

Answer: Option C — 5

Q464. What happens when a reference is declared but not initialized in C++?

A. Undefined behavior

B. Default-initialized to null

C. Initialized to a default value

D. Compilation error ✓

Answer: Option D — Compilation error

Q465. How do you declare a reference variable in C++?

A. type &ref; ✓

B. ref type;

C. type ref;

D. &type ref;

Answer: Option A — type &ref;

Q466. What is the purpose of a null pointer in C++?

A. Represents a pointer to a reserved memory location

B. Represents a pointer to a deleted object

C. Represents a pointer to a null object

D. Represents that the pointer does not point to a valid object ✓

Answer: Option D — Represents that the pointer does not point to a valid object

Q467. What is the output of the following code: int arr[3] = {1, 2, 3}; int *ptr = arr; cout << *(ptr + 1); in C++?

A. 1

B. 3

C. 2 ✓

D. Garbage value

Answer: Option C — 2

Q468. What is a dangling pointer in C++?

A. A pointer that points to a null value

B. A pointer that points to a deallocated memory ✓

C. A pointer that points to a reserved memory location

D. None of the above

Answer: Option B — A pointer that points to a deallocated memory

Q469. What is the address-of operator in C++?

A. *

B. ->

C. ::

D. & ✓

Answer: Option D — &

Q470. What is the output of the following code: int arr[] = {1, 2, 3}; cout << *arr; in C++?

A. 1 ✓

B. 2

C. 3

D. Compilation error

Answer: Option A — 1

Q471. How do you declare a constant pointer to an integer variable in C++?

A. const int * const ptr;

B. const int *ptr;

C. const *int ptr;

D. int *const ptr; ✓

Answer: Option D — int *const ptr;

Q472. What is a pointer to a pointer known as in C++?

A. Reference pointer

B. Pointer pointer

C. Double pointer ✓

D. Nested pointer

Answer: Option C — Double pointer

Q473. What does the 'new' keyword do in C++?

A. Declares a new variable

B. Allocates memory dynamically ✓

C. Returns the size of a variable

D. None of the above

Answer: Option B — Allocates memory dynamically

Q474. What is a null reference in C++?

A. A reference with a null value

B. A reference initialized with nullptr

C. A reference that doesn't refer to any object ✓

D. A reference that refers to a null object

Answer: Option C — A reference that doesn't refer to any object

Q475. What is the output of the following code: int x = 10; int *ptr = &x; cout << *ptr << " " << ptr; in C++?

A. 10 Memory address of x

B. 10 Value of x

C. Compilation error

D. 10 Address of x ✓

Answer: Option D — 10 Address of x

Q476. What is the correct way to pass a pointer to a function in C++?

A. By reference ✓

B. By value

C. By dereferencing the pointer

D. By declaring the pointer globally

Answer: Option A — By reference

Q477. What is a memory leak in C++?

A. Failure to allocate memory when needed

B. Allowing memory to be accessed by unauthorized users

C. Accessing an invalid memory location

D. Failure to deallocate memory after it's no longer needed ✓

Answer: Option D — Failure to deallocate memory after it's no longer needed

Q478. What is the purpose of the 'delete' keyword in C++?

A. Deletes a variable

B. Deletes a pointer

C. Deallocates memory dynamically ✓

D. Deallocates memory statically

Answer: Option C — Deallocates memory dynamically

Q479. What does the 'nullptr' keyword signify when used as a pointer value in C++?

A. A pointer to a null reference

B. A null pointer ✓

C. A pointer to a null object

D. None of the above

Answer: Option B — A null pointer

Q480. What is a pointer arithmetic in C++?

A. Assigning arithmetic expressions to a pointer

B. Using pointers to perform arithmetic operations

C. Performing arithmetic operations on pointers ✓

D. Storing arithmetic operations in a pointer

Answer: Option C — Performing arithmetic operations on pointers

---

Q481. What is the purpose of the 'this' pointer in C++?

A. Points to the next object instance

B. Points to the previous object instance

C. Points to the base class object instance

D. Points to the current object instance ✓

Answer: Option D — Points to the current object instance

Q482. What is the output of the following code: char str[] = "Hello"; cout << sizeof(str); in C++?

A. 5

B. 4

C. Compilation error

D. 6 ✓

Answer: Option D — 6

Q483. What does the 'sizeof' operator return when used with a pointer variable in C++?

A. Size of the pointer variable

B. Size of the data type pointed to by the pointer ✓

C. Address of the pointer variable

D. Address of the data pointed to by the pointer

Answer: Option B — Size of the data type pointed to by the pointer

Q484. What is the purpose of the 'const' keyword in a pointer declaration in C++?

A. Indicates that the pointer itself is constant

B. Indicates that the value pointed to is constant ✓

C. Indicates that the pointer can be reassigned

D. Indicates that the pointer points to a constant value

Answer: Option B — Indicates that the value pointed to is constant

Q485. What is the output of the following code: int arr[3] = {1, 2, 3}; cout << *(arr + 2); in C++?

A. 1

B. 2

C. 3 ✓

D. Compilation error

Answer: Option C — 3

Q486. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int const p = 5; cout << ++p; return 0; }

A. 5

B. 6

C. Error ✓

D. 8

Answer: Option C — Error

Q487. What we can't do on a void pointer?

A. pointer arithmetic ✓

B. pointer functions

C. pointer objects

D. pointer functions & objects

Answer: Option A — pointer arithmetic

Q488. Choose the right option. string* x, y;

A. x is a pointer to a string, y is a string ✓

B. y is a pointer to a string, x is a string

C. both x and y are pointers to string types

D. y is a pointer to a string

Answer: Option A — x is a pointer to a string, y is a string

Q489. What is size of generic pointer in C++ (in 32-bit platform)?

A. 2

B. 4 ✓

C. 8

D. 0

Answer: Option B — 4

Q490. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int n = 5; void *p = &n; int *pi = static_cast<int*>(p); cout << *pi << endl; return 0;

}

A. 5 ✓

B. 6

C. compile time error

D. runtime error

Answer: Option A — 5

Q491. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; int main(int argc, char const *argv[]) { int a = 5; int &p = a; cout<<p; return 0; }

A. 5 ✓

B. Run-time error

C. Segmentation fault

D. Compile-time error

Answer: Option A — 5

Q492. The correct statement for a function that takes pointer to a float, a pointer to a pointer to a char and returns a pointer to a pointer to a integer is . . . . . .

A. int **fun(float**, char**)

B. int *fun(float*, char*)

C. int **fun(float*, char) ✓**

D. int ***fun(*float, **char)

Answer: Option C — int **fun(float*, char)**

Q493. What will be the output of the following C++ code? #include <iostream> using namespace std; void print (char * a) { cout << a << endl; } int main () { const char * a = "Hello world"; print(const_cast (a)); return 0; }

A. Hello world ✓

B. Hello

C. world

D. compile time error

Answer: Option A — Hello world

Q494. Identify the incorrect statement.

A. Reference is the alternate name of the object

B. A reference value once defined can be reassigned ✓

C. A reference value once defined cannot be reassigned

D. Reference is the alternate name of the variable

Answer: Option B — A reference value once defined can be reassigned

Q495. Which of the following function must use reference.

A. Assignment operator function

B. Copy Constructor ✓

C. Destructor

D. Parameterized constructor

Answer: Option B — Copy Constructor

Q496. How are the constants declared?

A. const keyword

B. #define preprocessor

C. both const keyword and #define preprocessor ✓

D. $define

Answer: Option C — both const keyword and #define preprocessor

Q497. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; int main(int argc, char const *argv[]) { int a = 5; int *p = &a; int &q = a; cout<<p<<endl; cout<<q<<endl; return 0; }

A. Address of a followed by 5 in next line ✓

B. Address of p followed by 5 in next line

C. Address of a followed by Address of a in next line

D. Address of p followed by Address of q in next line

Answer: Option A — Address of a followed by 5 in next line

Q498. How to declare a wide character in the string literal?

A. L prefix ✓

B. l prefix

C. W prefix

D. Z prefix

Answer: Option A — L prefix

Q499. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; int main(int argc, char const *argv[]) { int a = 5; int *p = &a; int &q = a; cout<<*p<<endl; cout<<*q<<endl; return 0; }

A. Address of a followed by 5 in next line

B. Address of p followed by 5 in next line

C. Run time error

D. Compile time error ✓

Answer: Option D — Compile time error

Q500. Pick the correct statement about references.

A. References can be assigned value NULL

B. References once assigned cannot be changed to refer another variable ✓

C. Reference should not be initialized when created

D. Reference is the same as pointers

Answer: Option B — References once assigned cannot be changed to refer another variable

---

Q501. What will be the output of the following C++ code? #include <iostream> using namespace std; int main () { int numbers[5]; int * p; p = numbers; *p = 10; p++; *p = 20; p = &numbers[2]; *p = 30; p = numbers + 3; *p = 40; p = numbers; *(p + 4) = 50; for (int n = 0; n < 5; n++) cout << numbers[n] << ","; return 0; }

A. 10,20,30,40,50, ✓

B. 1020304050

C. compile error

D. runtime error

Answer: Option A — 10,20,30,40,50,

Q502. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; void func(const int &a) { int temp = 10; a = temp;

cout<<a; } int main(int argc, char const *argv[]) { int a = 5; func(a); return 0; }

A. 5

B. 10

C. Error ✓

D. Segmentation fault

Answer: Option C — Error

Q503. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int i; char c; void *data; i = 2; c = 'd'; data = &i; cout << "the data points to the integer value" << data; data = &c; cout << "the data now points to the character" << data; return 0; }

A. 2d

B. two memory addresses ✓

C. 3d

D. 4d

Answer: Option B — two memory addresses

Q504. Regarding the following statement which of the statements is true? const int a = 100;

A. Declares a variable a with 100 as its initial value

B. Declares a construction a with 100 as its initial value

C. Declares a constant a whose value will be 100 ✓

D. Constructs an integer type variable with an as identifier and 100 as the value

Answer: Option C — Declares a constant a whose value will be 100

Q505. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { char *ptr; char Str[] = "abcdefg"; ptr = Str; ptr += 5; cout << ptr; return 0; }

A. fg ✓

B. cdef

C. defg

D. abcd

Answer: Option A — fg

Q506. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; int main(int argc, char const *argv[]) { int a = 5; int &q = a; cout<<&a<<endl; cout<<&q<<endl; return 0; }

A. 5 5

B. Address of p followed by 5 in next line

C. 5 followed by Address of a in next line

D. Address of a followed by Address of a in next line ✓

Answer: Option D — Address of a followed by Address of a in next line

Q507. The pointer can point to any variable that is not declared with which of these?

A. const

B. volatile

C. both const & volatile ✓

D. static

Answer: Option C — both const & volatile

Q508. What will be the output of the following C++ code? #include <iostream> using namespace std; void swap(int &a, int &b); int main() { int a = 5, b = 10; swap(a, b); cout << "In main " << a << b; return 0; } void swap(int &a, int &b) { int temp; temp = a; a = b; b = temp; cout << "In swap " << a << b; }

A. In swap 105 In main 105 ✓

B. In swap 105 In main 510

C. In swap 510 In main 105

D. In swap 510 In main 510

Answer: Option A — In swap 105 In main 105

Q509. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int a = 5, b = 10, c = 15; int *arr[ ] = {&a, &b, &c}; cout << arr[1]; return 0; }

A. 5

B. 10

C. 15

D. it will return some random number ✓

Answer: Option D — it will return some random number

Q510. What will be the output of the following C++ code? #include<iostream> using namespace std; int main() { int x = 10; int& ref = x; ref = 20; cout << x << endl ; x = 30; cout << ref << endl; return 0; }

A. 20 30 ✓

B. 10 10

C. 10 20

D. 10 30

Answer: Option A — 20 30

Q511. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int a[2][4] = {3, 6, 9, 12, 15, 18, 21, 24}; cout << *(a[1] + 2) << *(*(a + 1) + 2) << 2[1[a]]; return 0; }

A. 15 18 21

B. 21 21 21 ✓

C. 24 24 24

D. Compile time error

Answer: Option B — 21 21 21

Q512. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int i; const char *arr[] = {"C", "C++", "Java", "VBA"}; const char *(*ptr)[4] = &arr; cout << ++(*ptr)[2]; return 0; }

A. ava ✓

B. java

C. c++

D. compile time error

Answer: Option A — ava

Q513. Pick the correct statement about references in C++.

A. References stores the address of variables

B. References and variables both have the same address ✓

C. References use dereferencing operator(*) to access the value of variable its referencing

D. References were also available in C

Answer: Option B — References and variables both have the same address

Q514. A void pointer cannot point to which of these?

A. methods in c++

B. class member in c++

C. methods & class member in c++

D. none of the mentioned ✓

Answer: Option D — none of the mentioned

Q515. Identify the correct sentence regarding inequality between reference and pointer.

A. we can not create the array of reference ✓

B. we can create the Array of reference

C. we can use reference to reference

D. we can use variable

Answer: Option A — we can not create the array of reference

Q516. Which of the following is illegal?

A. int *ip;

B. string s, *sp = 0;

C. int i; double* dp = &i; ✓

D. int *pi = 0;

Answer: Option C — int i; double* dp = &i;

Q517. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; int main(int argc, char const *argv[]) { int a = 5; int *p = &a; int *(&q) = p; cout<<q; return 0; }

A. 5

B. Address of pointer a ✓

C. Address of pointer p

D. Error

Answer: Option B — Address of pointer a

Q518. Which one of the following is not a possible state for a pointer.

A. hold the address of the specific object

B. point one past the end of an object

C. zero

D. point to a type ✓

Answer: Option D — point to a type

Q519. Which value can we not assign to reference?

A. integer

B. floating

C. unsigned

D. null ✓

Answer: Option D — null

Q520. What will be the output of the following C++ code? #include <iostream> using namespace std; int f(int &x, int c) { c = c - 1; if (c == 0) return 1; x = x + 1; return f(x, c) * x; } int main(int argc, char const *argv[]) { int a = 4; cout<<f(a,a); return 0; }

A. 343 ✓

B. 336

C. 120

D. 840

Answer: Option A — 343

Q521. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; int main(int argc, char const *argv[]) { int &q = NULL; cout<<q; return 0; }

A. NULL

B. 0

C. Address of NULL

D. Error ✓

Answer: Option D — Error

Q522. Which reference modifier is used to define the reference variable?

A. & ✓

B. $

C. #

D. @

Answer: Option A — &

Q523. The operator used for dereferencing or indirection is . . . . . .

A. * ✓

B. &

C. ->

D. –>>

Answer: Option A — *

Q524. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; int main(int argc, char const *argv[]) { int &q = 5; cout<<q; return 0; }

A. 5

B. Run-time error

C. Segmentation fault

D. Compile-time error ✓

Answer: Option D — Compile-time error

Q525. When does the void pointer can be dereferenced?

A. when it doesn't point to any value

B. when it cast to another type of object ✓

C. using delete keyword

D. using shift keyword

Answer: Option B — when it cast to another type of object

Q526. The constants are also called as . . . . . .

A. const

B. preprocessor

C. literals ✓

D. variables

Answer: Option C — literals

Q527. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int arr[] = {4, 5, 6, 7}; int *p = (arr + 1); cout << *arr + 9; return 0; }

A. 12

B. 5

C. 13 ✓

D. error

Answer: Option C — 13

Q528. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; int main(int argc, char const *argv[]) { int &p; int a = 5; &p = a; cout<<p; return 0; }

A. 5

B. 55

C. Error ✓

D. Segmentation fault

Answer: Option C — Error

Q529. Which of the following is incorrect?

A. References cannot be NULL

B. A reference must be initialized when declared

C. Once a reference is declared, it cannot be modified later to reference another object i.e. it cannot be reset

D. References cannot refer to a constant value ✓

Answer: Option D — References cannot refer to a constant value

Q530. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { char arr[20]; int i; for(i = 0; i < 10; i++) *(arr + i) = 65 + i; *(arr + i) = '\0'; cout << arr; return(0); }

A. ABCDEFGHIJ ✓

B. AAAAAAAAAA

C. JJJJJJJJ

D. AAAAAAJJJJ

Answer: Option A — ABCDEFGHIJ

Q531. What are the parts of the literal constants?

A. integer numerals

B. floating-point numerals

C. strings and boolean values

D. all of the mentioned ✓

Answer: Option D — all of the mentioned

Q532. What is the meaning of the following declaration? int(*p[5])();

A. p is pointer to function

B. p is array of pointer to function ✓

C. p is pointer to such function which return type is the array

D. p is pointer to array of function

Answer: Option B — p is array of pointer to function

Q533. What will happen in the following C++ code snippet? int a = 100, b = 200; int *p = &a, *q = &b; p = q;

A. b is assigned to a

B. p now points to b ✓

C. a is assigned to b

D. q now points to a

Answer: Option B — p now points to b

Q534. What will be the output of the following C++ code? #include <iostream> using namespace std; #define PI 3.14159 int main () { float r = 2; float circle; circle = 2 * PI * r; cout << circle; return 0; }

A. 12.5664 ✓

B. 13.5664

C. 10

D. 15

Answer: Option A — 12.5664

Q535. What is the difference between references and pointers?

A. References are an alias for a variable whereas pointer stores the address of a variable ✓

B. References and pointers are similar

C. References stores address of variables whereas pointer points to variables

D. Pointers are an alias for a variable whereas references stores the address of a variable

Answer: Option A — References are an alias for a variable whereas pointer stores the address of a variable

---

Q536. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int *p; void *vp; if (vp == p) cout << "equal"; return 0; }

A. equal ✓

B. no output

C. compile error

D. runtime error

Answer: Option A — equal

Q537. What will be the output of the following C++ code? #include<iostream> using namespace std; int &fun() { static int x = 10; return x; } int main() { fun() = 30; cout << fun(); return 0; }

A. 30 ✓

B. 10

C. Error

D. Segmentation fault

Answer: Option A — 30

Q538. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; int main(int argc, char const *argv[]) { int a = 5; int *p = &a; int &q = p; cout<<p; return 0; }

A. 5

B. Run-time error

C. Segmentation fault

D. Compile-time error ✓

Answer: Option D — Compile-time error

Q539. What will be the output of the following C++ code? #include <iostream> using namespace std; int func(void *Ptr); int main() { char *Str = "abcdefghij"; func(Str); return 0; } int func(void *Ptr) { cout << Ptr; return 0; }

A. abcdefghij

B. address of string "abcdefghij" ✓

C. compile time error

D. runtime error

Answer: Option B — address of string "abcdefghij"

Q540. Which of the following statement(s) is/are correct?

A. * operator is used to declare a reference

B. A reference variable defined to refer a particular variable can refer to any other variable also

C. References must always be initialized inside classes

D. A variable can have more than one references ✓

Answer: Option D — A variable can have more than one references

---

Q541. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int a = 5, c; void *p = &a; double b = 3.14; p = &b; c = a + b; cout << c << '\n' << p; return 0; }

A. 8, memory address ✓

B. 8.14

C. memory address

D. 12

Answer: Option A — 8, memory address

Q542. What are the references in C++?

A. An alternative name for already existing variables ✓

B. A pointer to a variable

C. A new type of variables

D. A new type of constant variable

Answer: Option A — An alternative name for already existing variables

Q543. The difference between x and 'x' is?

A. The first one refers to a variable whose identifier is x and the second one refers to the character constant x ✓

B. The first one is a character constant x and the second one is the string literal x

C. Both are same

D. Both are string literal

Answer: Option A — The first one refers to a variable whose identifier is x and the second one refers to the character constant x

Q544. What does the following statement mean? int (*fp)(char*)

A. pointer to a pointer

B. pointer to an array of chars

C. pointer to function taking a char* argument and returns an int ✓

D. function taking a char* argument and returning a pointer to int

Answer: Option C — pointer to function taking a char* argument and returns an int

Q545. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int arr[] = {4, 5, 6, 7}; int *p = (arr + 1); cout << arr; return 0; }

A. 4

B. 5

C. address of arr ✓

D. 7

Answer: Option C — address of arr

Q546. Which of the following statement is not true about preprocessor directives?

A. These are lines read and processed by the preprocessor

B. They do not produce any code by themselves

C. These must be written on their own line

D. They end with a semicolon ✓

Answer: Option D — They end with a semicolon

Q547. What will be the output of the following C++ code? #include<iostream> using namespace std; int &fun() { int x = 10; return x; } int main() { fun() = 30; cout << fun(); return 0; }

A. 30

B. 10

C. Error

D. Segmentation fault ✓

Answer: Option D — Segmentation fault

Q548. How a reference is different from a pointer?

A. A reference cannot be null

B. A reference once established cannot be changed

C. The reference doesn't need an explicit dereferencing mechanism

D. All of the mentioned ✓

Answer: Option D — All of the mentioned

Q549. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int arr[] = {4, 5, 6, 7}; int *p = (arr + 1); cout << *p; return 0; }

A. 4

B. 5 ✓

C. 6

D. 7

Answer: Option B — 5

Q550. What does a reference provide?

A. Alternate name for the class

B. Alternate name for the variable ✓

C. Alternate name for the pointer

D. Alternate name for the object

Answer: Option B — Alternate name for the variable

Q551. Which of the following operator is used while declaring references?

A. *

B. & ✓

C. ^

D. ->

Answer: Option B — &

Q552. The void pointer can point to which type of objects?

A. int

B. float

C. double

D. all of the mentioned ✓

Answer: Option D — all of the mentioned

Q553. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int a = 9; int & aref = a; a++; cout << "The value of a is " << aref; return 0; }

A. 9

B. 10 ✓

C. error

D. 11

Answer: Option B — 10

Structures And Unions In C++

Q554. What is a structure in C++?

A. A collection of functions

B. A reserved keyword in C++

C. A user-defined data type containing variables of different data types ✓

D. A way to declare arrays of data

Answer: Option C — A user-defined data type containing variables of different data types

Q555. Which keyword is used to define a structure in C++?

A. typedef

B. struct ✓

C. union

D. None of the above

Answer: Option B — struct

Q556. What is the correct way to access members of a structure in C++?

A. Using the star operator

B. Using the double colon operator

C. Using the dot operator ✓

D. Using the arrow operator

Answer: Option C — Using the dot operator

Q557. What is the purpose of typedef in C++ structures?

A. To define a new structure

B. To initialize structure members

C. To create a pointer to a structure

D. To create an alias for data types ✓

Answer: Option D — To create an alias for data types

Q558. How do you declare a structure variable in C++?

A. structName variableName; ✓

B. struct variableName;

C. variableName structName;

D. variableName = structName;

Answer: Option A — structName variableName;

Q559. What is the output of the following code: struct Student { int roll; }; Student s1; s1.roll = 101; cout << s1.roll; in C++?

A. Undefined behavior

B. 0

C. Compilation error

D. 101 ✓

Answer: Option D — 101

Q560. What is the purpose of using structures in C++?

A. To perform mathematical operations

B. To define loops in the program

C. To group related data together ✓

D. To represent true/false values

Answer: Option C — To group related data together

Q561. What is the correct way to pass a structure to a function in C++?

A. By value

B. By reference

C. By pointer

D. All of the above ✓

Answer: Option D — All of the above

Q562. What is a union in C++?

A. A reserved keyword in C++

B. A user-defined data type that allows storing different data types in the same memory location ✓

C. A way to declare arrays of data

D. None of the above

Answer: Option B — A user-defined data type that allows storing different data types in the same memory location

Q563. What is the difference between a structure and a union in C++?

A. Structures cannot contain member functions

B. Unions are used for mathematical operations

C. In a structure, each member has its own memory space, while in a union, all members share the same memory space ✓

D. Structures are used for data organization

Answer: Option C — In a structure, each member has its own memory space, while in a union, all members share the same memory space

Q564. What is the size of the following structure in C++: struct Employee { int id; char name[20]; float salary; };

A. 28 bytes

B. 20 bytes

C. 8 bytes

D. 32 bytes ✓

Answer: Option D — 32 bytes

Q565. What is the correct syntax to define a structure pointer in C++?

A. structName *ptr; ✓

B. ptr *structName;

C. struct *ptrName;

D. *ptrName structName;

Answer: Option A — structName *ptr;

Q566. What is the output of the following code: union Data { int x; char y; }; Data d; d.x = 10; d.y = 'A'; cout << d.x << " " << d.y; in C++?

A. Compilation error

B. A A

C. 10 10

D. 0.416666667 ✓

Answer: Option D — 0.416666667

Q567. What is the purpose of using unions in C++?

A. To group related data together

B. To define loops in the program

C. To save memory space ✓

D. To represent true/false values

Answer: Option C — To save memory space

Q568. What does the 'sizeof' operator return when used with a structure in C++?

A. Size of the last member of the structure

B. Size of the first member of the structure

C. Size of the structure ✓

D. Address of the structure

Answer: Option C — Size of the structure

Q569. What is the correct syntax to declare a structure with a tag name in C++?

A. structName { /* members */ };

B. struct { /* members */ } structName;

C. structName typedef { /* members */ };

D. struct structName { /* members */ }; ✓

Answer: Option D — struct structName { /* members */ };

Q570. What is the correct way to access structure members through a pointer in C++?

A. Using the star operator (*)

B. Using the double colon operator (::)

C. Using the dot operator (.)

D. Using the arrow operator (->) ✓

Answer: Option D — Using the arrow operator (->)

Q571. What is the difference between a structure and a class in C++?

A. Structures support inheritance

B. Classes support only member functions

C. By default, members of a class are private, while members of a structure are public ✓

D. Structures cannot contain member functions

Answer: Option C — By default, members of a class are private, while members of a structure are public

Q572. What is the output of the following code: struct Point { int x, y; }; Point p1; p1.x = 10; p1.y = 20; cout << p1.x << " " << p1.y; in C++?

A. Garbage value 20 10

B. 10 20 ✓

C. Compilation error

D. None of the above

Answer: Option B — 10 20

Q573. What is the purpose of the 'typedef' keyword in C++ structures?

A. To create a pointer to a structure

B. To initialize structure members

C. To create an alias for a data type ✓

D. To define a new structure

Answer: Option C — To create an alias for a data type

Q574. Which keyword is used to declare a union in C++?

A. class

B. typedef

C. struct

D. union ✓

Answer: Option D — union

Q575. What is the correct syntax to define a union in C++?

A. union unionName { /* members */ }; ✓

B. unionName { /* members */ };

C. union { /* members */ } unionName;

D. unionName typedef { /* members */ };

Answer: Option A — union unionName { /* members */ };

Q576. What is the output of the following code: struct Book { char title[50]; int pages; }; Book b; cout << sizeof(b); in C++?

A. 50

B. Depends on the compiler ✓

C. 52

D. 60

Answer: Option B — Depends on the compiler

Q577. What is the output of the following code: union Number { int x; float y; }; Number n; n.x = 10; cout << n.y; in C++?

A. Compilation error

B. 10

C. Garbage value ✓

D. 0

Answer: Option C — Garbage value

Q578. What is a dangling pointer in C++?

A. A pointer that points to a deallocated memory ✓

B. A pointer that points to a valid memory

C. A pointer that points to a null value

D. A pointer that points to a reserved memory location

Answer: Option A — A pointer that points to a deallocated memory

Q579. The declaration of the structure is also called as?

A. structure creator

B. structure signifier

C. structure specifier ✓

D. structure creator & signifier

Answer: Option C — structure specifier

Q580. What will be the output of the following C++ code? #include <iostream> using namespace std; struct sec { int a; char b; }; int main() { struct sec s ={25,50}; struct sec *ps =(struct sec *)&s; cout << ps->a << ps->b; return 0; }

A. 252 ✓

B. 253

C. 254

D. 262

Answer: Option A — 252

Q581. What will be the output of the following C++ code? #include <iostream> using namespace std; struct Time { int hours; int minutes; int seconds; }; int toSeconds(Time now); int main() { Time t; t.hours = 5; t.minutes = 30; t.seconds = 45; cout << "Total seconds: " << toSeconds(t) << endl; return 0; } int toSeconds(Time now) { return 3600 * now.hours + 60 * now.minutes + now.seconds; }

A. 19845 ✓

B. 20000

C. 15000

D. 19844

Answer: Option A — 19845

Q582. What will be the output of the following C++ code? #include <iostream> #include <string.h> using namespace std; int main() { struct student { int num; char name[25]; }; student stu; stu.num = 123; strcpy(stu.name, "John"); cout << stu.num << endl; cout << stu.name << endl; return 0; }

A. 123 john ✓

B. john john

C. compile time error

D. runtime error

Answer: Option A — 123 john

Q583. What will be used when terminating a structure?

A. :

B. }

C. ; ✓

D. ;;

Answer: Option C — ;

Q584. The data elements in the structure are also known as what?

A. objects

B. members ✓

C. data

D. objects & data

Answer: Option B — members

Q585. Which of the following is a properly defined structure?

A. struct {int a;}

B. struct a_struct {int a;}

C. struct a_struct int a;

D. struct a_struct {int a;}; ✓

Answer: Option D — struct a_struct {int a;};

Q586. What will happen when the structure is declared?

A. it will not allocate any memory ✓

B. it will allocate the memory

C. it will be declared and initialized

D. it will be declared

Answer: Option A — it will not allocate any memory

Q587. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { struct ShoeType { string style; double price; }; ShoeType shoe1, shoe2; shoe1.style = "Adidas"; shoe1.price = 9.99; cout << shoe1.style << " $ "<< shoe1.price; shoe2 = shoe1; shoe2.price = shoe2.price / 9; cout << shoe2.style << " $ "<< shoe2.price; return 0; }

A. Adidas

$9.99Adidas$

1.11 ✓

B. Adidas

$9.99Adidas$

9.11

C. Adidas

$9.99Adidas$

11.11

D. Adidas

$11.11Adidas$

11.11

Answer: Option A — Adidas

$9.99Adidas$

1.11

Q588. Which of the following accesses a variable in structure *b?

A. b->var; ✓

B. b.var;

C. b-var;

D. b>var;

Answer: Option A — b->var;

Object Oriented Programming In C++

Q589. Which keyword is used to define a class in C++?

A. class ✓
B. struct
C. typedef
D. interface
Answer: Option A — class

Q590. What is the process of creating an instance of a class called in C++?

A. Encapsulation
B. Polymorphism
C. Inheritance
D. Instantiation ✓
Answer: Option D — Instantiation

Q591. What is inheritance in C++?

A. The process of creating multiple instances of a class
B. The process of hiding the implementation details of a class
C. The process of deriving a new class from an existing class ✓
D. The process of defining functions within a class
Answer: Option C — The process of deriving a new class from an existing class

Q592. What is the access specifier that allows members to be accessed from outside the class in C++?

A. protected
B. public ✓
C. friend
D. None of the above
Answer: Option B — public

Q593. Which concept in C++ allows a class to have multiple functions with the same name but different parameter lists?

A. Polymorphism
B. Encapsulation
C. Overloading ✓
D. Inheritance
Answer: Option C — Overloading

Q594. What is encapsulation in C++?

A. The process of creating multiple instances of a class
B. The process of hiding the implementation details of a class
C. The process of defining functions within a class
D. The bundling of data with the methods that operate on that data ✓
Answer: Option D — The bundling of data with the methods that operate on that data

Q595. Which keyword is used to inherit a class in C++?

A. public ✓
B. private

C. protected

D. extends

Answer: Option A — public

Q596. What is polymorphism in C++?

A. The ability of a function to perform different tasks based on the object that it is acting upon ✓

B. The process of creating multiple instances of a class

C. The process of hiding the implementation details of a class

D. The process of defining functions within a class

Answer: Option A — The ability of a function to perform different tasks based on the object that it is acting upon

Q597. Which of the following is true about constructors in C++?

A. They can be overloaded

B. They are used to destroy objects

C. They are called explicitly by the programmer

D. They are automatically called when an object is created ✓

Answer: Option D — They are automatically called when an object is created

Q598. What is the process of wrapping data and functions into a single unit called in C++?

A. Inheritance

B. Polymorphism

C. Encapsulation ✓

D. Abstraction

Answer: Option C — Encapsulation

Q599. What is dynamic polymorphism in C++?

A. The process of hiding the implementation details of a class

B. The ability of a function to perform different tasks based on the object that it is acting upon at runtime ✓

C. The process of defining functions within a class

D. None of the above

Answer: Option B — The ability of a function to perform different tasks based on the object that it is acting upon at runtime

Q600. Which keyword is used to specify that a function cannot be overridden in a derived class in C++?

A. abstract

B. virtual

C. final ✓

D. override

Answer: Option C — final

Q601. What is an abstract class in C++?

A. A class with no members

B. A class with only private members

C. A class with no implementation details

D. A class that cannot be instantiated and is used as a base class for other classes ✓

Answer: Option D — A class that cannot be instantiated and is used as a base class for other classes

Q602. What is the purpose of friend functions in C++?

A. To allow access to private and protected members of a class from outside the class ✓

B. To create multiple instances of a class

C. To hide the implementation details of a class

D. To define functions within a class

Answer: Option A — To allow access to private and protected members of a class from outside the class

Q603. What is the access specifier that allows members to be accessed only within the same class or by derived classes in C++?

A. internal

B. private

C. public

D. protected ✓

Answer: Option D — protected

Q604. What is a constructor in C++?

A. A function used to create multiple instances of a class

B. A function used to destroy objects

C. A special member function that initializes objects ✓

D. A function that defines the implementation of a class

Answer: Option C — A special member function that initializes objects

Q605. Which keyword is used to access the base class members from a derived class in C++?

A. super

B. this

C. parent

D. base ✓

Answer: Option D — base

Q606. What is operator overloading in C++?

A. Redefining a function within a class

B. Defining multiple functions with the same name but different arguments ✓

C. Defining new operators for existing types

D. None of the above

Answer: Option B — Defining multiple functions with the same name but different arguments

Q607. What is the purpose of a destructor in C++?

A. To provide an interface for a class

B. To encapsulate data

C. To clean up resources before an object is destroyed ✓

D. To create objects

Answer: Option C — To clean up resources before an object is destroyed

Q608. Which access specifier allows a member to be accessible only within its own class and derived classes in C++?

A. public
B. private
C. internal
D. protected ✓
Answer: Option D — protected

Q609. What is function overriding in C++?

A. Defining a function in the derived class with the same signature as in the base class ✓
B. Defining multiple functions with the same name but different arguments
C. Redefining a function within a class
D. Defining new operators for existing types
Answer: Option A — Defining a function in the derived class with the same signature as in the base class

Q610. What is the purpose of the 'static' keyword in C++?

A. To access class member functions
B. To specify the type of a variable
C. To allocate memory dynamically
D. To create class-level variables and functions ✓
Answer: Option D — To create class-level variables and functions

---

Q611. What is the difference between 'struct' and 'class' in C++?

A. The default access level of members in a struct is public, while in a class, it is private
B. Classes allow inheritance, while structs do not
C. Structs have all members public by default, while classes have them private by default ✓
D. Structs are used for inheritance, while classes are not
Answer: Option C — Structs have all members public by default, while classes have them private by default

Q612. What is the purpose of the 'virtual' keyword in C++?

A. To create class-level variables and functions
B. To allocate memory dynamically
C. To enable polymorphism through function overriding ✓
D. To specify the type of a variable
Answer: Option C — To enable polymorphism through function overriding

Q613. Which concept allows a derived class to inherit properties and behavior from a base class in C++?

A. Encapsulation
B. Inheritance ✓
C. Abstraction
D. None of the above
Answer: Option B — Inheritance

Q614. What is the correct syntax for inheriting a class in C++?

A. class Derived : Base {}
B. class Derived inherits Base {}
C. class Derived : public Base {} ✓
D. class Derived extends Base {}

Answer: Option C — class Derived : public Base {}

Q615. What is the purpose of access specifiers in C++ classes?

A. To specify the type of a variable
B. To enable polymorphism through function overriding
C. To create class-level variables and functions
D. To control the visibility of class members ✓
Answer: Option D — To control the visibility of class members

---

Q616. What is the purpose of the 'friend' keyword in C++?

A. To create class-level variables and functions
B. To specify the type of a variable
C. To enable polymorphism through function overriding
D. To allow a function or class to access private and protected members of another class ✓
Answer: Option D — To allow a function or class to access private and protected members of another class

Q617. What is the concept of data hiding in C++?

A. Inheritance
B. Polymorphism
C. Encapsulation ✓
D. Abstraction
Answer: Option C — Encapsulation

Q618. Which keyword is used to access the superclass constructor in a derived class constructor in C++?

A. parent
B. super ✓
C. derived
D. None of the above
Answer: Option B — super

Q619. What is the access specifier that restricts access to members of a class to only its friend classes and member functions in C++?

A. friend
B. public
C. private ✓
D. protected
Answer: Option C — private

Q620. What does the term "this" pointer refer to in C++?

A. Pointer to the base class object instance
B. Pointer to the next object instance
C. Pointer to the current object instance ✓
D. Pointer to the previous object instance
Answer: Option C — Pointer to the current object instance

Q621. Which type of inheritance allows a class to inherit properties and behavior from multiple base classes in C++?

A. Single inheritance
B. Hierarchical inheritance
C. Multilevel inheritance
D. Multiple inheritance ✓
Answer: Option D — Multiple inheritance

Q622. What is the difference between an object and a class in C++?

A. A class and an object are the same thing
B. A class represents a specific instance, whereas an object represents a general template
C. A class is a blueprint for creating objects, whereas an object is an instance of a class ✓
D. An object is a blueprint for creating classes, whereas a class is an instance of an object
Answer: Option C — A class is a blueprint for creating objects, whereas an object is an instance of a class

Q623. What is the process of hiding the implementation details of a class and showing only the necessary features to the outside world called in C++?

A. Encapsulation
B. Inheritance
C. Polymorphism
D. Abstraction ✓
Answer: Option D — Abstraction

Q624. What is the purpose of using virtual functions in C++?

A. To achieve runtime polymorphism ✓
B. To create multiple instances of a class
C. To hide the implementation details of a class
D. To define functions within a class
Answer: Option A — To achieve runtime polymorphism

Q625. Which keyword is used to prevent a class from being instantiated in C++?

A. abstract
B. sealed
C. final ✓
D. static
Answer: Option C — final

Q626. Which operator is used to access members of a class through a pointer in C++?

A. Star operator (*)
B. Double colon (::)
C. Dot operator (.)
D. Arrow operator (->) ✓
Answer: Option D — Arrow operator (->)

Q627. What is the term for the ability of a class to provide different implementations of a function depending on the types of its parameters in C++?

A. Function overriding
B. Function hiding
C. Function overloading ✓
D. Function polymorphism

Answer: Option C — Function overloading

Q628. Which access specifier allows members of a class to be accessed from anywhere in the program in C++?

A. protected

B. public ✓

C. friend

D. None of the above

Answer: Option B — public

Q629. What is the default access specifier for members of a class in C++?

A. friend

B. protected

C. private ✓

D. public

Answer: Option C — private

Q630. Which keyword is used to make a class inheritable by other classes in C++?

A. virtual

B. inheritable

C. extends ✓

D. base

Answer: Option C — extends

Q631. What is the purpose of destructors in C++?

A. To initialize objects of a class

B. To define objects within a class

C. To perform calculations with object data

D. To release resources acquired by the object ✓

Answer: Option D — To release resources acquired by the object

Q632. What will be the output of the following C++ code? #include<iostream> using namespace std; class Base { public: Base() { cout<<"Constructing Base \n"; } virtual~Base() { cout<<"Destructing Base \n"; } }; class Derived: public Base { public: Derived() { cout<<"Constructing Derived \n"; } ~Derived() { cout<<"Destructing Derived \n"; } }; int main(void) { Derived *d = new Derived(); Base *b = d; delete b; return 0; }

A. Constructing Base Constructing Derived Destructing Base

B. Constructing Base Constructing Derived Destructing Derived Destructing Base ✓

C. Constructing Base Constructing Derived Destructing Base Destructing Derived

D. Constructing Derived Constructing Base Destructing Base Destructing Derived

Answer: Option B — Constructing Base Constructing Derived Destructing Derived Destructing Base

Q633. Which concept allows you to reuse the written code?

A. Encapsulation

B. Abstraction

C. Inheritance ✓

D. Polymorphism

Answer: Option C — Inheritance

Q634. What is virtual inheritance?

A. C++ technique to avoid multiple copies of the base class into children/derived class ✓

B. C++ technique to avoid multiple inheritances of classes

C. C++ technique to enhance multiple inheritance

D. C++ technique to ensure that a private member of the base class can be accessed somehow

Answer: Option A — C++ technique to avoid multiple copies of the base class into children/derived class

Q635. How structures and classes in C++ differ?

A. In Structures, members are public by default whereas, in Classes, they are private by default ✓

B. In Structures, members are private by default whereas, in Classes, they are public by default

C. Structures by default hide every member whereas classes do not

D. Structures cannot have private members whereas classes can have

Answer: Option A — In Structures, members are public by default whereas, in Classes, they are private by default

---

Q636. Which of the following class allows to declare only one object of it?

A. Abstract class

B. Virtual class

C. Singleton class ✓

D. Friend class

Answer: Option C — Singleton class

Q637. Which of the following is correct about new and malloc? i. new is an operator whereas malloc is a function ii. new calls constructor malloc does not iii. new returns required pointer whereas malloc returns void pointer and needs to be typecast

A. i and ii

B. ii and iii

C. i and iii

D. i, ii and iii ✓

Answer: Option D — i, ii and iii

Q638. How run-time polymorphisms are implemented in C++?

A. Using Inheritance

B. Using Virtual functions

C. Using Templates

D. Using Inheritance and Virtual functions ✓

Answer: Option D — Using Inheritance and Virtual functions

Q639. Out of the following, which is not a member of the class?

A. Static function

B. Friend function ✓

C. Constant function

D. Virtual function

Answer: Option B — Friend function

Q640. What is the correct syntax of declaring array of pointers of integers of size 10 in C++?

A. int arr = new int[10];

B. int **arr = new int*[10]; ✓

C. int *arr = new int[10];

D. int *arr = new int*[10];

Answer: Option B — int **arr = new int*[10];

Q641. Which of the following explains Polymorphism?

A. int func(int, int); float func1(float, float);

B. int func(int); int func(int);

C. int func(float); float func(int, int, char); ✓

D. int func(); int new_func();

Answer: Option C — int func(float); float func(int, int, char);

Q642. Which of the following feature of OOPs is not used in the following C++ code? class A { int i; public: void print(){cout<<"hello"<<i;} } class B: public A { int j; public: void assign(int a){j = a;} }

A. Abstraction

B. Encapsulation

C. Inheritance

D. Polymorphism ✓

Answer: Option D — Polymorphism

Q643. Why references are different from pointers?

A. A reference cannot be made null

B. A reference cannot be changed once initialized

C. No extra operator is needed for dereferencing of a reference

D. All of the mentioned ✓

Answer: Option D — All of the mentioned

Q644. Which of the following explains the overloading of functions?

A. Virtual polymorphism

B. Transient polymorphism

C. Ad-hoc polymorphism ✓

D. Pseudo polymorphism

Answer: Option C — Ad-hoc polymorphism

Q645. What is the difference between delete and delete[] in C++?

A. delete is used to delete normal objects whereas delete[] is used to pointer objects

B. delete is a keyword whereas delete[] is an identifier

C. delete is used to delete single object whereas delete[] is used to multiple(array/pointer of) objects ✓

D. delete is syntactically correct but delete[] is wrong and hence will give an error if used in any case

Answer: Option C — delete is used to delete single object whereas delete[] is used to multiple(array/pointer of) objects

---

Q646. Which of the following is correct?

A. Base class pointer object cannot point to a derived class object

B. Derived class pointer object cannot point to a base class object ✓

C. A derived class cannot have pointer objects

D. A base class cannot have pointer objects

Answer: Option B — Derived class pointer object cannot point to a base class object

Q647. What is the other name used for functions inside a class?

A. Member variables

B. Member functions ✓

C. Class functions

D. Class variables

Answer: Option B — Member functions

Q648. Which of the following is an abstract data type?

A. int

B. float

C. class ✓

D. string

Answer: Option C — class

Q649. What happens if the following C++ statement is compiled and executed? int *ptr = NULL; delete ptr;

A. The program compiled successfully but throws an error during run-time

B. The program gives a compile-time error

C. The program is not semantically correct

D. The program is compiled and executed successfully ✓

Answer: Option D — The program is compiled and executed successfully

Q650. What will be the output of the following C++ code? #include <iostream> using namespace std; class A { int a; A() { a = 5;} }; int main() { A *obj = new A; cout << obj->a; }

A. 5

B. Garbage value

C. Compile-time error ✓

D. Run-time error

Answer: Option C — Compile-time error

Q651. Which of the following is not a type of Constructor?

A. Friend constructor ✓

B. Copy constructor

C. Default constructor

D. Parameterized constructor

Answer: Option A — Friend constructor

Q652. Which of the following cannot be a friend?

A. Function

B. Class

C. Object ✓

D. Operator function

Answer: Option C — Object

Q653. How access specifiers in Class helps in Abstraction?

A. They does not helps in any way

B. They allows us to show only required things to outer world ✓

C. They help in keeping things together

D. Abstraction concept is not used in classes

Answer: Option B — They allows us to show only required things to outer world

Q654. Which of the following approach is used by C++?

A. Top-down

B. Bottom-up ✓

C. Left-right

D. Right-left

Answer: Option B — Bottom-up

Q655. Which of the following provides a programmer with the facility of using object of a class inside other classes?

A. Inheritance

B. Composition ✓

C. Abstraction

D. Encapsulation

Answer: Option B — Composition

Q656. Which concept means the addition of new components to a program as it runs?

A. Data hiding

B. Dynamic binding

C. Dynamic loading ✓

D. Dynamic typing

Answer: Option C — Dynamic loading

Q657. What does polymorphism in OOPs mean?

A. Concept of allowing overiding of functions ✓

B. Concept of hiding data

C. Concept of keeping things in differnt modules/files

D. Concept of wrapping things into a single unit

Answer: Option A — Concept of allowing overiding of functions

Q658. What will be the output of the following C++ code? #include <iostream> using namespace std; class A{ public: A(){ cout<<"Constructor called\n"; } ~A(){ cout<<"Destructor called\n"; } }; int main(int argc, char const *argv[]) { A *a = new A[5]; delete a; return 0; }

A. "Constructor called" five times and then "Destructor called" five times

B. "Constructor called" five times and then "Destructor called" once

C. Error

D. Segmentation fault ✓

Answer: Option D — Segmentation fault

Q659. Wrapping data and its related functionality into a single entity is known as . . . . . .

A. Abstraction

B. Encapsulation ✓

C. Polymorphism

D. Modularity

Answer: Option B — Encapsulation

Q660. What will be the output of the following C++ code? #include<iostream> using namespace std; class Base { public: Base() { cout<<"Constructing Base \n"; } ~Base() { cout<<"Destructing

Base \n"; } }; class Derived: public Base { public: Derived() { cout<<"Constructing Derived \n"; } ~Derived() { cout<<"Destructing Derived \n"; } }; int main(void) { Derived *d = new Derived(); Base *b = d; delete b; return 0; }

A. Constructing Base Constructing Derived Destructing Base ✓

B. Constructing Base Constructing Derived Destructing Derived Destructing Base

C. Constructing Base Constructing Derived Destructing Base Destructing Derived

D. Constructing Derived Constructing Base Destructing Base Destructing Derived

Answer: Option A — Constructing Base Constructing Derived Destructing Base

---

Q661. C++ is . . . . . .

A. procedural programming language

B. object oriented programming language

C. functional programming language

D. both procedural and object oriented programming language ✓

Answer: Option D — both procedural and object oriented programming language

Q662. What will be the output of the following C++ code? #include <iostream> using namespace std; class A{ public: A(){ cout<<"Constructor called\n"; } ~A(){ cout<<"Destructor called\n"; } }; int main(int argc, char const *argv[]) { A *a = new A[5]; delete[] a; return 0; }

A. "Constructor called" five times and then "Destructor called" five times ✓

B. "Constructor called" five times and then "Destructor called" once

C. Error

D. Segmentation fault

Answer: Option A — "Constructor called" five times and then "Destructor called" five times

Q663. How compile-time polymorphisms are implemented in C++?

A. Using Inheritance

B. Using Virtual functions

C. Using Templates ✓

D. Using Inheritance and Virtual functions

Answer: Option C — Using Templates

Q664. What does modularity mean?

A. Hiding part of program

B. Subdividing program into small independent parts ✓

C. Overriding parts of program

D. Wrapping things into single unit

Answer: Option B — Subdividing program into small independent parts

Q665. How many types of polymorphism are there in C++?

A. 1

B. 2 ✓

C. 3

D. 4

Answer: Option B — 2

Q666. Which of the following shows multiple inheritances?

A. A->B->C

B. A->B; A->C

C. A,B->C ✓

D. B->A

Answer: Option C — A,B->C

Q667. What happens if a pointer is deleted twice in a program as shown in the following C++ statements? int *ptr = new int; delete ptr; delete ptr;

A. Undefined behaviour ✓

B. Syntactically incorrect

C. Semantically incorrect

D. The program runs perfectly

Answer: Option A — Undefined behaviour

Q668. Which of the following is correct about new and malloc?

A. Both are available in C

B. Pointer object initialization of a class with both new and malloc calls the constructor of that class

C. Pointer object initialization of a class using new involves constructor call whereas using malloc does not involve constructor call ✓

D. Pointer object initialization of a class using malloc involves constructor call whereas using new does not involve constructor call

Answer: Option C — Pointer object initialization of a class using new involves constructor call whereas using malloc does not involve constructor call

Classes And Objects In C++

Q669. What is a class in C++?

A. A collection of functions

B. A reserved keyword in C++

C. A blueprint for creating objects ✓

D. A way to declare arrays of data

Answer: Option C — A blueprint for creating objects

Q670. Which keyword is used to create an object of a class in C++?

A. new

B. create

C. object

D. None of the above ✓

Answer: Option D — None of the above

Q671. What is an object in C++?

A. A reserved keyword in C++

B. An instance of a class ✓

C. A way to declare arrays of data

D. None of the above

Answer: Option B — An instance of a class

Q672. What is the purpose of a constructor in C++?

A. To hide the implementation details of a class

B. To create multiple instances of a class

C. To initialize the object's data members ✓

D. To destroy the object

Answer: Option C — To initialize the object's data members

Q673. Which access specifier allows members of a class to be accessed from outside the class in C++?

A. private

B. protected

C. friend

D. public ✓

Answer: Option D — public

Q674. What is the correct syntax to define a member function outside the class in C++?

A. returnType functionName() {}

B. returnType className::functionName() {} ✓

C. returnType className::functionName()

D. returnType functionName(className) {}

Answer: Option B — returnType className::functionName() {}

Q675. What is the process of combining data and functions into a single unit called in C++?

A. Encapsulation ✓

B. Inheritance

C. Polymorphism

D. Abstraction

Answer: Option A — Encapsulation

Q676. What does the 'this' pointer refer to in C++?

A. Pointer to the base class object instance

B. Pointer to the next object instance

C. Pointer to the previous object instance

D. Pointer to the current object instance ✓

Answer: Option D — Pointer to the current object instance

Q677. Which keyword is used to declare a member function of a class as constant in C++?

A. static

B. final

C. const ✓

D. virtual

Answer: Option C — const

Q678. What is the purpose of the 'static' keyword in C++ class members?

A. To initialize objects of a class

B. To create a single copy of the member shared by all objects ✓

C. To create multiple instances of a class

D. None of the above

Answer: Option B — To create a single copy of the member shared by all objects

Q679. Which type of function is used to destroy objects in C++?

A. Friend function

B. Member function

C. Destructor ✓

D. Constructor

Answer: Option C — Destructor

Q680. What is the default access specifier for members of a class in C++?

A. public

B. protected

C. None of the above

D. private ✓

Answer: Option D — private

Q681. Which operator is used to access members of an object in C++?

A. Dot operator (.) ✓

B. Arrow operator (->)

C. Double colon (::)

D. Star operator (*)

Answer: Option A — Dot operator (.)

Q682. What is the difference between a constructor and a destructor in C++?

A. Constructor is called when an object is destroyed, while destructor is called when an object is created

B. Constructor is used to initialize objects, while destructor is used to free resources

C. Constructor is called when an object is created, while destructor is called when an object is destroyed ✓

D. Constructor is used to free resources, while destructor is used to initialize objects

Answer: Option C — Constructor is called when an object is created, while destructor is called when an object is destroyed

Q683. What is the purpose of the destructor in C++?

A. To define objects within a class

B. To release resources acquired by the object ✓

C. To perform calculations with object data

D. None of the above

Answer: Option B — To release resources acquired by the object

Q684. Which keyword is used to declare a class in C++?

A. object

B. typedef

C. class ✓

D. struct

Answer: Option C — class

Q685. What is the purpose of the static member variable in C++?

A. To initialize objects of a class

B. To define objects within a class

C. To release resources acquired by the object

D. To maintain common data for all objects ✓

Answer: Option D — To maintain common data for all objects

Q686. Which operator is used to access members of a class through a pointer in C++?

A. Arrow operator (->) ✓

B. Dot operator (.)

C. Double colon (::)

D. Star operator (*)

Answer: Option A — Arrow operator (->)

Q687. What is the purpose of using friend functions in C++?

A. To create multiple instances of a class

B. To hide the implementation details of a class

C. To allow access to private and protected members of a class from outside the class ✓

D. To define functions within a class

Answer: Option C — To allow access to private and protected members of a class from outside the class

Q688. Which access specifier allows members of a class to be accessed only by the derived classes in C++?

A. public

B. protected ✓

C. friend

D. None of the above

Answer: Option B — protected

Q689. What is the term for a function that is declared inside a class but defined outside of it in C++?

A. Inline function

B. Friend function

C. Member function ✓

D. Static function

Answer: Option C — Member function

Q690. What is the process of wrapping data and functions into a single unit called in C++?

A. Inheritance

B. Polymorphism

C. Abstraction

D. Encapsulation ✓

Answer: Option D — Encapsulation

Q691. What is the purpose of the 'this' pointer in C++?

A. Pointer to the previous object instance

B. Pointer to the next object instance

C. Pointer to the current object instance ✓

D. Pointer to the base class object instance

Answer: Option C — Pointer to the current object instance

Q692. What is the purpose of using constructors in C++?

A. To hide the implementation details of a class

B. To create multiple instances of a class

C. To initialize the object's data members ✓

D. To destroy the object

Answer: Option C — To initialize the object's data members

Q693. What is a constructor in C++?

A. A function used to destroy objects

B. A function used to create objects

C. A function used to access class members

D. A special member function that initializes objects ✓

Answer: Option D — A special member function that initializes objects

Q694. Which keyword is used to create a copy constructor in C++?

A. copy

B. constructor

C. copy_constructor

D. None of the above ✓

Answer: Option D — None of the above

Q695. What is the purpose of a copy constructor in C++?

A. To hide the implementation details of a class

B. To destroy objects

C. To initialize an object using another object ✓

D. To create multiple instances of a class

Answer: Option C — To initialize an object using another object

---

Q696. Which access specifier allows members of a class to be accessed only by member functions of the same class and its derived classes in C++?

A. protected ✓

B. public

C. private

D. friend

Answer: Option A — protected

Q697. What happens if a class does not explicitly declare any constructors in C++?

A. A compilation error occurs

B. The class cannot be instantiated

C. The class is initialized with default values

D. A default constructor is provided by the compiler ✓

Answer: Option D — A default constructor is provided by the compiler

Q698. What is a destructor in C++?

A. A special member function that destroys objects ✓

B. A function used to create objects

C. A function used to initialize objects

D. A function used to access class members

Answer: Option A — A special member function that destroys objects

Q699. Which keyword is used to make a member function constant in C++?

A. virtual

B. final

C. static

D. const ✓

Answer: Option D — const

Q700. What is the difference between a constructor and a normal member function in C++?

A. Constructor is called explicitly when an object is created, while a member function is called automatically

B. Constructor has a return type, while a member function does not

C. Constructor is called automatically when an object is created, while a member function needs to be called explicitly ✓

D. Constructor can be overloaded, while a member function cannot

Answer: Option C — Constructor is called automatically when an object is created, while a member function needs to be called explicitly

---

Q701. What is the purpose of the keyword 'explicit' before a constructor in C++?

A. To specify the return type of the constructor

B. To prevent implicit conversions during object initialization ✓

C. To mark the constructor as static

D. None of the above

Answer: Option B — To prevent implicit conversions during object initialization

Q702. Which keyword is used to prevent inheritance in C++?

A. sealed

B. virtual

C. final ✓

D. override

Answer: Option C — final

Q703. What is the access specifier that allows members of a class to be accessed from anywhere in the program in C++?

A. public ✓

B. private

C. protected

D. internal

Answer: Option A — public

Q704. What is the difference between a shallow copy and a deep copy in C++?

A. Shallow copy is performed using copy constructors, while deep copy is performed using assignment operators

B. Shallow copy copies only the memory addresses of the objects, while deep copy copies the actual content ✓

C. Shallow copy is faster than deep copy

D. None of the above

Answer: Option B — Shallow copy copies only the memory addresses of the objects, while deep copy copies the actual content

Q705. What will be the output of the following C++ code? #include <iostream> #include <complex> using namespace std; int main () { complex<double> mycomplex (20.0, 2.0); cout << imag(mycomplex) << endl; return 0; }

A. 2 ✓

B. 20

C. 40

D. 30

Answer: Option A — 2

Q706. How many parameters does a conversion operator may take?

A. 0 ✓

B. 1

C. 2

D. as many as possible

Answer: Option A — 0

Q707. What is operator overloading in C++?

A. Overriding the operator meaning by the user defined meaning for user defined data type

B. Redefining the way operator works for user defined types

C. Ability to provide the operators with some special meaning for user defined data type

D. All of the mentioned ✓

Answer: Option D — All of the mentioned

Q708. What will be the output of the following C++ code? #include <iostream> using namespace std; class Integer { int i; public: Integer(int ii) : i(ii) {} const Integer operator+(const Integer& rv) const { cout << "operator+" << endl; return Integer(i + rv.i); } Integer& operator+=(const Integer& rv) { cout << "operator+=" << endl; i += rv.i; return *this; } }; int main() { int i = 1, j = 2, k = 3; k += i

+ j; Integer ii(1), jj(2), kk(3); kk += ii + jj; }

A. operator+ operator+= ✓

B. operator+= operator+

C. operator+ operator+

D. operator+ operator=

Answer: Option A — operator+ operator+=

Q709. Which of the following operator cannot be overloaded?

A. +

B. ?: ✓

C. –

D. %

Answer: Option B — ?:

Q710. Pick the correct statement.

A. Macros and inline functions are same thing

B. Macros looks like function calls but they are actually not ✓

C. Inline functions looks like function but they are not

D. Inline function are always large

Answer: Option B — Macros looks like function calls but they are actually not

---

Q711. Which is used to define the member of a class externally?

A. :

B. :: ✓

C. #

D. !!$

Answer: Option B — ::

Q712. What will be the output of the following C++ code? #include <iostream> using namespace std; ostream & operator<<(ostream & i, int n) { return i; } int main() { cout << 5 << endl; cin.get(); return 0; }

A. 5

B. 6

C. error ✓

D. runtime error

Answer: Option C — error

Q713. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class Box{ int capacity; bool operator<(Box b){ return this->capacity < b.capacity ? true : false; } public: Box(){} Box(double capacity){ this->capacity = capacity; } }; int main(int argc, char const *argv[]) { Box b1(10); Box b2 = Box(14); if(b1 < b2){ cout<<"Box 2 has large capacity."; } else{ cout<<"Box 1 has large capacity."; } return 0; }

A. Error ✓

B. Segmentation fault

C. Box 2 has large capacity

D. No output

Answer: Option A — Error

Q714. Which operator a pointer object of a class uses to access its data members and member functions?

A. .

B. -> ✓

C. :

D. ::

Answer: Option B — ->

Q715. What will be the output of the following C++ code? #include <iostream> using namespace std; int main () { int a, b; a = 10; b = 4; a = b; b = 7; cout << "a:"; cout << a; cout << " b:"; cout << b; return 0; }

A. a:4 b:7 ✓

B. a:10 b:4

C. a:4 b:10

D. a:4 b:6

Answer: Option A — a:4 b:7

Q716. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main () { std::string str ("Example."); str.back() = '!'; std::cout << str << endl; return 0; }

A. Example.!

B. Example.

C. Example! ✓

D. Example!.

Answer: Option C — Example!

Q717. Which is the correct example of a unary operator?

A. &

B. ==

C. — ✓

D. /

Answer: Option C — —

Q718. In case of non-static member functions how many maximum object arguments a binary operator overloaded function can take?

A. 1 ✓

B. 2

C. 3

D. 0

Answer: Option A — 1

Q719. Which of the following operator can be overloaded?

A. ?:

B. ::

C. .

D. == ✓

Answer: Option D — ==

Q720. How to store the large objects in c++ if it extends its allocated memory?

A. memory heap ✓

B. stack

C. queue

D. stack & queue

Answer: Option A — memory heap

Q721. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main () { string str ("microsoft"); string::reverse_iterator r; for (r = str.rbegin() ; r < str.rend(); r++) cout << *r; return 0; }

A. microsoft

B. micro

C. tfosorcim ✓

D. tfos

Answer: Option C — tfosorcim

Q722. How to declare the complex number?

A. (3, 4)

B. complex(3, 4) ✓

C. (3, 4i)

D. (3, 4g)

Answer: Option B — complex(3, 4)

Q723. Given the following C++ code. How would you define the < operator for Box class so that when boxes b1 and b2 are compared in if block the program gives correct result? #include <iostream> #include <string> using namespace std; class Box { int capacity; public: Box(){} Box(double capacity){ this->capacity = capacity; } }; int main(int argc, char const *argv[]) { Box b1(10); Box b2 = Box(14); if(b1 < b2){ cout<<"Box 2 has large capacity."; } else{ cout<<"Box 1 has large capacity."; } return 0; }

A. bool operator<(Box b) { return this->capacity < b.capacity ? true : false; } ✓

B. bool operator<(Box b) { return this->capacity > b.capacity ? true : false; }

C. bool operator<(Box b) { return b1 > b2 ? true : false; }

D. bool operator<(Box b) { return this < b ? true : false; }

Answer: Option A — bool operator<(Box b) { return this->capacity < b.capacity ? true : false; }

Q724. What will be the output of the following C++ code? #include <iostream> using namespace std; class sample { public: sample(int i) : m_i(i) { } public: int operator()(int i = 0) const { return m_i + i; } operator int () const { return m_i; } private: int m_i; friend int g(const sample&); }; int f(char c) { return c; } int main() { sample f(2); cout << f(2); return 0; }

A. 3

B. 4 ✓

C. 5

D. 6

Answer: Option B — 4

Q725. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main () { string str ("example."); str.front() = 'E'; cout << str << endl; return 0; }

A. Example

B. Example. ✓

C. example

D. example.

Answer: Option B — Example.

Q726. Which container in c++ will take large objects?

A. string

B. class

C. vector ✓

D. string & class

Answer: Option C — vector

Q727. How many types of models are available to create the user-defined data type?

A. 1

B. 2 ✓

C. 3

D. 4

Answer: Option B — 2

Q728. Which concepts does the Pre Increment use?

A. call by value

B. call by reference ✓

C. queue

D. call by name

Answer: Option B — call by reference

Q729. Which is the correct example of a binary operator?

A. ++

B. —

C. Dereferencing operator(*)

D. + ✓

Answer: Option D — +

Q730. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main () { string str = "HelloWorld!"; cout<<str.capacity(); cout<<str.size(); return 0; }

A. 1511 ✓

B. 1111

C. 1115

D. 010

Answer: Option A — 1511

Q731. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstring> using namespace std; int main(int argc, char const *argv[]) { string s("a"); cout<<s; return 0; }

A. a ✓

B. empty string

C. Error

D. Segmentation fault

Answer: Option A — a

Q732. Which of the following operator can be used to overload when that function is declared as friend function?

A. []

B. ()

C. ->

D. |= ✓

Answer: Option D — |=

Q733. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main(int argc, char const *argv[]) { char s1[6] = "Hello"; char s2[6] = "World"; char s3[12] = s1 + " " + s2; cout<<s3; return 0; }

A. Hello World

B. Hello

C. World

D. Error ✓

Answer: Option D — Error

Q734. The data members and functions of a class in C++ are by default . . . . . .

A. protected

B. private ✓

C. public

D. public & protected

Answer: Option B — private

Q735. How many arguments will the subscript operator will take for overloading?

A. 1 ✓

B. 2

C. 0

D. as many as possible

Answer: Option A — 1

Q736. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A { static int a; public: A() { cout<<"Object of A is created\n"; } void show() { a++; cout<<"a: "<<a<<endl; } }; class B { public: }; int main(int argc, char const *argv[]) { A a1, a2; A a3 = a1 + a2; return 0; }

A. Runs perfectly

B. Run-time Error

C. Segmentation fault

D. Compile-time Error ✓

Answer: Option D — Compile-time Error

Q737. #include <iostream> using namespace std; class three_d { int x, y, z; public: three_d() { x = y = z = 0; } three_d(int i, int j, int k) { x = i; y = j; z = k; } three_d operator()(three_d obj); three_d operator()(int a, int b, int c); friend ostream &operator<<(ostream &strm, three_d op); }; three_d three_d::operator()(three_d obj) { three_d temp; temp.x = (x + obj.x) / 2; temp.y = (y + obj.y) / 2; temp.z = (z + obj.z) / 2; return temp; } three_d three_d::operator()(int a, int b, int c) { three_d temp; temp.x = x + a; temp.y = y + b; temp.z = z + c; return temp; } ostream &operator<<(ostream &strm, three_d op) { strm << op.x << ", " << op.y << ", " << op.z << endl; return strm; } int main() { three_d objA(1, 2, 3), objB(10, 10, 10), objC; objC = objA(objB(100, 200, 300)); cout << objC;

return 0; }

A. 55, 106, 156 ✓

B. 55, 106

C. 55, 106, 159

D. 55, 106, 158

Answer: Option A — 55, 106, 156

Q738. What will be the output of the following C++ code? #include <iostream> using namespace std; class sample { private: int* i; int j; public: sample (int j); ~sample (); int& operator [] (int n); }; int& sample::operator [] (int n) { return i[n]; } sample::sample (int j) { i = new int [j]; j = j; } sample::~sample () { delete [] i; } int main () { sample m (5); m [0] = 25; m [1] = 20; m [2] = 15; m [3] = 10; m [4] = 5; for (int n = 0; n < 5; ++ n) cout << m [n]; return 0; }

A. 252015105 ✓

B. 510152025

C. 51015

D. 51015210

Answer: Option A — 252015105

Q739. How many specifiers are present in access specifiers in class?

A. 1

B. 2

C. 3 ✓

D. 4

Answer: Option C — 3

Q740. Which special character is used to mark the end of class?

A. ; ✓

B. :

C. #

D. $

Answer: Option A — ;

Q741. What will be the output of the following C++ code? #include <stdio.h> #include<iostream> using namespace std; int main() { int x = 5, y = 5; cout << ++x << --y << endl; return 0; }

A. 55

B. 64 ✓

C. 46

D. 45

Answer: Option B — 64

Q742. How to access the object in the class?

A. scope resolution operator

B. ternary operator

C. direct member access operator ✓

D. resolution operator

Answer: Option C — direct member access operator

Q743. What will be the output of the following C++ code? #include <iostream> using namespace std; class base { int val1, val2; public: int get() { val1 = 100; val2 = 300; } friend float mean(base

ob); }; float mean(base ob) { return float(ob.val1 + ob.val2) / 2; } int main() { base obj; obj.get(); cout << mean(obj); return 0; }

A. 200 ✓

B. 150

C. 100

D. 300

Answer: Option A — 200

Q744. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class complex { int i; int j; public: complex(int a, int b) { i = a; j = b; } complex operator+(complex c) { complex temp; temp.i = this->i + c.i; temp.j = this->j + c.j; return temp; } void show(){ cout<<"Complex Number: "<<i<<" + i"<<j<<endl; } }; int main(int argc, char const *argv[]) { complex c1(1,2); complex c2(3,4); complex c3 = c1 + c2; c3.show(); return 0; }

A. 4 + i6

B. 2 + i2

C. Error ✓

D. Segmentation fault

Answer: Option C — Error

Q745. In case of non-static member functions how many maximum object arguments a unary operator overloaded function can take?

A. 1

B. 2

C. 3

D. 0 ✓

Answer: Option D — 0

Q746. What will be the output of the following C++ code? #include <iostream> using namespace std; class sample1 { float i, j; }; class sample2 { int x, y; public: sample2 (int a, int b) { x = a; y = b; } int result() { return x + y; } }; int main () { sample1 d; sample2 * padd; padd = (sample2*) &d; cout<< padd->result(); return 0; }

A. 20

B. runtime error

C. random number

D. runtime error or random number ✓

Answer: Option D — runtime error or random number

Q747. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstring> using namespace std; int main(int argc, char const *argv[]) { const char *a = "Hello\0World"; cout<<a; return 0; }

A. Hello World

B. Hello ✓

C. World

D. Error

Answer: Option B — Hello

Q748. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A { static int a; public: void change(int i){ a = i; } void value_of_a(){ cout<<a; } }; int A::a = 5; int main(int argc, char const *argv[]) { A a1 = A(); A a2 = A(); A a3 = A();

a1.change(10); a1.value_of_a(); a2.value_of_a(); a3.value_of_a(); return 0; }

A. 1055

B. 555

C. 101010 ✓

D. 51010

Answer: Option C — 101010

Q749. Inline functions are avoided when . . . . . .

A. function contains static variables

B. function have recursive calls

C. function have loops

D. all of the mentioned ✓

Answer: Option D — all of the mentioned

Q750. What will be the output of the following C++ code? #include <iostream> using namespace std; class rect { int x, y; public: void val (int, int); int area () { return (x * y); } }; void rect::val (int a, int b) { x = a; y = b; } int main () { rect rect; rect.val (3, 4); cout << "rect area: " << rect.area(); return 0; }

A. rect area: 24

B. rect area: 12 ✓

C. compile error because rect is as used as class name and variable name in line #20

D. rect area: 56

Answer: Option B — rect area: 12

Q751. What is a friend function in C++?

A. A function which can access all the private, protected and public members of a class ✓

B. A function which is not allowed to access any member of any class

C. A function which is allowed to access public and protected members of a class

D. A function which is allowed to access only public members of a class

Answer: Option A — A function which can access all the private, protected and public members of a class

Q752. Pick the correct statement.

A. Friend functions are in the scope of a class

B. Friend functions can be called using class objects

C. Friend functions can be invoked as a normal function ✓

D. Friend functions can access only protected members not the private members

Answer: Option C — Friend functions can be invoked as a normal function

Q753. What will be the output of the following C++ code? #include <iostream> using namespace std; class Box { double length; double breadth; double height; public: double getVolume(void) { return length * breadth * height; } void setLength(double len) { length = len; } void setBreadth(double bre) { breadth = bre; } void setHeight(double hei) { height = hei; } Box operator+(const Box& b) { Box box; box.length = this->length + b.length; box.breadth = this->breadth + b.breadth; box.height = this->height + b.height; return box; } }; int main( ) { Box Box1; Box Box2; Box Box3; double volume = 0.0; Box1.setLength(6.0); Box1.setBreadth(7.0); Box1.setHeight(5.0); Box2.setLength(12.0); Box2.setBreadth(13.0); Box2.setHeight(10.0); volume = Box1.getVolume(); cout << "Volume of Box1 : " << volume <<endl; volume = Box2.getVolume(); cout << "Volume of Box2 : " << volume <<endl; Box3 = Box1 + Box2; volume = Box3.getVolume(); cout << "Volume

of Box3 : " << volume <<endl; return 0; }

A. Volume of Box1 : 210 Volume of Box2 : 1560 Volume of Box3 : 5400 ✓

B. Volume of Box1 : 200 Volume of Box2 : 1560 Volume of Box3 : 5400

C. Volume of Box1 : 210 Volume of Box2 : 1550 Volume of Box3 : 5400

D. Volume of Box1 : 200 Volume of Box2 : 1000 Volume of Box3 : 5260

Answer: Option A — Volume of Box1 : 210 Volume of Box2 : 1560 Volume of Box3 : 5400

Q754. How to declare operator function?

A. operator sign ✓

B. operator

C. name of the operator

D. name of the class

Answer: Option A — operator sign

Q755. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main () { string str ("Ubuntu"); cout << str.capacity(); cout << str.max_size(); return 0; }

A. 61073741820

B. 51073741820

C. 6 and max size depends on compiler

D. 15 9223372036854770000 ✓

Answer: Option D — 15 9223372036854770000

Q756. What is the use of functor?

A. It makes the object "callable" like a function ✓

B. It makes the class "callable" like a function

C. It makes the attribute "callable" like a function

D. It makes the argument "callable" like a function

Answer: Option A — It makes the object "callable" like a function

Q757. What will be the output of the following C++ code? #include <iostream> using namespace std; void duplicate (int& a, int& b, int& c) { a *= 2; b *= 2; c *= 2; } int main () { int x = 1, y = 3, z = 7; duplicate (x, y, z); cout << x << y << z; return 0; }

A. 1468

B. 2812

C. 2614 ✓

D. 2713

Answer: Option C — 2614

Q758. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int a = 20; int b = 10; int c = 15; int d = 5; int e; e = a + b * c / d; cout << e << endl ; return 0; }

A. 50 ✓

B. 60

C. 70

D. 90

Answer: Option A — 50

Q759. Which is used to tell the computer that where a pointer is pointing to?

A. dereference ✓

B. reference

C. heap operations

D. binary operations

Answer: Option A — dereference

Q760. What will be the output of the following C++ code? #include <iostream> #include <cmath> using namespace std; class Complex { private: double real; double imag; public: Complex(double r = 0.0, double i = 0.0) : real(r), imag(i) {} double mag() { return getMag(); } operator double () { return getMag(); } private: double getMag() { return sqrt(real * real + imag * imag); } }; int main() { Complex com(3.0, 4.0); cout << com.mag(); cout << com; return 0 }

A. 5 5 ✓

B. 4 5

C. 6 6

D. 7 5

Answer: Option A — 5 5

Q761. What is the scope of typedef defined data types?

A. inside that block only

B. whole program ✓

C. outside the program

D. main function

Answer: Option B — whole program

Q762. What is the associativity of add(+);?

A. right to left

B. left to right ✓

C. right to left & left to right

D. top to bottom

Answer: Option B — left to right

Q763. Which is called ternary operator?

A. ?: ✓

B. &&

C. |||

D. ===

Answer: Option A — ?:

Q764. How the objects are self-referenced in a member function of that class.

A. Using a special keyword object

B. Using this pointer ✓

C. Using * with the name of that object

D. By passing self as a parameter in the member function

Answer: Option B — Using this pointer

Q765. What will be the output of the following C++ code? #include <iostream> using namespace std; class number { int i; public: int geti(); void puti(int j); }; int number::geti() { return i; } void number::puti(int j) { i = j; } int main() { number s; s.puti(10); cout << s.geti( ); return 0; }

A. 10 ✓

B. 11

C. 20

D. 22

Answer: Option A — 10

Q766. How many objects can present in a single class?

A. 1

B. 2

C. 3

D. as many as possible ✓

Answer: Option D — as many as possible

Q767. How many member functions are there in this C++ class excluding constructors and destructors? class Box { int capacity; public: void print(); friend void show(); bool compare(); friend bool lost(); };

A. 1

B. 2 ✓

C. 3

D. 4

Answer: Option B — 2

Q768. In which form does the function call operator can be overloaded?

A. static member function

B. non-static member function ✓

C. dynamis_cast

D. static_cast

Answer: Option B — non-static member function

Constructors And Destructors In C++

Q769. What is a constructor in C++?

A. A function used to create objects

B. A special member function that initializes objects ✓

C. A function used to access class members

D. None of the above

Answer: Option B — A special member function that initializes objects

Q770. Which access specifier allows constructors to be called from anywhere in the program in C++?

A. friend

B. protected

C. public ✓

D. private

Answer: Option C — public

Q771. What happens if a class does not explicitly declare any constructors in C++?

A. A compilation error occurs

B. The class cannot be instantiated

C. The class is initialized with default values

D. A default constructor is provided by the compiler ✓

Answer: Option D — A default constructor is provided by the compiler

Q772. What is the purpose of a destructor in C++?

A. A special member function that destroys objects ✓

B. A function used to create objects

C. A function used to initialize objects

D. A function used to access class members

Answer: Option A — A special member function that destroys objects

Q773. What is the purpose of the 'explicit' keyword before a constructor in C++?

A. To mark the constructor as static

B. To specify the return type of the constructor

C. To allow implicit conversions during object initialization

D. To prevent implicit conversions during object initialization ✓

Answer: Option D — To prevent implicit conversions during object initialization

Q774. Which keyword is used to prevent inheritance in C++?

A. override

B. virtual

C. final ✓

D. sealed

Answer: Option C — final

Q775. What is the difference between a shallow copy and a deep copy in C++?

A. Shallow copy is performed using copy constructors, while deep copy is performed using assignment operators

B. Shallow copy copies only the memory addresses of the objects, while deep copy copies the actual content ✓

C. Shallow copy is faster than deep copy

D. None of the above

Answer: Option B — Shallow copy copies only the memory addresses of the objects, while deep copy copies the actual content

---

Q776. What is the process of hiding the implementation details of a class and showing only the necessary features to the outside world called in C++?

A. Abstraction ✓

B. Polymorphism

C. Encapsulation

D. Inheritance

Answer: Option A — Abstraction

Q777. What is a constructor initialization list in C++?

A. A list of destructor calls

B. A list of initializations for member variables ✓

C. A list of default arguments for the constructor

D. None of the above

Answer: Option B — A list of initializations for member variables

Q778. What is the access specifier that allows members of a class to be accessed from anywhere in the program in C++?

A. private

B. protected

C. internal

D. public ✓

Answer: Option D — public

Q779. What is the purpose of using the 'explicit' keyword in front of a constructor declaration in C++?

A. To specify the return type of the constructor

B. To prevent implicit conversions during object initialization ✓

C. To mark the constructor as static

D. None of the above

Answer: Option B — To prevent implicit conversions during object initialization

Q780. Which type of constructor is automatically called when an object is created without any arguments in C++?

A. Destructor

B. Copy constructor

C. Default constructor ✓

D. Parameterized constructor

Answer: Option C — Default constructor

Q781. Which of the following statements about destructors in C++ is true?

A. Destructors can be overloaded

B. Destructors can be inherited

C. Destructors are called when an object is created

D. Destructors cannot have parameters ✓

Answer: Option D — Destructors cannot have parameters

Q782. In C++, what is the purpose of the 'this' pointer in a class member function?

A. It points to the memory location of the current object instance ✓

B. It points to the previous object instance

C. It points to the next object instance

D. It points to the base class object instance

Answer: Option A — It points to the memory location of the current object instance

Q783. Which keyword is used to prevent a class from being inherited in C++?

A. sealed

B. virtual

C. override

D. final ✓

Answer: Option D — final

Q784. In C++, what is the difference between a shallow copy and a deep copy?

A. Shallow copy copies the actual content of the objects, while deep copy copies only the memory addresses

B. Shallow copy is faster than deep copy

C. Shallow copy copies only the memory addresses of the objects, while deep copy copies the actual content ✓

D. Shallow copy is slower than deep copy

Answer: Option C — Shallow copy copies only the memory addresses of the objects, while deep copy copies the actual content

Q785. What is the difference between a constructor and a normal member function in C++?

A. Constructor can be overloaded, while a member function cannot

B. Constructor has a return type, while a member function does not

C. Constructor is called explicitly when an object is created, while a member function is called automatically

D. Constructor is called automatically when an object is created, while a member function needs to be called explicitly ✓

Answer: Option D — Constructor is called automatically when an object is created, while a member function needs to be called explicitly

---

Q786. What happens if a class does not define any constructors in C++?

A. The class will have a compilation error

B. The class cannot be compiled

C. A default constructor is automatically provided ✓

D. The class cannot have any objects instantiated

Answer: Option C — A default constructor is automatically provided

Q787. What is the purpose of the 'static' keyword in C++ class members?

A. To hide the implementation details of a class

B. To initialize objects of a class

C. To create a single copy of the member shared by all objects ✓

D. To create multiple instances of a class

Answer: Option C — To create a single copy of the member shared by all objects

Q788. What is the purpose of the destructor in C++?

A. To initialize objects

B. To create objects

C. To release resources acquired by the object ✓

D. To access class members

Answer: Option C — To release resources acquired by the object

Q789. Which keyword is used to create a copy constructor in C++?

A. copy

B. constructor

C. copy_constructor ✓

D. None of the above

Answer: Option C — copy_constructor

Q790. What is the purpose of using constructor initialization lists in C++?

A. To override the default constructor

B. To create multiple constructors for a class

C. To initialize class members before the body of the constructor executes ✓

D. To define constructors with multiple arguments

Answer: Option C — To initialize class members before the body of the constructor executes

---

Q791. What is the destructor called automatically when an object is destroyed in C++?

A. .~className()

B. className::~()

C. ~()

D. ~className() ✓

Answer: Option D — ~className()

Q792. Which keyword is used to prevent an object from being copied in C++?

A. nocopy

B. prevent_copy

C. delete ✓

D. new

Answer: Option C — delete

Q793. What happens if a destructor is made private in a C++ class?

A. It is a syntax error

B. The destructor will still be called when needed

C. It prevents object destruction

D. The compiler will throw an error ✓

Answer: Option D — The compiler will throw an error

Q794. In C++, which type of constructor is called when an object is created without any arguments?

A. Default constructor ✓

B. Parameterized constructor

C. Copy constructor

D. Destructor

Answer: Option A — Default constructor

Q795. What is the primary purpose of the explicit keyword in a C++ constructor?

A. To mark the constructor as static

B. To specify the return type of the constructor

C. To allow implicit type conversions

D. To prevent implicit type conversions ✓

Answer: Option D — To prevent implicit type conversions

Q796. In C++, what is the order of constructor and destructor calls for a class hierarchy?

A. Constructors: Derived to Base, Destructors: Base to Derived

B. Constructors and Destructors follow the same order

C. Constructors: Base to Derived, Destructors: Derived to Base ✓

D. Constructors: Derived to Base, Destructors: Derived to Base

Answer: Option C — Constructors: Base to Derived, Destructors: Derived to Base

Q797. What happens if a constructor throws an exception in C++?

A. The destructor is called to deallocate resources

B. The object is not fully constructed ✓

C. The object is still constructed but may have undefined behavior

D. None of the above

Answer: Option B — The object is not fully constructed

Q798. In C++, which function cannot be inherited but can be overridden in a derived class?

A. Default Constructor

B. Parameterized Constructor

C. Destructor ✓

D. Copy Constructor

Answer: Option C — Destructor

Q799. Which of the following statements about constructor initialization lists in C++ is correct?

A. They cannot be used in constructors with default arguments

B. They can only be used in copy constructors

C. They can only be used for primitive data types

D. They must be used for initializing all members of a class ✓

Answer: Option D — They must be used for initializing all members of a class

Q800. Which of the following statements is true about constructors in C++?

A. Constructors can have a return type

B. Constructors cannot be overloaded ✓

C. Constructors can be called explicitly

D. Constructors can be virtual

Answer: Option B — Constructors cannot be overloaded

Q801. What is the purpose of using default arguments in C++ constructors?

A. To allow the constructor to be called without arguments

B. To provide a default value for constructor arguments ✓

C. To specify the order of constructor execution

D. To prevent the use of constructor arguments

Answer: Option B — To provide a default value for constructor arguments

Q802. Which statement about constructor inheritance in C++ is correct?

A. Constructors cannot be inherited

B. Constructors are inherited with the same access specifier ✓

C. Constructors can only be inherited privately

D. Constructors are inherited with different names

Answer: Option B — Constructors are inherited with the same access specifier

Q803. Which statement about destructors in C++ is true?

A. Destructors can be overloaded

B. Destructors cannot have any arguments ✓

C. Destructors can have a return type

D. Destructors can be called explicitly

Answer: Option B — Destructors cannot have any arguments

Q804. Which operator is used to access members of an object in C++?

A. Dot operator (.) ✓

B. Arrow operator (->)

C. Double colon (::)

D. Star operator (*)

Answer: Option A — Dot operator (.)

Q805. What is the access specifier that allows constructors to be called only from member functions of the same class and its derived classes in C++?

A. protected

B. public

C. private ✓

D. friend

Answer: Option C — private

Q806. What happens if the destructor is explicitly defined in a C++ class, but the constructor is not defined?

A. It will cause a compilation error

B. The class will be instantiated with default values

C. The destructor will not be called ✓

D. The class will not be able to create objects

Answer: Option C — The destructor will not be called

Q807. What is the purpose of using the delete keyword in C++ destructors?

A. To deallocate memory for dynamically allocated objects

B. To call the parent class destructor

C. To prevent the destructor from being called ✓

D. To explicitly call the destructor

Answer: Option C — To prevent the destructor from being called

Q808. Which of the following is correct?

A. Destructors can be virtual ✓

B. There can be more than one destructor in a class

C. Destructor definition starts with !

D. Destructor is used to initialize objects

Answer: Option A — Destructors can be virtual

Q809. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A{ mutable int a; public: A(){ cout<<"A's Constructor called\n"; } ~A(){ cout<<"A's Destructor called\n"; } }; class B{ A a; public: B(){ cout<<"B's Constructor called\n"; } ~B(){ cout<<"B's Destructor called\n"; } }; int main(int argc, char const *argv[]) { B b1; }

A. A's Constructor called B's Constructor called

B. A's Destructor called B's Destructor called

C. A's Constructor called B's Constructor called B's Destructor called A's Destructor called ✓

D. A's Constructor called B's Constructor called A's Destructor called B's Destructor called

Answer: Option C — A's Constructor called B's Constructor called B's Destructor called A's Destructor called

Q810. How constructors are different from other member functions of the class?

A. Constructor has the same name as the class itself

B. Constructors do not return anything

C. Constructors are automatically called when an object is created

D. All of the mentioned ✓

Answer: Option D — All of the mentioned

Q811. How destructor overloading is done?

A. By changing the number of parameters

B. By changing the parameters type

C. By changing both the number of parameters and their type

D. No chance for destructor overloading ✓

Answer: Option D — No chance for destructor overloading

Q812. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A{ mutable int a; public: A(){ cout<<"Default constructor called\n"; } A(const A& a){ cout<<"Copy Constructor called\n"; } }; int main(int argc, char const *argv[]) { A obj; A a1 = obj; A a2(obj); }

A. Default constructor called Copy Constructor called

B. Default constructor called Copy Constructor called Copy Constructor called ✓

C. Default constructor called Default constructor called Copy Constructor called

D. Copy Constructor called Default constructor called Copy Constructor called

Answer: Option B — Default constructor called Copy Constructor called Copy Constructor called

Q813. When destructors are called?

A. When a program ends

B. When a function ends

C. When a delete operator is used

D. All of the mentioned ✓

Answer: Option D — All of the mentioned

Q814. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A{ mutable int a; public: A(){ cout<<"A's Constructor called\n"; } ~A(){ cout<<"A's Destructor called\n"; } }; class B: public A{ public: B(){ cout<<"B's Constructor called\n"; } ~B(){ cout<<"B's Destructor called\n"; } }; int main(int argc, char const *argv[]) { B b1; }

A. A's Constructor called B's Constructor called

B. A's Destructor called B's Destructor called

C. A's Constructor called B's Constructor called B's Destructor called A's Destructor called ✓

D. A's Constructor called B's Constructor called A's Destructor called B's Destructor called

Answer: Option C — A's Constructor called B's Constructor called B's Destructor called A's Destructor called

Q815. What is the difference between constructors and destructors?

A. They have a different function name

B. Constructors does not have return type whereas destructors do have

C. Constructors allow function parameters whereas destructors do not ✓

D. Constructors does not function parameters

Answer: Option C — Constructors allow function parameters whereas destructors do not

---

Q816. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A{ mutable int a; public: A(){ cout<<"A's Constructor called\n"; } ~A(){ cout<<"A's Destructor called\n"; } }; class B{ static A a; public: B(){ cout<<"B's Constructor called\n"; } ~B(){ cout<<"B's Destructor called\n"; } }; int main(int argc, char const *argv[]) { B b1; }

A. A's Constructor called B's Constructor called

B. B's Constructor called B's Destructor called ✓

C. A's Constructor called B's Constructor called B's Destructor called A's Destructor called

D. A's Constructor called B's Constructor called A's Destructor called B's Destructor called

Answer: Option B — B's Constructor called B's Destructor called

Q817. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A{ int a; public: A(int i){ a = i; } void assign(int i){ a = i; } int return_value(){ return a; } }; int main(int argc, char const *argv[]) { A obj; obj.assign(5); cout<<obj.return_value(); }

A. 5

B. 55

C. Error ✓

D. Segmentation Fault

Answer: Option C — Error

Q818. How many types of constructors are there in C++?

A. 1

B. 2

C. 3 ✓

D. 4

Answer: Option C — 3

Q819. What will be the output of the following C++ code? #include <iostream> using namespace std; class A{ A(){ cout<<"Constructor called"; } }; int main(int argc, char const *argv[]) { A a;

return 0; }

A. Constructor called

B. Nothing printed

C. Error ✓

D. Segmentation fault

Answer: Option C — Error

Q820. What will be the output of the following C++ code? #include <iostream> using namespace std; class A{ public: int a; A(){ cout<<"Constructor called"; } }; int main(int argc, char const *argv[]) { A *a1 = (A*)malloc(sizeof(A)); return 0; }

A. Constructor called

B. Nothing printed ✓

C. Error

D. Segmentation fault

Answer: Option B — Nothing printed

Q821. Which of the following constructors are provided by the C++ compiler if not defined in a class?

A. Default constructor

B. Assignment constructor

C. Copy constructor

D. All of the mentioned ✓

Answer: Option D — All of the mentioned

Q822. What will be the output of the following C++ code? #include <iostream> using namespace std; class A{ public: A(){ cout<<"Constructor called"; } }; int main(int argc, char const *argv[]) { A *a; return 0; }

A. Constructor called

B. Nothing printed ✓

C. Error

D. Segmentation fault

Answer: Option B — Nothing printed

Q823. What will be the output of the following C++ code? #include <iostream> using namespace std; class A{ public: int a; A(int a=0){ this->a = a; } }; int main(int argc, char const *argv[]) { A a1, a2(10); cout<<a2.a; return 0; }

A. 010 ✓

B. 100

C. 001

D. Error

Answer: Option A — 010

Q824. What will be the output of the following C++ code? #include <iostream> using namespace std; class A{ public: int a; A(){ cout<<"Constructor called"; } } a; int main(int argc, char const *argv[]) { return 0; }

A. Constructor called ✓

B. Nothing printed

C. Error

D. Segmentation fault

Answer: Option A — Constructor called

Q825. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A{ mutable int a; public: A(){ cout<<"A's Constructor called\n"; } ~A(){ cout<<"A's Destructor called\n"; } }; class B{ static A a; public: B(){ cout<<"B's Constructor called\n"; } ~B(){ cout<<"B's Destructor called\n"; } }; A B::a; int main(int argc, char const *argv[]) { return 0; }

A. A's Constructor called A's Destructor called ✓

B. B's Constructor called B's Destructor called

C. A's Constructor called B's Constructor called B's Destructor called A's Destructor called

D. A's Constructor called B's Constructor called A's Destructor called B's Destructor called

Answer: Option A — A's Constructor called A's Destructor called

---

Q826. What will be the output of the following C++ code? #include <iostream> using namespace std; class A{ public: int a; A(int a){ this->a = a; } }; int main(int argc, char const *argv[]) { A a1, a2(10); cout<<a2.a; return 0; }

A. 10

B. Compile time error ✓

C. Run-time error

D. No output

Answer: Option B — Compile time error

Q827. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A{ mutable int a; public: A(){ cout<<"A's default constructor called\n"; } A(const A& a){ cout<<"A's copy Constructor called\n"; } }; class B{ A obj; public: B(){ cout<<"B's Constructor called\n"; } }; int main(int argc, char const *argv[]) { B b1; B b2; }

A. B's Constructor called B's Constructor called

B. B's Constructor called A's default constructor called B's Constructor called A's default constructor called

C. A's default constructor called B's Constructor called A's default constructor called B's Constructor called ✓

D. A's default constructor called B's Constructor called A's copy Constructor called

Answer: Option C — A's default constructor called B's Constructor called A's default constructor called B's Constructor called

Q828. Why constructors are efficient instead of a function init() defined by the user to initialize the data members of an object?

A. Because user may forget to call init() using that object leading segmentation fault

B. Because user may call init() more than once which leads to overwriting values

C. Because user may forget to define init() function

D. All of the mentioned ✓

Answer: Option D — All of the mentioned

Q829. What will be the output of the following C++ code? #include <iostream> using namespace std; class A{ A(){ cout<<"A's Constructor called\n"; } }; class B { public: A a; B(){ cout<<"B's constructor called\ns"; } }; int main(int argc, char const *argv[]) { B b; return 0; }

A. A's Constructor called B's constructor called

B. B's Constructor called A's constructor called

C. Error ✓

D. Segmentation fault

Answer: Option C — Error

Q830. What is a copy constructor?

A. A constructor that allows a user to move data from one object to another

B. A constructor to initialize an object with the values of another object ✓

C. A constructor to check the whether to objects are equal or not

D. A constructor to kill other copies of a given object.

Answer: Option B — A constructor to initialize an object with the values of another object

---

Q831. When a copy constructor is called?

A. When an object of the class is returned by value

B. When an object of the class is passed by value to a function

C. When an object is constructed based on another object of the same class

D. All of the mentioned ✓

Answer: Option D — All of the mentioned

Q832. Which of the following represents the correct explicit call to a constructor of class A? class A{ int a; public: A(int i) { a = i; } }

A. A a(5);

B. A a;

C. A a = A(5); ✓

D. A a = A();

Answer: Option C — A a = A(5);

Q833. What will be the output of the following C++ code? #include <iostream> using namespace std; class A{ ~A(){} }; class B { public: A a; }; int main(int argc, char const *argv[]) { B b; return 0; }

A. A's Constructor called B's constructor called

B. B's Constructor called A's constructor called

C. Error ✓

D. Segmentation fault

Answer: Option C — Error

Q834. How many parameters does a default constructor require?

A. 1

B. 2

C. 0 ✓

D. 3

Answer: Option C — 0

Q835. In the following C++ code how many times the string "A's constructor called" will be printed? #include <iostream> #include <string> using namespace std; class A{ int a; public: A(){ cout<<"A's constructor called"; } }; class B{ static A a; public: B(){ cout<<"B's constructor called"; } static A get(){ return a; } }; A B::a; int main(int argc, char const *argv[]) { B b; A a1 = b.get(); A a2 = b.get(); A a3 = b.get(); }

A. 3

B. 4

C. 2

D. 1 ✓

Answer: Option D — 1

Q836. What is the role of destructors in Classes?

A. To modify the data whenever required

B. To destroy an object when the lifetime of an object ends ✓

C. To initialize the data members of an object when it is created

D. To call private functions from the outer world

Answer: Option B — To destroy an object when the lifetime of an object ends

Q837. What is syntax of defining a destructor of class A?

A. A(){}

B. ~A(){} ✓

C. A::A(){}

D. ~A(){};

Answer: Option B — ~A(){}

Q838. What happens if a user forgets to define a constructor inside a class?

A. Error occurs

B. Segmentation fault

C. Objects are not created properly

D. Compiler provides a default constructor to avoid faults/errors ✓

Answer: Option D — Compiler provides a default constructor to avoid faults/errors

Q839. What will be the output of the following C++ code? #include <iostream> using namespace std; class A{ public: int a; }; int main(int argc, char const *argv[]) { A a1 = {10}; A a2 = a1; cout<<a1.a<<a2.a; return 0; }

A. 1010 ✓

B. 87368746

C. Error

D. Segmentation fault

Answer: Option A — 1010

Q840. What will be the output of the following C++ code? #include <iostream> using namespace std; class A{ public: int a; A(){ cout<<"Constructor called\n"; } } a; int main(int argc, char const *argv[]) { cout<<"Main function\n"; return 0; }

A. Constructor called Main function ✓

B. Main function Constructor called

C. Error

D. Segmentation fault

Answer: Option A — Constructor called Main function

Q841. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A{ int a; A(){ a = 5; } public: void assign(int i){ a = i; } int return_value(){ return a; } }; int main(int argc, char const *argv[]) { A obj; obj.assign(10); cout<<obj.return_value(); }

A. 5

B. 10

C. Error ✓

D. Segmentation fault

Answer: Option C — Error

Q842. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A{ mutable int a; public: A(){ cout<<"A's Constructor called\n"; } ~A(){ cout<<"A's Destructor called\n"; } }; class B: public A{ A a; public: B(){ cout<<"B's Constructor called\n"; } ~B(){ cout<<"B's Destructor called\n"; } }; int main(int argc, char const *argv[]) { B b1; }

A. A's Constructor called B's Constructor called

B. A's Destructor called B's Destructor called

C. A's Constructor called A's Constructor called B's Constructor called B's Destructor called A's Destructor called A's Destructor called ✓

D. A's Constructor called B's Constructor called A's Destructor called B's Destructor called

Answer: Option C — A's Constructor called A's Constructor called B's Constructor called B's Destructor called A's Destructor called A's Destructor called

Q843. What will be the output of the following C++ code? #include <iostream> using namespace std; class A{ A(){ cout<<"A's Constructor called\n"; } friend class B; }; class B { public: A a; B(){ cout<"B's constructor called\ns"; } }; int main(int argc, char const *argv[]) { B b; return 0; }

A. A's Constructor called B's constructor called ✓

B. B's Constructor called A's constructor called

C. Error

D. Segmentation fault

Answer: Option A — A's Constructor called B's constructor called

Q844. What is the role of a constructor in classes?

A. To modify the data whenever required

B. To destroy an object

C. To initialize the data members of an object when it is created ✓

D. To call private functions from the outer world

Answer: Option C — To initialize the data members of an object when it is created

Q845. How many Destructors are allowed in a Class?

A. 1 ✓

B. 2

C. 3

D. Any number

Answer: Option A — 1

Inheritance In C++

Q846. In C++, which type of inheritance allows a class to inherit properties and behavior from multiple base classes?

A. Multilevel inheritance
B. Hierarchical inheritance
C. Multiple inheritance ✓
D. Single inheritance
Answer: Option C — Multiple inheritance

Q847. What is the access specifier that restricts access to members of a class to only its derived classes in C++?

A. public
B. private
C. friend
D. protected ✓
Answer: Option D — protected

Q848. What is the process of defining a new class based on an existing class in C++ called?

A. Inheritance ✓
B. Composition
C. Encapsulation
D. Polymorphism
Answer: Option A — Inheritance

Q849. Which keyword is used to signify inheritance in C++?

A. extends
B. inherit
C. using
D. : public ✓
Answer: Option D — : public

Q850. In C++, which access specifier allows members of a base class to be accessible in the derived class but not to the outside world?

A. friend
B. private
C. public
D. protected ✓
Answer: Option D — protected

Q851. What is the diamond problem in C++?

A. Ambiguity in single inheritance
B. Ambiguity in hierarchical inheritance
C. Ambiguity in multiple inheritance ✓
D. Ambiguity in multilevel inheritance
Answer: Option C — Ambiguity in multiple inheritance

Q852. What is the default access specifier for members of a class in C++ when using inheritance?

A. protected

B. private ✓

C. friend

D. None of the above

Answer: Option B — private

Q853. What happens if a derived class redefines a method that is already defined in its base class in C++?

A. Compilation error

B. Both methods are available separately

C. Method in the derived class overrides the base class method ✓

D. Method in the base class overrides the derived class method

Answer: Option C — Method in the derived class overrides the base class method

Q854. In C++, which type of inheritance is not supported directly but can be achieved using a combination of other types of inheritance?

A. Multiple inheritance

B. Single inheritance

C. Hierarchical inheritance

D. Hybrid inheritance ✓

Answer: Option D — Hybrid inheritance

Q855. What is the name for the process by which a class inherits features from another class in C++?

A. Derivation ✓

B. Combination

C. Composition

D. Encapsulation

Answer: Option A — Derivation

Q856. In C++, if a derived class does not define a constructor, which constructor of the base class is called during the object creation of the derived class?

A. No constructor is called

B. Destructor of the base class

C. Parameterized constructor of the base class

D. Default constructor of the base class ✓

Answer: Option D — Default constructor of the base class

Q857. What is the process of creating a new class using an existing class as a base in C++?

A. Base class

B. Derived object

C. Derived class ✓

D. Base object

Answer: Option C — Derived class

Q858. What is the name given to the feature of C++ where a class inherits features from more than one base class?

A. Hierarchical inheritance

B. Multiple inheritance ✓

C. Multilevel inheritance

D. None of the above

Answer: Option B — Multiple inheritance

Q859. In C++, which type of inheritance allows a class to inherit from multiple classes directly or indirectly?

A. Multilevel inheritance

B. Hierarchical inheritance

C. Multiple inheritance ✓

D. Single inheritance

Answer: Option C — Multiple inheritance

Q860. What is the process of defining a new class by extending an existing class in C++?

A. Base class

B. Inherited class

C. Subclass

D. Derived class ✓

Answer: Option D — Derived class

Q861. In C++, which access specifier allows the members of a base class to be accessible in the derived class as well as from outside the class?

A. public ✓

B. private

C. protected

D. friend

Answer: Option A — public

Q862. What is the term used to describe a derived class having more than one base class in C++?

A. Multilevel inheritance

B. Hierarchical inheritance

C. Single inheritance

D. Multiple inheritance ✓

Answer: Option D — Multiple inheritance

Q863. In C++, what is the process of creating a new class from an existing class known as?

A. Composition

B. Encapsulation

C. Inheritance ✓

D. Polymorphism

Answer: Option C — Inheritance

Q864. Which keyword is used to denote the access specifier in a derived class when inheriting from a base class in C++?

A. using

B. inherits

C. extends ✓

D. : protected

Answer: Option C — extends

Q865. In C++, which type of inheritance involves inheriting from multiple base classes, but only inheriting one level deep from each base class?

A. Hierarchical inheritance

B. Multilevel inheritance ✓

C. Single inheritance

D. None of the above

Answer: Option B — Multilevel inheritance

Q866. What is the diamond problem in C++ inheritance?

A. A situation where the derived class cannot override the base class method

B. A situation where the base class cannot be inherited by more than one derived class

C. A situation where ambiguity arises due to multiple inheritance chains ✓

D. A situation where ambiguity arises due to lack of inheritance

Answer: Option C — A situation where ambiguity arises due to multiple inheritance chains

Q867. In C++, which access specifier allows the members of a base class to be accessible in the derived class and its derived classes, but not from outside the class hierarchy?

A. public

B. private

C. friend

D. protected ✓

Answer: Option D — protected

Q868. What is the default access specifier for a base class when inheriting in C++?

A. private ✓

B. public

C. protected

D. friend

Answer: Option A — private

Q869. In C++, what happens if a derived class inherits multiple base classes and each base class contains a member with the same name?

A. The member of the derived class must explicitly specify which base class member to access

B. The member of the derived class will access the member from the base class with the same name ✓

C. A compilation error occurs

D. The member of the derived class accesses all members with the same name from all base classes

Answer: Option B — The member of the derived class will access the member from the base class with the same name

Q870. Which keyword is used to prevent a class from being inherited in C++?

A. final ✓

B. sealed

C. prevent_inheritance

D. noinherit

Answer: Option A — final

Q871. In C++, what is the process of creating a new class using an existing class as a blueprint and adding new functionalities to it?

A. Derivation
B. Composition
C. Inheritance ✓
D. Polymorphism
Answer: Option C — Inheritance

Q872. What is the term used to describe a class that inherits from another class in C++?

A. Base class
B. Child class
C. Parent class
D. Derived class ✓
Answer: Option D — Derived class

Q873. In C++, what happens if a derived class redefines a method that is already defined in its base class?

A. Method in the base class overrides the derived class method
B. Both methods are available for use
C. Method in the derived class overrides the base class method ✓
D. A compilation error occurs
Answer: Option C — Method in the derived class overrides the base class method

Q874. In C++, which type of inheritance involves a class inheriting from multiple classes directly or indirectly?

A. Hierarchical inheritance
B. Multiple inheritance ✓
C. Multilevel inheritance
D. None of the above
Answer: Option B — Multiple inheritance

Q875. What is the name given to the process of creating a new class by inheriting properties and behaviors from an existing class in C++?

A. Polymorphism
B. Encapsulation
C. Inheritance ✓
D. Composition
Answer: Option C — Inheritance

Q876. In C++, which access specifier allows the members of a base class to be accessible in the derived class, but not from outside the class?

A. public
B. private
C. friend
D. protected ✓
Answer: Option D — protected

Q877. What is the term used to describe a class that inherits properties and behaviors from another class in C++?

A. Base class
B. Child class

C. Derived class ✓

D. Parent class

Answer: Option C — Derived class

Q878. In C++, what is the default access specifier for a base class when inheriting?

A. private

B. public

C. protected ✓

D. friend

Answer: Option C — protected

Q879. What is the process called when a new class is created by inheriting properties and behaviors from an existing class in C++?

A. Composition

B. Encapsulation

C. Inheritance ✓

D. Polymorphism

Answer: Option C — Inheritance

Q880. In C++, what is the process called when a class inherits properties and behaviors from multiple base classes?

A. Hierarchical inheritance

B. Multiple inheritance ✓

C. Multilevel inheritance

D. None of the above

Answer: Option B — Multiple inheritance

Q881. What happens if a derived class inherits multiple base classes and each base class contains a member with the same name in C++?

A. The member of the derived class must explicitly specify which base class member to access

B. The member of the derived class will access the member from the base class with the same name ✓

C. A compilation error occurs

D. The member of the derived class accesses all members with the same name from all base classes

Answer: Option B — The member of the derived class will access the member from the base class with the same name

Q882. In C++, which access specifier allows the members of a base class to be accessible in the derived class, its derived classes, but not from outside the class hierarchy?

A. friend

B. private

C. protected ✓

D. public

Answer: Option C — protected

Q883. In C++, which keyword is used to denote the access specifier in a derived class when inheriting from a base class?

A. using

B. inherits

C. extends ✓

D. : protected

Answer: Option C — extends

Q884. In C++, what happens if a derived class inherits from multiple base classes and each base class has a member with the same name?

A. The member of the derived class must explicitly specify which base class member to access

B. The member of the derived class will access the member from the base class with the same name ✓

C. A compilation error occurs

D. The member of the derived class accesses all members with the same name from all base classes

Answer: Option B — The member of the derived class will access the member from the base class with the same name

Q885. Which type of inheritance involves a class inheriting from multiple classes directly or indirectly in C++?

A. Multilevel inheritance

B. Hierarchical inheritance

C. Single inheritance

D. Multiple inheritance ✓

Answer: Option D — Multiple inheritance

Q886. Which specifier makes all the data members and functions of base class inaccessible by the derived class?

A. private ✓

B. protected

C. public

D. both private and protected

Answer: Option A — private

Q887. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class Mammal { public: Mammal(){ cout<<"I'm a Mammal\n"; } ~Mammal(){} }; class Human: public Mammal { public: Human(){ cout<<"I'm a Human\n"; } ~Human(){} }; class Male: public Human { public: Male(){ cout<<"I'm a Male\n"; } ~Male(){} }; class Female: public Human { public: Female(){ cout<<"I'm a Female\n"; } ~Female(){} }; int main(int argc, char const *argv[]) { Male M; return 0; }

A. I'm a Mammal I'm a Human I'm a Male ✓

B. I'm a Mammal I'm a Human I'm a Female

C. I'm a Human I'm a Male

D. I'm a Mammal I'm a Male

Answer: Option A — I'm a Mammal I'm a Human I'm a Male

Q888. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A { float d; public: virtual void func(){ cout<<"Hello this is class A\n"; } }; class B: public A { int a = 15; public: void func(){ cout<<"Hello this is class B\n"; } }; int main(int argc, char const *argv[]) { A *a; a->func(); return 0; }

A. Hello this is class A

B. Hello this is class B

C. Error

D. Segmentation Fault ✓

Answer: Option D — Segmentation Fault

Q889. Which statement is incorrect about virtual function.

A. They are used to achieve runtime polymorphism

B. They are used to hide objects ✓

C. Each virtual function declaration starts with the virtual keyword

D. All of the mentioned

Answer: Option B — They are used to hide objects

Q890. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A { int a; public: A(){ a = 0; } void show(){ a++; cout<<"a: "<<a<<endl; } }; class B: public A { public: }; int main(int argc, char const *argv[]) { B b; b.show(); return 0; }

A. a: 1 ✓

B. a: 0

C. Error

D. Segmentation fault

Answer: Option A — a: 1

Q891. Which is the correct syntax of declaring a virtual function?

A. virtual int func(); ✓

B. virtual int func(){};

C. inline virtual func();

D. inline virtual func(){};

Answer: Option A — virtual int func();

Q892. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A { int a; public: A(){ a = 0; } static void show(){ a++; cout<<a; } }; class B: public A { public: }; int main(int argc, char const *argv[]) { B b; b.show(); return 0; }

A. 1

B. 0

C. Segmentation fault

D. Error ✓

Answer: Option D — Error

Q893. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A { public: virtual A(){ cout<<"A's Constructor\n"; } }; class B: public A { public: A(){ cout<<"Present inside the class B\n"; } }; int main(int argc, char const *argv[]) { A a; return 0; }

A. A's Constructor

B. Present inside the class B

C. Error ✓

D. Segmentation fault

Answer: Option C — Error

Q894. What happens if the following C++ program is compiled? #include <iostream> #include <string> using namespace std; class A { int a; public: A(){ a = 0; } void show(){ a++; cout<<"a: "<<a<<endl; } }; class B: private A { public: void show(){ show(); } }; int main(int argc, char const

***argv[]) { B b; b.show(); return 0; }**

A. Error because of the conflicts between two show() function in class B

B. Program will compile successfully ✓

C. Error due to self call in show() function

D. Error because show() function from class A is derived privately

Answer: Option B — Program will compile successfully

Q895. What will be the output of the following C++ code? #include <iostream> using namespace std; class Base { public: virtual void print() const = 0; }; class DerivedOne : public Base { public: void print() const { cout << "DerivedOne\n"; } }; class DerivedTwo : public Base { public: void print() const { cout << "DerivedTwo\n"; } }; class Multiple : public DerivedOne, public DerivedTwo { public: void print() const { DerivedTwo :: print(); } }; int main() { int i; Multiple both; DerivedOne one; DerivedTwo two; Base *array[3]; array[0] = &both; array[1] = &one; array[2] = &two; array[i] -> print(); return 0; }

A. DerivedOne

B. DerivedTwo

C. Error ✓

D. DerivedThree

Answer: Option C — Error

Q896. Which design patterns benefit from the multiple inheritances?

A. Adapter and observer pattern ✓

B. Code pattern

C. Glue pattern

D. Star pattern

Answer: Option A — Adapter and observer pattern

Q897. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A { public: virtual static void show(){ cout<<"class A\n"; } }; class B: public A { public: static void show(){ cout<<"class B\n"; } }; int main(int argc, char const *argv[]) { A *a = new A(); B b; a = &b; a->show(); return 0; }

A. class B

B. Error ✓

C. Segmentation fault

D. class A

Answer: Option B — Error

Q898. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A { float d; public: A(){ cout<<"Constructor of class A\n"; } }; class B: public A { int a = 15; public: B(){ cout<<"Constructor of class B\n"; } }; int main(int argc, char const *argv[]) { B b; return 0; }

A. Constructor of class A Constructor of class B ✓

B. Constructor of class A

C. Constructor of class B

D. Constructor of class B Constructor of class A

Answer: Option A — Constructor of class A Constructor of class B

Q899. What will be the output of the following C++ code? #include <iostream> using namespace std; class student { public: int rno , m1 , m2 ; void get() { rno = 15, m1 = 10, m2 = 10; } }; class

sports { public: int sm; void getsm() { sm = 10; } }; class statement:public student,public sports { int tot,avg; public: void display() { tot = (m1 + m2 + sm); avg = tot / 3; cout << tot; cout << avg; } }; int main() { statement obj; obj.get(); obj.getsm(); obj.display(); }

A. 3100

B. 3010 ✓

C. 2010

D. 1010

Answer: Option B — 3010

Q900. What is a pure virtual function?

A. A virtual function defined inside the base class

B. A virtual function that has no definition relative to the base class ✓

C. A virtual function that is defined inside the derived class

D. Any function that is made virtual

Answer: Option B — A virtual function that has no definition relative to the base class

---

Q901. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class Mammal { public: virtual void Define(){ cout<<"I'm a Mammal\n"; } }; class Human: public Mammal { public: void Define(){ cout<<"I'm a Human\n"; } }; class Male: public Human { public: void Define(){ cout<<"I'm a Male\n"; } }; class Female: public Human { public: void Define(){ cout<<"I'm a Female\n"; } }; int main(int argc, char const *argv[]) { Mammal *M = new Mammal(); Male m; Female f; *M = m; M->Define(); M = &m; M->Define(); return 0; }

A. I'm a Male I'm a Male

B. I'm a Male I'm a Mammal

C. I'm a Mammal I'm a Male ✓

D. I'm a Mammal I'm a Mammal

Answer: Option C — I'm a Mammal I'm a Male

Q902. What will be the output of the following C++ code? #include <iostream> using namespace std; struct a { int count; }; struct b { int* value; }; struct c : public a, public b { }; int main() { c* p = new c; p->value = 0; cout << "Inherited"; return 0; }

A. Inherited ✓

B. Error

C. Runtime error

D. None of the above

Answer: Option A — Inherited

Q903. What will be the output of the following C++ code? #include <iostream> using namespace std; class Base1 { protected: int SampleDataOne; public: Base1() { SampleDataOne = 100; } ~Base1() { } int SampleFunctOne() { return SampleDataOne; } }; class Base2 { protected: int SampleDataTwo; public: Base2() { SampleDataTwo = 200; } ~Base2() { } int SampleFunctTwo() { return SampleDataTwo; } }; class Derived1 : public Base1, public Base2 { int MyData; public: Derived1() { MyData = 300; } ~Derived1() { } int MyFunct() { return (MyData + SampleDataOne + SampleDataTwo); } }; int main() { Base1 SampleObjOne; Base2 SampleObjTwo; Derived1 SampleObjThree; cout << SampleObjThree.Base1 :: SampleFunctOne() << endl; cout << SampleObjThree.Base2 :: SampleFunctTwo() << endl; return 0; }

A. 100

B. 200

C. Both 100 & 200 ✓

D. 150

Answer: Option C — Both 100 & 200

Q904. Which of the following advantages we lose by using multiple inheritances?

A. Dynamic binding

B. Polymorphism

C. Both Dynamic binding & Polymorphism ✓

D. Constructor

Answer: Option C — Both Dynamic binding & Polymorphism

Q905. Pick the correct statement.

A. Virtual function can have different names in the base and derived class

B. Virtual function cannot be applied in Multiple Inheritance classes

C. Virtual function are different in definitions only ✓

D. Virtual function does early binding

Answer: Option C — Virtual function are different in definitions only

---

Q906. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A { int a; public: A(){ a = 0; } void show(){ a++; cout<<"a: "<<a<<endl; } }; class B: private A { public: }; int main(int argc, char const *argv[]) { B b; b.show(); return 0; }

A. Error ✓

B. Segmentation fault

C. a: 1

D. a: 0

Answer: Option A — Error

Q907. What is Inheritance in C++?

A. Wrapping of data into a single class

B. Deriving new classes from existing classes ✓

C. Overloading of classes

D. Classes with same names

Answer: Option B — Deriving new classes from existing classes

Q908. What is a virtual function in C++?

A. Any member function of a class

B. All functions that are derived from the base class

C. All the members that are accessing base class data members

D. All the functions which are declared in the base class and is re-defined/overridden by the derived class ✓

Answer: Option D — All the functions which are declared in the base class and is re-defined/overridden by the derived class

Q909. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A { float d; public: int a; void change(int i){ a = i; } void value_of_a(){ cout<<a; } }; class B: public A { int a = 15; public: void print(){ cout<<a; } }; int main(int argc, char const *argv[]) { B b; b.change(10); b.print(); b.value_of_a(); return 0; }

A. 1010

B. 1510 ✓

C. 1515

D. 5110

Answer: Option B — 1510

Q910. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A { float d; public: virtual void func(){ cout<<"Hello this is class A\n"; } }; class B: public A { int a = 15; public: void func(){ cout<<"Hello this is class B\n"; } }; int main(int argc, char const *argv[]) { A *a = new A(); B b; a = &b; a->func(); return 0; }

A. Hello this is class A

B. Hello this is class B ✓

C. Error

D. Segmentation Fault

Answer: Option B — Hello this is class B

Q911. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class Mammal { public: virtual void Define(){ cout<<"I'm a Mammal\n"; } }; class Human: public Mammal { private: void Define(){ cout<<"I'm a Human\n"; } }; int main(int argc, char const *argv[]) { Mammal *M = new Mammal(); Human H; M = &H; M->Define(); return 0; }

A. Error

B. Segmnetation fault

C. I'm a Human ✓

D. Garbage Value

Answer: Option C — I'm a Human

Q912. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A{ float d; public: virtual void func(){ cout<<"Hello this is class A\n"; } }; class B: public A{ int a = 15; public: void func(){ cout<<"Hello this is class B\n"; } }; int main(int argc, char const *argv[]) { B b; b.func(); return 0; }

A. Hello this is class B ✓

B. Hello this is class A

C. Error

D. Segmentation fault

Answer: Option A — Hello this is class B

Q913. Which concept of OOPs is shown by Virtual Functions?

A. Inheritance

B. Encapsulation

C. Polymorphism ✓

D. Abstraction

Answer: Option C — Polymorphism

Q914. Virtual functions in C++ tells the compiler to perform . . . . . on such functions.

A. static binding

B. late binding ✓

C. compile time binding

D. no binding

Answer: Option B — late binding

Q915. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class Mammal { public: virtual void Define(){ cout<<"I'm a Mammal\n"; } }; class Human: public Mammal { public: void Define(){ cout<<"I'm a Human\n"; } }; class Male: public Human { public: void Define(){ cout<<"I'm a Male\n"; } }; class Female: public Human { public: void Define(){ cout<<"I'm a Female\n"; } }; int main(int argc, char const *argv[]) { Mammal *M; Male m; Female f; *M = m; M->Define(); return 0; }

A. I'm a Male

B. I'm a Mammal

C. Error

D. Segmentation Fault ✓

Answer: Option D — Segmentation Fault

Q916. The concept of deciding which function to invoke during runtime is called . . . . . .

A. late binding

B. dynamic linkage

C. static binding

D. both late binding and dynamic linkage ✓

Answer: Option D — both late binding and dynamic linkage

Q917. What is the order of Destructors call when the object of derived class B is declared, provided class B is derived from class A?

A. Destructor of A followed by B

B. Destructor of B followed by A ✓

C. Destructor of A only

D. Destructor of B only

Answer: Option B — Destructor of B followed by A

Q918. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; class A { int a, b; float d; public: void change(int i){ a = i; } void value_of_a(){ cout<<a; } }; class B: private A { }; int main(int argc, char const *argv[]) { B b; cout<<sizeof(B); return 0; }

A. 8

B. 12 ✓

C. Error

D. Segmentation fault

Answer: Option B — 12

Q919. What are the things are inherited from the base class?

A. Constructor and its destructor

B. Operator=() members

C. Friends

D. All of the mentioned ✓

Answer: Option D — All of the mentioned

Q920. What is the order of Constructors call when the object of derived class B is declared, provided class B is derived from class A?

A. Constructor of A followed by B ✓

B. Constructor of B followed by A

C. Constructor of A only

D. Constructor of B only

Answer: Option A — Constructor of A followed by B

Q921. What will be the output of the following C++ code? #include <iostream> using namespace std; class polygon { protected: int width, height; public: void set_values (int a, int b) { width = a; height = b;} }; class output1 { public: void output (int i); }; void output1::output (int i) { cout << i << endl; } class rectangle: public polygon, public output1 { public: int area () { return (width * height); } }; class triangle: public polygon, public output1 { public: int area () { return (width * height / 2); } }; int main () { rectangle rect; triangle trgl; rect.set_values (4, 5); trgl.set_values (4, 5); rect.output (rect.area()); trgl.output (trgl.area()); return 0; }

A. 20

B. 10

C. 20 10 ✓

D. 30

Answer: Option C — 20 10

Q922. Which symbol is used to create multiple inheritances?

A. Dot

B. Comma ✓

C. Dollar

D. star

Answer: Option B — Comma

Q923. How many specifiers are used to derive a class?

A. 1

B. 2

C. 3 ✓

D. 4

Answer: Option C — 3

Q924. If a class is derived privately from a base class then . . . . . .

A. no members of the base class is inherited

B. all members are accessible by the derived class

C. all the members are inherited by the class but are hidden and cannot be accessible ✓

D. no derivation of the class gives an error

Answer: Option C — all the members are inherited by the class but are hidden and cannot be accessible

Q925. What is meant by multiple inheritance?

A. Deriving a base class from derived class

B. Deriving a derived class from base class

C. Deriving a derived class from more than one base class ✓

D. Deriving a derived base class

Answer: Option C — Deriving a derived class from more than one base class

---

Polymorphism In C++

Q926. What is polymorphism in C++?

A. A feature that allows objects of different classes to have the same name.

B. A feature that allows functions to perform different tasks based on the object they are called with.

C. A feature that allows the same function to have different implementations. ✓

D. A feature that allows inheritance of multiple classes.

Answer: Option C — A feature that allows the same function to have different implementations.

Q927. Which type of polymorphism is achieved by function overloading in C++?

A. Compile-time polymorphism ✓

B. Run-time polymorphism

C. Static polymorphism

D. Dynamic polymorphism

Answer: Option A — Compile-time polymorphism

Q928. What is dynamic polymorphism in C++?

A. Achieved through function overloading.

B. Achieved through function overriding.

C. Achieved using pointers or references to base class objects. ✓

D. Achieved through template functions.

Answer: Option C — Achieved using pointers or references to base class objects.

Q929. How is polymorphism implemented in C++?

A. By defining multiple functions with the same name but different parameters.

B. By using function pointers.

C. By using virtual functions. ✓

D. By defining multiple classes with the same name.

Answer: Option C — By using virtual functions.

Q930. Which keyword is used to achieve runtime polymorphism in C++?

A. override

B. virtual ✓

C. final

D. polymorphic

Answer: Option B — virtual

Q931. What is the use of the 'virtual' keyword in C++?

A. It indicates that a function can be overridden in derived classes. ✓

B. It indicates that a function cannot be overridden in derived classes.

C. It indicates that a function must be called in the derived class.

D. It indicates that a function can be called from anywhere.

Answer: Option A — It indicates that a function can be overridden in derived classes.

Q932. What is the role of a virtual destructor in C++?

A. Ensures proper destruction of derived class objects when deleted through a base class pointer. ✓

B. Ensures proper destruction of base class objects.

C. Ensures proper memory allocation for virtual functions.

D. Ensures proper memory alignment for polymorphic objects.

Answer: Option A — Ensures proper destruction of derived class objects when deleted through a base class pointer.

Q933. Which type of polymorphism is achieved by function overriding in C++?

A. Compile-time polymorphism

B. Run-time polymorphism

C. Static polymorphism

D. Dynamic polymorphism ✓

Answer: Option D — Dynamic polymorphism

Q934. In C++, can a derived class have its own definition of a function that is already defined in the base class?

A. Yes ✓

B. No

C. It depends on the access specifier of the base class function.

D. It depends on the data type of the base class function.

Answer: Option A — Yes

Q935. Which type of inheritance is necessary for achieving polymorphism in C++?

A. Single inheritance

B. Multiple inheritance

C. Hierarchical inheritance

D. Multilevel inheritance ✓

Answer: Option D — Multilevel inheritance

Q936. What happens if a virtual function is not overridden in the derived class in C++?

A. Compilation error

B. Run-time error

C. No error, the base class function is called. ✓

D. No error, the derived class function is called.

Answer: Option C — No error, the base class function is called.

Q937. What is function overloading in C++?

A. Defining multiple functions with the same name but different parameters. ✓

B. Defining multiple functions with the same name and same parameters.

C. Defining a function that can accept multiple data types.

D. Defining a function inside another function.

Answer: Option A — Defining multiple functions with the same name but different parameters.

Q938. What is compile-time polymorphism in C++?

A. Polymorphism resolved during compile time. ✓

B. Polymorphism resolved during runtime.

C. Polymorphism resolved during linking.

D. Polymorphism resolved during execution.

Answer: Option A — Polymorphism resolved during compile time.

Q939. In C++, can a function be both virtual and static at the same time?

A. Yes

B. No ✓

C. Yes, but only for member functions.

D. Yes, but only for non-member functions.

Answer: Option B — No

Q940. Which of the following is not a type of polymorphism in C++?

A. Parametric polymorphism ✓

B. Ad-hoc polymorphism

C. Compile-time polymorphism

D. Run-time polymorphism

Answer: Option A — Parametric polymorphism

Q941. What is the purpose of a pure virtual function in C++?

A. It forces derived classes to implement the function. ✓

B. It prevents derived classes from implementing the function.

C. It allows the base class to override the derived class's function.

D. It allows multiple inheritance.

Answer: Option A — It forces derived classes to implement the function.

Q942. How is polymorphism different from overloading in C++?

A. Polymorphism involves different implementations of the same function name. ✓

B. Overloading involves multiple functions with the same name but different parameters.

C. Polymorphism is resolved at compile time, while overloading is resolved at runtime.

D. Polymorphism is a feature of C++ while overloading is a feature of Java.

Answer: Option A — Polymorphism involves different implementations of the same function name.

Q943. What is the concept of function overriding in C++?

A. It involves a derived class defining a function that redefines a base class function. ✓

B. It involves a base class defining a function that is already defined in the derived class.

C. It involves multiple functions with the same name but different implementations.

D. It involves calling a function multiple times within the same scope.

Answer: Option A — It involves a derived class defining a function that redefines a base class function.

Q944. Which keyword is used to prevent further inheritance of a class in C++?

A. sealed

B. final ✓

C. restrict

D. abstract

Answer: Option B — final

Q945. What is the role of the 'virtual' keyword in function declaration in C++?

A. It indicates that the function can be accessed globally.

B. It indicates that the function can be overridden in derived classes. ✓

C. It indicates that the function is immutable.

D. It indicates that the function has no return type.

Answer: Option B — It indicates that the function can be overridden in derived classes.

Q946. Which type of polymorphism allows a class to have multiple methods with the same name but different signatures?

A. Run-time polymorphism

B. Operator overloading

C. Parametric polymorphism

D. Ad-hoc polymorphism ✓

Answer: Option D — Ad-hoc polymorphism

Q947. What is the purpose of a virtual destructor in C++?

A. To prevent memory leaks when deleting objects of derived classes through base class pointers.

B. To prevent memory leaks when deleting objects of base classes.

C. To ensure proper destruction of derived class objects when deleted through a base class pointer. ✓

D. To ensure proper memory alignment for polymorphic objects.

Answer: Option C — To ensure proper destruction of derived class objects when deleted through a base class pointer.

Q948. Which of the following is true about virtual functions in C++?

A. They cannot be overloaded.

B. They can be static.

C. They are resolved at compile-time.

D. They are resolved at runtime. ✓

Answer: Option D — They are resolved at runtime.

Q949. What is the syntax for calling a virtual function from a derived class constructor in C++?

A. virtual_function()

B. Base::virtual_function() ✓

C. this->virtual_function()

D. Derived::virtual_function()

Answer: Option B — Base::virtual_function()

Q950. What is the difference between static binding and dynamic binding in C++?

A. Static binding occurs at compile time while dynamic binding occurs at runtime. ✓

B. Static binding occurs at runtime while dynamic binding occurs at compile time.

C. Static binding occurs when linking while dynamic binding occurs when executing.

D. Static binding occurs in inheritance while dynamic binding occurs in polymorphism.

Answer: Option A — Static binding occurs at compile time while dynamic binding occurs at runtime.

Q951. In C++, can a derived class have more than one base class with a virtual function?

A. Yes ✓

B. No

C. Yes, but only if the virtual functions have the same name.

D. Yes, but only if the virtual functions have different return types.

Answer: Option A — Yes

Q952. Which type of polymorphism is achieved by function templates in C++?

A. Compile-time polymorphism ✓

B. Run-time polymorphism

C. Static polymorphism

D. Dynamic polymorphism

Answer: Option A — Compile-time polymorphism

Q953. What happens if a pure virtual function is not implemented in a derived class in C++?

A. Compilation error ✓

B. Run-time error

C. No error, the base class function is called.

D. No error, the derived class function is called.

Answer: Option A — Compilation error

Q954. Which C++ feature allows a function to behave differently based on the number or types of its arguments?

A. Operator overloading

B. Function overloading ✓

C. Inheritance

D. Encapsulation

Answer: Option B — Function overloading

Q955. In C++, what is the effect of declaring a destructor as virtual?

A. It prevents memory leaks

B. It allows dynamic binding ✓

C. It enables polymorphism

D. It raises a compilation error

Answer: Option B — It allows dynamic binding

Q956. Which type of polymorphism allows different classes to be treated as instances of the same class through a common interface?

A. Compile-time polymorphism

B. Run-time polymorphism ✓

C. Static polymorphism

D. Dynamic polymorphism

Answer: Option B — Run-time polymorphism

Q957. How can you achieve runtime polymorphism in C++?

A. By overloading functions

B. By using virtual functions ✓

C. By using friend functions

D. By using template functions

Answer: Option B — By using virtual functions

Q958. In C++, which keyword can be used to prevent a virtual function from being overridden in derived classes?

A. override

B. final ✓

C. virtual

D. extend

Answer: Option B — final

Q959. What is the main advantage of polymorphism in C++?

A. Code reusability ✓

B. Code optimization

C. Memory efficiency

D. Type safety

Answer: Option A — Code reusability

Q960. Which C++ keyword is used to access the base class within a derived class?

A. super

B. base ✓

C. this

D. parent

Answer: Option B — base

Q961. What is the difference between function overloading and function overriding in C++?

A. Overloading deals with static binding ✓

B. Overloading deals with dynamic binding

C. Overriding deals with static binding

D. Overriding deals with dynamic binding

Answer: Option A — Overloading deals with static binding

Q962. In C++, can you overload the return type of a function?

A. Yes

B. No ✓

C. Only if the function is virtual

D. Only if the function is static

Answer: Option B — No

Q963. Which type of polymorphism is not supported in C++?

A. Compile-time polymorphism

B. Run-time polymorphism

C. Ad-hoc polymorphism

D. Parametric polymorphism ✓

Answer: Option D — Parametric polymorphism

Q964. What is the primary reason for using virtual functions in C++?

A. To improve code readability

B. To ensure proper destruction

C. To enable polymorphism ✓

D. To avoid memory leaks

Answer: Option C — To enable polymorphism

Q965. How can you achieve polymorphism in C++?

A. By using function overloading

B. By using function overriding

C. By using both function overloading and overriding ✓

D. By using encapsulation

Answer: Option C — By using both function overloading and overriding

---

Q966. In C++, can you declare a constructor as virtual?

A. Yes

B. No ✓

C. Only if it has parameters

D. Only if it is a copy constructor

Answer: Option B — No

Q967. What is the result of attempting to instantiate an object of an abstract class in C++?

A. Compilation error ✓

B. Run-time error

C. No error, but the program may crash

D. No error, the object is instantiated

Answer: Option A — Compilation error

File Handling In C++

Q968. Which header file is required for file handling in C++?

A. <iostream>

B. <fstream> ✓

C. <file>

D. <filestream>

Answer: Option B — <fstream>

Q969. What is the function used to open a file in C++?

A. open() ✓

B. read()

C. fopen()

D. start()

Answer: Option A — open()

Q970. How do you close a file in C++?

A. file.close() ✓

B. closeFile()

C. fclose()

D. endFile()

Answer: Option A — file.close()

Q971. Which mode is used to open a file for writing in C++?

A. read

B. append

C. write ✓

D. update

Answer: Option C — write

Q972. What is the correct syntax for writing data to a file in C++?

A. file.write(data)

B. write(file, data)

C. file << data ✓

D. data << file

Answer: Option C — file << data

Q973. How do you check if a file has been successfully opened in C++?

A. file.is_open() ✓

B. isFileOpen()

C. file.opened()

D. isOpen(file)

Answer: Option A — file.is_open()

Q974. Which of the following is used to read data from a file in C++?

A. file.read()

B. read(file)

C. file >> data ✓

D. data >> file

Answer: Option C — file >> data

Q975. What does the seekg() function do in C++ file handling?

A. Sets the get position pointer ✓

B. Sets the put position pointer

C. Sets the end of file pointer

D. Sets the beginning of file pointer

Answer: Option A — Sets the get position pointer

Q976. Which function is used to delete a file in C++?

A. deleteFile()

B. removeFile()

C. delete()

D. remove() ✓

Answer: Option D — remove()

Q977. Which file mode is used to open a file for reading and writing in C++?

A. append

B. readwrite

C. write

D. rplus ✓

Answer: Option D — rplus

Q978. How do you check if the end of a file has been reached in C++?

A. file.end_of_file()

B. endOfFile(file)

C. file.eof() ✓

D. eof(file)

Answer: Option C — file.eof()

Q979. Which function is used to get the current position of the file pointer in C++?

A. getpos()

B. currentPos()

C. tellg() ✓

D. position()

Answer: Option C — tellg()

Q980. What is the purpose of the ios::app flag in C++ file handling?

A. Appends data to the end of the file

B. Opens the file in append mode ✓

C. Opens the file in write mode

D. Opens the file in read mode

Answer: Option B — Opens the file in append mode

Q981. Which function is used to check if a file exists in C++?

A. file.exists()

B. exists(file) ✓

C. file.opened()

D. open(file)

Answer: Option B — exists(file)

Q982. How do you create a new file in C++?

A. newFile()

B. createFile() ✓

C. file.create()

D. ofstream file()

Answer: Option B — createFile()

Q983. What is the function used to create a new file in C++?

A. create()

B. newFile()

C. fopen()

D. ofstream file() ✓

Answer: Option D — ofstream file()

Q984. Which file mode is used to open a file for reading only in C++?

A. write

B. append

C. read ✓

D. ronly

Answer: Option C — read

Q985. How do you check if a file exists before opening it in C++?

A. file.exists() ✓

B. exists(file)

C. file.is_open()

D. isOpen(file)

Answer: Option A — file.exists()

Q986. Which of the following is not a file stream class in C++?

A. ifstream

B. ofstream

C. fstream

D. filestream ✓

Answer: Option D — filestream

Q987. What is the function used to write data to a file in C++?

A. file.put()

B. write(file, data)

C. file << data ✓

D. data << file

Answer: Option C — file << data

Q988. Which function is used to set the file position in C++ file handling?

A. file.seek()

B. seekg() ✓

C. file.position()

D. setPosition()

Answer: Option B — seekg()

Q989. What is the purpose of the ios::ate flag in C++ file handling?

A. Sets the get position pointer at the beginning of the file

B. Sets the get position pointer at the end of the file ✓

C. Sets the get position pointer at the current position in the file

D. Sets the get position pointer to the specified position in the file

Answer: Option B — Sets the get position pointer at the end of the file

Q990. In C++, how do you check if a file has reached the end while reading?

A. file.eof() ✓

B. eof(file)

C. file.end_of_file()

D. endOfFile(file)

Answer: Option A — file.eof()

Q991. Which function is used to rewind the file pointer to the beginning of the file in C++?

A. rewind()

B. reset()

C. file.rewind()

D. file.seekg(0) ✓

Answer: Option D — file.seekg(0)

Q992. What is the function used to read a single character from a file in C++?

A. file.getch()

B. readChar()

C. file.readChar()

D. get() ✓

Answer: Option D — get()

Q993. Which of the following is not a valid file mode in C++?

A. w ✓

B. a

C. r

D. x

Answer: Option A — w

Q994. How do you delete a file in C++?

A. file.delete()

B. deleteFile()

C. removeFile() ✓

D. remove()

Answer: Option C — removeFile()

Q995. Which function is used to return the current file pointer position in C++?

A. currentPos()

B. tellg() ✓

C. position()

D. getFilePos()

Answer: Option B — tellg()

Q996. Which function is used to check if a file is open or not in C++?

A. isOpen()
B. isFileOpen()
C. file.is_open() ✓
D. fileOpened()
Answer: Option C — file.is_open()

Q997. What happens if you try to open a file that does not exist in C++ without specifying any flags?

A. The file will be created
B. An error will occur ✓
C. The program will wait indefinitely
D. The file will be opened in read mode
Answer: Option B — An error will occur

Q998. Which file mode in C++ opens a file for reading and writing, and truncates it to zero length if it exists?

A. readwrite
B. truncate
C. wplus ✓
D. truncwrite
Answer: Option C — wplus

Q999. What is the difference between fstream and ofstream in C++?

A. fstream can read and write, ofstream can only write ✓
B. ofstream can read and write, fstream can only write
C. ofstream is used for text files, fstream is used for binary files
D. fstream is used for text files, ofstream is used for binary files
Answer: Option A — fstream can read and write, ofstream can only write

Q1000. How do you check if a file operation in C++ has encountered an error?

A. file.is_error()
B. file.error()
C. file.fail() ✓
D. file.has_error()
Answer: Option C — file.fail()

Q1001. What is the purpose of the ios::binary flag in C++ file handling?

A. Opens the file in binary mode ✓
B. Opens the file in text mode
C. Opens the file in append mode
D. Opens the file in read mode
Answer: Option A — Opens the file in binary mode

Q1002. Which of the following is used to close a file in C++?

A. close()
B. closeFile()

C. fclose()

D. file.close() ✓

Answer: Option D — file.close()

Q1003. In C++, how do you check if a file operation has reached the end of the file?

A. file.eof() ✓

B. eof(file)

C. file.end_of_file()

D. endOfFile(file)

Answer: Option A — file.eof()

Q1004. Which function is used to set the position of the file pointer in C++ file handling?

A. seekg()

B. seekp() ✓

C. file.setpos()

D. setpos()

Answer: Option B — seekp()

Q1005. How do you check if a file exists in C++ before opening it?

A. file.exists() ✓

B. exists(file)

C. file.opened()

D. open(file)

Answer: Option A — file.exists()

Q1006. What is the purpose of the ios::trunc flag in C++ file handling?

A. Truncates the file if it exists and opens it in write mode ✓

B. Opens the file in trunc mode

C. Opens the file in read mode

D. Opens the file in append mode

Answer: Option A — Truncates the file if it exists and opens it in write mode

Q1007. Which file mode in C++ opens a file for reading and writing without truncating it if it exists?

A. readwrite

B. append

C. wplus

D. readplus ✓

Answer: Option D — readplus

Q1008. Which function is used to read a line from a file in C++?

A. getline() ✓

B. readLine()

C. file.getLine()

D. file.readline()

Answer: Option A — getline()

Q1009. By default, all the files in C++ are opened in . . . . . mode.

A. Text ✓

B. Binary

C. ISCII

D. VTC

Answer: Option A — Text

Q1010. Which header file is required to use file I/O operations?

A. <ifstream>

B. <ostream>

C. <fstream> ✓

D. <iostream>

Answer: Option C — <fstream>

Q1011. Which function is used to reposition the file pointer?

A. moveg()

B. seekg() ✓

C. changep()

D. go_p()

Answer: Option B — seekg()

Q1012. Which of the following is not used as a file opening mode?

A. ios::trunc ✓

B. ios::binary

C. ios::in

D. ios::ate

Answer: Option A — ios::trunc

Q1013. Which of the following is used to create an output stream?

A. ofstream ✓

B. ifstream

C. iostream

D. fsstream

Answer: Option A — ofstream

Q1014. What is the use of ios::trunc mode?

A. To open a file in input mode

B. To open a file in output mode

C. To truncate an existing file to half

D. To truncate an existing file to zero ✓

Answer: Option D — To truncate an existing file to zero

Q1015. Which of the following is not used to seek file pointer?

A. ios::set ✓

B. ios::end

C. ios::cur

D. ios::beg

Answer: Option A — ios::set

Q1016. Which of the following is the default mode of the opening using the ifstream class?

A. ios::in ✓

B. ios::out
C. ios::app
D. ios::trunc
Answer: Option A — ios::in

Q1017. Which of the following is used to create a stream that performs both input and output operations?

A. ofstream
B. ifstream
C. iostream
D. fstream ✓
Answer: Option D — fstream

Q1018. Which of the following is used to move the file pointer to start of a file?

A. ios::beg ✓
B. ios::start
C. ios::cur
D. ios::first
Answer: Option A — ios::beg

Q1019. Which of the following statements are correct? 1. It is not possible to combine two or more file opening mode in open() method. 2. It is possible to combine two or more file opening mode in open() method. 3. ios::in and ios::out are input and output file opening mode respectively.

A. 1, 3
B. 2, 3 ✓
C. 3 only
D. 1, 2
Answer: Option B — 2, 3

Q1020. What is the return type open() method?

A. int
B. char
C. bool ✓
D. float
Answer: Option C — bool

Q1021. Which of the following is the default mode of the opening using the ofstream class?

A. ios::in
B. ios::out ✓
C. ios::app
D. ios::trunc
Answer: Option B — ios::out

Q1022. Which function is used in C++ to get the current position of file pointer in a file?

A. tell_p() ✓
B. get_pos()
C. get_p()
D. tell_pos()

Answer: Option A — tell_p()

Q1023. Which of the following is the default mode of the opening using the fstream class?

A. ios::in

B. ios::out

C. ios::in|ios::out ✓

D. ios::trunc

Answer: Option C — ios::in|ios::out

Exception Handling In C++

Q1024. What is exception handling in C++?

A. A method to handle errors during runtime ✓

B. A method to handle errors during compile time

C. A method to handle errors during linking

D. A method to handle errors during execution

Answer: Option A — A method to handle errors during runtime

Q1025. Which keyword is used to throw an exception in C++?

A. raise

B. throw ✓

C. catch

D. try

Answer: Option B — throw

Q1026. What is the purpose of the 'try' block in exception handling in C++?

A. To catch exceptions

B. To throw exceptions

C. To define the block of code that may generate exceptions ✓

D. To handle exceptions

Answer: Option C — To define the block of code that may generate exceptions

Q1027. Which of the following is not a standard exception class in C++?

A. IOException ✓

B. std::exception

C. std::runtime_error

D. std::out_of_range

Answer: Option A — IOException

Q1028. What is the purpose of the 'catch' block in exception handling in C++?

A. To throw exceptions

B. To define the block of code that may generate exceptions

C. To catch and handle exceptions ✓

D. To handle errors during compile time

Answer: Option C — To catch and handle exceptions

Q1029. Which of the following is not a part of the exception handling mechanism in C++?

A. try block

B. throw keyword

C. assert function ✓

D. catch block

Answer: Option C — assert function

Q1030. What does the 'std::exception' class represent in C++ exception handling?

A. It is a base class for all standard C++ exceptions ✓

B. It is used to handle file exceptions

C. It is used to handle memory exceptions

D. It is used to handle arithmetic exceptions

Answer: Option A — It is a base class for all standard C++ exceptions

---

Q1031. How do you catch all exceptions in C++?

A. catch (...) ✓

B. catch (Exception e)

C. catch (AllExceptions)

D. catch (std::exception)

Answer: Option A — catch (...)

Q1032. Which function is used to retrieve the description of the exception in C++?

A. get_message()

B. what() ✓

C. catch_msg()

D. exception_desc()

Answer: Option B — what()

Q1033. What happens if an exception is thrown but not caught in C++?

A. The program terminates abruptly ✓

B. The program continues execution normally

C. The program waits indefinitely

D. The exception is automatically caught

Answer: Option A — The program terminates abruptly

Q1034. Which of the following statements is true about the 'finally' block in C++ exception handling?

A. It must be present after every 'try' block

B. It is executed if an exception is caught

C. It is executed regardless of whether an exception is thrown or not ✓

D. It is optional and is not required

Answer: Option C — It is executed regardless of whether an exception is thrown or not

Q1035. What is the purpose of re-throwing an exception in C++?

A. To handle the exception at a different location ✓

B. To ignore the exception

C. To propagate the exception up the call stack

D. To handle the exception at the same location

Answer: Option A — To handle the exception at a different location

---

Q1036. Which header file is necessary for exception handling in C++?

A. <exception> ✓

B. <error>

C. <assert>

D. <throw>

Answer: Option A — <exception>

Q1037. What is the function of the 'assert' macro in C++?

A. It throws an exception if a condition is false ✓

B. It catches exceptions

C. It handles arithmetic errors

D. It ensures proper memory allocation

Answer: Option A — It throws an exception if a condition is false

Q1038. How do you define your own exception class in C++?

A. class MyException : public std::exception ✓

B. class MyException extends Exception

C. class MyException extends Throwable

D. class MyException : exception

Answer: Option A — class MyException : public std::exception

Q1039. What is the default behavior of an uncaught exception in C++?

A. It leads to undefined behavior

B. It causes the program to crash ✓

C. It is automatically caught and handled

D. It is logged to a file

Answer: Option B — It causes the program to crash

Q1040. What is the purpose of the 'std::runtime_error' class in C++ exception handling?

A. It represents errors detected during runtime ✓

B. It represents errors detected during compile time

C. It represents logical errors in the code

D. It represents memory allocation errors

Answer: Option A — It represents errors detected during runtime

---

Q1041. How do you specify that a function may throw an exception in C++?

A. Using the 'throws' keyword

B. Using the 'exception' keyword

C. Using the 'throws()' specifier

D. Using the 'noexcept' specifier ✓

Answer: Option D — Using the 'noexcept' specifier

Q1042. Which of the following is true about the 'noexcept' specifier in C++?

A. It indicates that a function may throw an exception

B. It indicates that a function will never throw an exception ✓

C. It automatically catches exceptions

D. It is used for exception handling

Answer: Option B — It indicates that a function will never throw an exception

Q1043. What is the purpose of the 'std::bad_alloc' class in C++ exception handling?

A. It represents errors related to dynamic memory allocation ✓

B. It represents errors related to file handling

C. It represents errors related to string manipulation

D. It represents errors related to type conversion

Answer: Option A — It represents errors related to dynamic memory allocation

Q1044. What happens if an exception is thrown inside a 'catch' block in C++?

A. The exception is automatically caught and handled

B. The program continues normally

C. The program terminates unexpectedly ✓

D. It leads to an infinite loop

Answer: Option C — The program terminates unexpectedly

Q1045. How do you re-throw the current exception in C++?

A. throw ✓

B. throw e

C. rethrow

D. throw e.what()

Answer: Option A — throw

Q1046. Which of the following statements about exception handling in C++ is correct?

A. It is an alternative to error codes ✓

B. It is faster and more efficient than error codes

C. It is used only for compile-time errors

D. It cannot handle runtime errors

Answer: Option A — It is an alternative to error codes

Q1047. What is the purpose of the 'std::exception_ptr' class in C++ exception handling?

A. It represents a pointer to an exception object ✓

B. It represents a pointer to the exception handler

C. It represents a pointer to the exception stack

D. It represents a pointer to the exception catch block

Answer: Option A — It represents a pointer to an exception object

Q1048. How do you catch multiple exceptions of different types in C++?

A. Using multiple catch blocks

B. Using a single catch block with multiple exception types ✓

C. Using the 'catch (...) ' block

D. Using nested catch blocks

Answer: Option B — Using a single catch block with multiple exception types

Q1049. Which keyword is used to define a custom exception class in C++?

A. exception

B. custom_exception

C. throw_exception

D. class ✓

Answer: Option D — class

Q1050. What is the purpose of the 'std::terminate' function in C++?

A. It is called when an exception is thrown

B. It is called when a program terminates normally

C. It is called when an exception is caught

D. It is called when an uncaught exception causes program termination ✓

Answer: Option D — It is called when an uncaught exception causes program termination

---

Q1051. What is the purpose of the 'nothrow' parameter in the 'new' operator in C++?

A. It prevents the 'new' operator from throwing exceptions ✓

B. It forces the 'new' operator to throw exceptions

C. It specifies the size of the allocated memory

D. It specifies the alignment of the allocated memory

Answer: Option A — It prevents the 'new' operator from throwing exceptions

Q1052. How do you handle exceptions thrown from constructor initialization lists in C++?

A. Using try-catch blocks inside the constructor

B. Using try-catch blocks outside the constructor ✓

C. Using the 'noexcept' specifier

D. Exceptions from constructor initialization lists cannot be caught

Answer: Option B — Using try-catch blocks outside the constructor

Q1053. What is the purpose of the 'std::bad_function_call' class in C++ exception handling?

A. It represents errors in function calls ✓

B. It represents errors in lambda expressions

C. It represents errors in function pointers

D. It represents errors in function templates

Answer: Option A — It represents errors in function calls

Q1054. Which statement about the 'finally' block in C++ exception handling is true?

A. It must always be followed by a 'catch' block

B. It must always be followed by a 'try' block

C. It is executed regardless of whether an exception is thrown or not ✓

D. It is optional and can be omitted

Answer: Option C — It is executed regardless of whether an exception is thrown or not

Q1055. What is the purpose of the 'std::nested_exception' class in C++ exception handling?

A. It represents exceptions nested within other exceptions ✓

B. It represents exceptions thrown by nested functions

C. It represents exceptions caught by nested try-catch blocks

D. It represents exceptions that occur in nested loops

Answer: Option A — It represents exceptions nested within other exceptions

---

Q1056. How do you catch an exception by reference in C++?

A. Using a catch block with an ampersand (&) after the exception type ✓

B. Using a catch block with an asterisk (*) after the exception type

C. Using a catch block with no specifier after the exception type

D. Using a catch block with a 'ref' keyword after the exception type

Answer: Option A — Using a catch block with an ampersand (&) after the exception type

Q1057. What is the purpose of the 'std::logic_error' class in C++ exception handling?

A. It represents errors in logical operations ✓

B. It represents errors in runtime logic

C. It represents errors in memory operations

D. It represents errors in arithmetic operations

Answer: Option A — It represents errors in logical operations

Q1058. What happens if an exception is thrown inside a destructor in C++?

A. It leads to undefined behavior ✓

B. It is automatically caught and handled

C. It causes the program to crash

D. It is logged to a file

Answer: Option A — It leads to undefined behavior

Q1059. Which keyword is used to specify that a function will not throw any exceptions in C++?

A. nothrow

B. noexcept ✓

C. exception

D. noexception

Answer: Option B — noexcept

Q1060. What is the purpose of the 'std::uncaught_exception' function in C++?

A. It checks if there are any uncaught exceptions ✓

B. It handles uncaught exceptions

C. It throws an exception

D. It catches an exception

Answer: Option A — It checks if there are any uncaught exceptions

---

Q1061. Which standard header file is required for exception handling in C++?

A. <stdexcept> ✓

B. <exception>

C. <error>

D. <assert>

Answer: Option A — <stdexcept>

Q1062. How do you specify that a function will throw no exceptions in C++?

A. Using the 'noexcept' specifier ✓

B. Using the 'nothrow' specifier

C. Using the 'throw' keyword

D. Using the 'catch' block

Answer: Option A — Using the 'noexcept' specifier

Q1063. What is the purpose of the 'std::exception_ptr::rethrow' function in C++?

A. It rethrows the current exception ✓

B. It creates a new exception

C. It catches an exception

D. It checks for exceptions

Answer: Option A — It rethrows the current exception

Q1064. Which keyword is used to specify that a function will throw any exceptions in C++?

A. throw

B. throwall

C. throwexception

D. throws ✓

Answer: Option D — throws

Q1065. What is the purpose of the 'std::unexpected' function in C++ exception handling?

A. It is called when an unexpected exception occurs ✓

B. It handles unexpected exceptions

C. It throws an unexpected exception

D. It catches an unexpected exception

Answer: Option A — It is called when an unexpected exception occurs

---

Q1066. Which statement about exception handling in C++ is true?

A. An exception thrown inside a 'try' block must always be caught ✓

B. An exception thrown inside a 'catch' block must always be rethrown

C. An exception can only be caught once

D. An exception can only be thrown once

Answer: Option A — An exception thrown inside a 'try' block must always be caught

Q1067. What is the behavior of the 'std::terminate' function in C++?

A. It is called when an uncaught exception occurs

B. It terminates the program ✓

C. It handles exceptions

D. It catches exceptions

Answer: Option B — It terminates the program

Q1068. Uncaught exception leads to . . . . . .

A. termination of program ✓

B. successful execution of programs

C. no effect on the program

D. execution of other functions of the program starts

Answer: Option A — termination of program

Q1069. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; void func(int a, int b) { if(b == 0){ throw "This value of b will make the product zero. " "So please provide positive values.\n"; } else{ cout<<"Product of "<<a<<" and "<<b<<" is: "<<a*b<<endl; } } int main() { try{ func(5,0); } catch(const char* e){ cout<<e; } }

A. 0

B. 5

C. This value of b will make the product zero. So please provide positive values. ✓

D. Product of 5 and 0 is: 0

Answer: Option C — This value of b will make the product zero. So please provide positive values.

Q1070. What is an error in C++?

A. Violation of syntactic and semantic rules of a languages ✓

B. Missing of Semicolon

C. Missing of double quotes

D. Violation of program interface

Answer: Option A — Violation of syntactic and semantic rules of a languages

---

Q1071. What are the different types of exceptions?

A. 1

B. 2 ✓

C. 3

D. 4

Answer: Option B — 2

Q1072. Why do we need to handle exceptions?

A. To avoid unexpected behaviour of a program during run-time ✓

B. To let compiler remove all exceptions by itself

C. To successfully compile the program

D. To get correct output

Answer: Option A — To avoid unexpected behaviour of a program during run-time

Q1073. Which keyword is used to throw an exception?

A. try

B. throw ✓

C. throws

D. except

Answer: Option B — throw

Q1074. How Exception handling is implemented in the C++ program?

A. Using Exception keyword

B. Using try-catch block ✓

C. Using Exception block

D. Using Error handling schedules

Answer: Option B — Using try-catch block

Q1075. Header file used for exception handling in C++?

A. <cstdlib>

B. <string>

C. <handler>

D. <exception> ✓

Answer: Option D — <exception>

Q1076. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; class A { int a; public: A(){} }; class B: public A { int b; public: B(){} }; void func() { B b; throw b; } int main() { try{ func(); } catch(B b){ cout<<"Caught B Class\n"; } catch(A a){ cout<<"Caught A Class\n"; } }

A. Caught B Class ✓

B. Caught A Class

C. Compile-time error

D. Run-time error

Answer: Option A — Caught B Class

Q1077. Where should we place catch block of the derived class in a try-catch block?

A. Before the catch block of Base class ✓

B. After the catch block of Base class

C. Anywhere in the sequence of catch blocks

D. After all the catch blocks

Answer: Option A — Before the catch block of Base class

Q1078. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; class A { int a; public: A(){} }; class B: public A { int b; public: B(){} }; void func1() { B b; throw b; } void func2() { A a; throw a; } int main() { try{ func1(); } catch(...){ cout<<"Caught All types of exceptions\n"; } try{ func2(); } catch(B b){

cout<<"Caught All types of exceptions\n"; } }

A. Caught All types of exceptions

B. Caught All types of exceptions Aborted (core dumped) ✓

C. Error

D. Caught All types of exceptions Caught All types of exceptions

Answer: Option B — Caught All types of exceptions Aborted (core dumped)

Q1079. An uncaught handler returns to . . . . . .

A. main function

B. its caller

C. its callee ✓

D. waits there for some time

Answer: Option C — its callee

Q1080. What is an exception in C++ program?

A. A problem that arises during the execution of a program ✓

B. A problem that arises during compilation

C. Also known as the syntax error

D. Also known as semantic error

Answer: Option A — A problem that arises during the execution of a program

---

Q1081. Which part of the try-catch block is always fully executed?

A. try part

B. catch part

C. finally part ✓

D. throw part

Answer: Option C — finally part

Q1082. By default, what a program does when it detects an exception?

A. Continue running

B. Results in the termination of the program ✓

C. Calls other functions of the program

D. Removes the exception and tells the programmer about an exception

Answer: Option B — Results in the termination of the program

Q1083. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; void func(int a, int b) { if(b == 0){ throw "This value of b will make the product zero. " "So please provide positive values.\n"; } else{ cout<<"Product of "<<a<<" and "<<b<<" is: "<<a*b<<endl; } } int main() { try{ func(5,0); } catch(char* e){ cout<<e; } }

A. 0

B. Aborted (core dumped) ✓

C. This value of b will make the product zero. So please provide positive values.

D. Product of 5 and 0 is: 0

Answer: Option B — Aborted (core dumped)

Q1084. What is Re-throwing an exception means in C++?

A. An exception that is thrown again as it is not handled by that catching block

B. An exception that is caught twice

C. An exception that is not handled in one caught hence thrown again

D. All of the mentioned ✓

Answer: Option D — All of the mentioned

Q1085. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; void func(int a, int b) { if(b < 1){ throw b; } else{ cout<<"Product of "<<a<<" and "<<b<<" is: "<<a*b<<endl; } } int main() { try { try { func(5,-1); } catch(int b) { if(b==0) throw "value of b is zero\n"; else throw "value of b is less than zero\n"; } } catch(const char* e) { cout<<e; } }

A. value of b is zero

B. value of b is less than zero ✓

C. Product of 5 and -1 is: -5

D. Aborted(core dumped)

Answer: Option B — value of b is less than zero

Q1086. What id the syntax for catching any type of exceptions?

A. catch(Exception e)

B. catch(...) ✓

C. catch(Exception ALL)

D. catch(ALL)

Answer: Option B — catch(...)

Q1087. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; class A { int a; public: A(){} }; class B: public A { int b; public: B(){} }; void func() { B b; throw b; } int main() { try{ func(); } catch(B *b){ cout<<"Caught B Class\n"; } catch(A a){ cout<<"Caught A Class\n"; } }

A. Caught B Class

B. Caught A Class ✓

C. Compile-time error

D. Run-time error

Answer: Option B — Caught A Class

Q1088. Which of the following is an exception in C++?

A. Divide by zero ✓

B. Semicolon not written

C. Variable not declared

D. An expression is wrongly written

Answer: Option A — Divide by zero

Q1089. What will be the output of the following C++ code? #include <iostream> #include <string> #include <cstdlib> using namespace std; class A { int a; public: A(){} }; class B: public A { int b; public: B(){} }; void func() { B b; throw b; } int main() { try{ func(); } catch(A a){ cout<<"Caught A Class\n"; } catch(B b){ cout<<"Caught B Class\n"; } }

A. Caught B Class

B. Caught A Class ✓

C. Compile-time error

D. Run-time error

Answer: Option B — Caught A Class

Q1090. What happens when this C++ program is compiled? #include <iostream> #include <string> #include <cstdlib> using namespace std; class A { int a; public: A(){} }; class B: public A { int b; public: B(){} }; void func() { B b; throw b; } int main() { try{ func(); } catch(A a){ cout<<"Caught A Class\n"; } catch(B b){ cout<<"Caught B Class\n"; } }

A. The program compiles successfully without any errors or warnings

B. Compile-time error occurs

C. The program compiles successfully with warnings ✓

D. The program gives both errors and warnings

Answer: Option C — The program compiles successfully with warnings

---

Q1091. What is the difference between error and exception?

A. Both are the same

B. Errors can be handled at the run-time but the exceptions cannot

C. Exceptions can be handled at the run-time but the errors cannot ✓

D. Both can be handled during run-time

Answer: Option C — Exceptions can be handled at the run-time but the errors cannot

Standard Template Library (Stl) In C++

Q1092. Which header file is used to include the vector container in C++ STL?

A. <vector> ✓

B. <array>

C. <list>

D. <map>

Answer: Option A — <vector>

Q1093. What does STL stand for in C++?

A. Simple Template Library

B. Standard Template Library ✓

C. System Template Library

D. Structured Template Library

Answer: Option B — Standard Template Library

Q1094. What is the purpose of the 'std::sort' function in C++ STL?

A. To sort elements of a container ✓

B. To search for an element in a container

C. To insert elements into a container

D. To remove elements from a container

Answer: Option A — To sort elements of a container

Q1095. Which container in C++ STL does not allow duplicate elements?

A. vector

B. list

C. set ✓

D. map

Answer: Option C — set

Q1096. What is the time complexity of accessing an element in a vector in C++ STL?

A. O(1) ✓

B. O(log n)

C. O(n)

D. O(n log n)

Answer: Option A — O(1)

Q1097. Which algorithm in C++ STL is used to find the maximum element in a range?

A. max_element ✓

B. find_max

C. maximum

D. find_max_element

Answer: Option A — max_element

Q1098. What is the purpose of the 'std::map' container in C++ STL?

A. To store elements in key-value pairs ✓

B. To store elements in a sorted sequence

C. To store elements in a stack

D. To store elements in a queue

Answer: Option A — To store elements in key-value pairs

Q1099. Which container in C++ STL is implemented as a doubly linked list?

A. vector

B. list ✓

C. deque

D. stack

Answer: Option B — list

Q1100. What is the purpose of the 'std::set' container in C++ STL?

A. To store unique elements in sorted order ✓

B. To store duplicate elements in sorted order

C. To store unique elements in random order

D. To store duplicate elements in random order

Answer: Option A — To store unique elements in sorted order

Q1101. What does the 'std::reverse' algorithm do in C++ STL?

A. Reverses the order of elements in a range ✓

B. Sorts the elements in descending order

C. Finds the maximum element in a range

D. Finds the minimum element in a range

Answer: Option A — Reverses the order of elements in a range

Q1102. Which container in C++ STL does not preserve the order of elements?

A. vector

B. list

C. set ✓

D. map

Answer: Option C — set

Q1103. What is the time complexity of searching for an element in a set in C++ STL?

A. O(1)

B. O(log n) ✓

C. O(n)

D. O(n log n)

Answer: Option B — O(log n)

Q1104. Which algorithm in C++ STL is used to merge two sorted ranges into a single sorted range?

A. merge ✓

B. sort

C. concatenate

D. combine

Answer: Option A — merge

Q1105. What is the purpose of the 'std::unordered_map' container in C++ STL?

A. To store elements in key-value pairs with constant-time access ✓

B. To store elements in a sorted sequence with constant-time access

C. To store elements in a stack with constant-time access

D. To store elements in a queue with constant-time access

Answer: Option A — To store elements in key-value pairs with constant-time access

---

Q1106. Which algorithm in C++ STL is used to remove all occurrences of a value from a container?

A. remove

B. erase ✓

C. delete

D. eliminate

Answer: Option B — erase

Q1107. What is the purpose of the 'std::deque' container in C++ STL?

A. To provide dynamic array-like functionality with efficient insertion and deletion at both ends ✓

B. To store elements in a sorted sequence

C. To store elements in a stack

D. To store elements in a queue

Answer: Option A — To provide dynamic array-like functionality with efficient insertion and deletion at both ends

Q1108. Which algorithm in C++ STL is used to count the number of occurrences of a value in a range?

A. count ✓

B. find

C. search

D. find_count

Answer: Option A — count

Q1109. What is the time complexity of inserting an element into a set in C++ STL?

A. O(log n) ✓

B. O(1)

C. O(n)

D. O(n log n)

Answer: Option A — O(log n)

Q1110. Which container in C++ STL is typically implemented using a red-black tree?

A. vector

B. list

C. set ✓

D. map

Answer: Option C — set

Q1111. What is the purpose of the 'std::priority_queue' container in C++ STL?

A. To store elements in a sorted sequence with constant-time access to the largest element

B. To store elements in a sorted sequence with constant-time access to the smallest element

C. To store elements in a stack with constant-time access to the largest element ✓

D. To store elements in a queue with constant-time access to the largest element

Answer: Option C — To store elements in a stack with constant-time access to the largest element

Q1112. Which algorithm in C++ STL is used to find the first occurrence of a value in a range?

A. find ✓

B. search

C. locate

D. find_first

Answer: Option A — find

Q1113. What is the time complexity of removing an element from a list in C++ STL?

A. O(1) ✓

B. O(log n)

C. O(n)

D. O(n log n)

Answer: Option A — O(1)

Q1114. What is the purpose of the 'std::stack' container adapter in C++ STL?

A. To provide last-in, first-out (LIFO) access to elements ✓

B. To provide first-in, first-out (FIFO) access to elements

C. To store elements in a sorted sequence

D. To store elements in a hash table

Answer: Option A — To provide last-in, first-out (LIFO) access to elements

Q1115. Which algorithm in C++ STL is used to find the last occurrence of a value in a range?

A. find_last

B. rfind

C. search_last

D. find_end ✓

Answer: Option D — find_end

Q1116. What is the time complexity of searching for an element in a map in C++ STL?

A. O(log n) ✓

B. O(1)

C. O(n)

D. O(n log n)

Answer: Option A — O(log n)

Q1117. What is the purpose of the 'std::multiset' container in C++ STL?

A. To store multiple elements in sorted order ✓

B. To store multiple elements in random order

C. To store unique elements in sorted order

D. To store unique elements in random order

Answer: Option A — To store multiple elements in sorted order

Q1118. Which algorithm in C++ STL is used to find the union of two sorted ranges?

A. union

B. merge ✓

C. combine

D. join

Answer: Option B — merge

Q1119. What is the purpose of the 'std::queue' container adapter in C++ STL?

A. To provide first-in, first-out (FIFO) access to elements ✓

B. To provide last-in, first-out (LIFO) access to elements

C. To store elements in a sorted sequence

D. To store elements in a hash table

Answer: Option A — To provide first-in, first-out (FIFO) access to elements

Q1120. What is the time complexity of removing an element from a priority queue in C++ STL?

A. O(log n) ✓

B. O(1)

C. O(n)

D. O(n log n)

Answer: Option A — O(log n)

Q1121. Which algorithm in C++ STL is used to find the intersection of two sorted ranges?

A. intersect

B. set_intersection ✓

C. common

D. intersect_range

Answer: Option B — set_intersection

Q1122. What is the purpose of the 'std::unordered_set' container in C++ STL?

A. To store unique elements in a hash table ✓

B. To store elements in a sorted sequence

C. To store elements in a stack

D. To store elements in a queue

Answer: Option A — To store unique elements in a hash table

Q1123. Which algorithm in C++ STL is used to find the median of a range of elements?

A. median

B. find_median

C. partial_sort

D. nth_element ✓

Answer: Option D — nth_element

Q1124. What is the purpose of the 'std::unordered_multiset' container in C++ STL?

A. To store multiple elements in a hash table ✓

B. To store unique elements in a sorted sequence

C. To store unique elements in a hash table

D. To store multiple elements in a sorted sequence

Answer: Option A — To store multiple elements in a hash table

Q1125. Which algorithm in C++ STL is used to find the difference between two sorted ranges?

A. difference

B. set_difference ✓

C. distinct

D. find_difference

Answer: Option B — set_difference

Q1126. What is the time complexity of searching for an element in an unordered_set in C++ STL?

A. O(1) ✓

B. O(log n)

C. O(n)

D. O(n log n)

Answer: Option A — O(1)

Q1127. Which algorithm in C++ STL is used to find the symmetric difference between two sets?

A. symmetric_difference

B. set_symmetric_difference ✓

C. distinct

D. find_symmetric_difference

Answer: Option B — set_symmetric_difference

Q1128. What is the purpose of the 'std::unordered_multimap' container in C++ STL?

A. To store multiple key-value pairs in a hash table ✓

B. To store key-value pairs in a sorted sequence

C. To store unique key-value pairs in a hash table

D. To store multiple key-value pairs in a sorted sequence

Answer: Option A — To store multiple key-value pairs in a hash table

Q1129. Which algorithm in C++ STL is used to rotate the elements in a range?

A. rotate ✓

B. shift

C. swap

D. rotate_range

Answer: Option A — rotate

Q1130. What is the purpose of the 'std::bitset' container in C++ STL?

A. To represent a fixed-size sequence of bits ✓

B. To represent a dynamic array

C. To store elements in a sorted sequence

D. To store elements in a hash table

Answer: Option A — To represent a fixed-size sequence of bits

---

Q1131. Which algorithm in C++ STL is used to shuffle the elements in a range randomly?

A. random_shuffle

B. shuffle ✓

C. mix

D. randomize

Answer: Option B — shuffle

Q1132. What is the purpose of the 'std::forward_list' container in C++ STL?

A. To provide a singly linked list implementation ✓

B. To provide a doubly linked list implementation

C. To store elements in a sorted sequence

D. To store elements in a hash table

Answer: Option A — To provide a singly linked list implementation

Q1133. Which algorithm in C++ STL is used to sort a range of elements using quicksort?

A. quick_sort

B. sort ✓

C. qsort

D. quicksort

Answer: Option B — sort

Q1134. What is the purpose of the 'std::array' container in C++ STL?

A. To represent a fixed-size array ✓

B. To represent a dynamic array

C. To store elements in a sorted sequence

D. To store elements in a hash table

Answer: Option A — To represent a fixed-size array

Q1135. Which algorithm in C++ STL is used to compute the sum of elements in a range?

A. sum

B. accumulate ✓

C. total

D. add_range

Answer: Option B — accumulate

Q1136. What will be the output of the following C++ code? #include <stdio.h> #include <math.h> int main () { printf ("%lf\n", fmod (5.3, 2)); printf ("%lf\n", fmod (18.5, 4.2)); return 0; }

A. 1.300

B. 1.700

C. 1.300 1.700 ✓

D. 1.65

Answer: Option C — 1.300 1.700

Q1137. What will be the output of the following C++ code? #include <typeinfo> #include <iostream> using namespace std; class shape { public: virtual void myvirtualfunc() const {} }; class mytriangle: public shape { public: virtual void myvirtualfunc() const { }; }; int main() { shape shape_instance; shape &ref_shape = shape_instance; try { mytriangle &ref_mytriangle = dynamic_cast (ref_shape); } catch (bad_cast) { cout << "Caught: bad_cast exception\n"; } return 0; }

A. Caught standard exception

B. No exception arises

C. Caught: bad_cast exception ✓

D. Caught: cast

Answer: Option C — Caught: bad_cast exception

Q1138. Which is best for coding the standard library for c++?

A. no trailing underscores on names

B. complex objects are returned by value

C. have a member-swap()

D. all of the mentioned ✓

Answer: Option D — all of the mentioned

Q1139. Where are standard exception classes grouped?

A. namespace std ✓

B. error

C. catch

D. final

Answer: Option A — namespace std

Q1140. What will be the output of the following C++ code? #include <iostream> #include <algorithm> #include <vector> using namespace std; int main () { vector<int> myvector; for (int i = 1; i < 6; ++i) myvector.push_back(i); reverse(myvector.begin(), myvector.end()); for (vector<int> :: iterator it = myvector.begin(); it != myvector.end(); ++it) cout << ' ' << *it; return 0; }

A. 1 2 3 4 5

B. 5 4 3 2 1 ✓

C. 0 1 2 3 4

D. 5 4 1 2 3

Answer: Option B — 5 4 3 2 1

Q1141. How many parameters are used in frexp function?

A. 1

B. 2 ✓

C. 3

D. 4

Answer: Option B — 2

Q1142. Pick out parameter for rehash method in unordered_set in c++?

A. count ✓

B. size

C. hash

D. type

Answer: Option A — count

Q1143. What are Container Adaptors?

A. Containers that implements data structures which can be accessed sequentially

B. Containers that implements sorted data structures for fast search in O(logn)

C. Containers that implements unsorted(hashed) data structures for quick search in O(1)

D. Containers that provide a different interface for sequential containers ✓

Answer: Option D — Containers that provide a different interface for sequential containers

Q1144. What are Unordered Associative Containers?

A. Containers that implements data structures which can be accessed sequentially

B. Containers that implements sorted data structures for fast search in O(logn)

C. Containers that implements unsorted(hashed) data structures for quick search in O(1) ✓

D. Containers that implements data structures which can be accessed non-sequentially

Answer: Option C — Containers that implements unsorted(hashed) data structures for quick search in O(1)

Q1145. How does the limits.h header file can be represented in C++?

A. limits

B. limit

C. climits ✓

D. dlimits

Answer: Option C — climits

Q1146. What will be the output of the following C++ code? #include <iostream> #include <vector> using namespace std; int main() { vector<int> v; v.assign(10, 42); for (int i = 0; i < v.size(); i++) { cout << v[i] << " "; } }

A. 42

B. 42 42

C. 424

D. 42 for 10 times ✓

Answer: Option D — 42 for 10 times

Q1147. Pick out the wrong header file.

A. <algorithm>

B. <containers>

C. <iterators>

D. <process> ✓

Answer: Option D — <process>

Q1148. What will be the output of the following C++ code? #include <stdio.h> #include <stdlib.h> int main () { char s[] = "365.24 29.53"; char* p; double d1, d2; d1 = strtod (s, &p); d2 = strtod (p, NULL); printf ("%.2f\n", d1/d2); return 0; }

A. 12

B. 12.37 ✓

C. 13

D. 15

Answer: Option B — 12.37

Q1149. What will be the output of the following C++ code? #include <iostream> #include <list> #include <queue> using namespace std; int main() { queue<char> q; q.push('a'); q.push('b'); q.push('c'); cout << q.front(); q.pop(); cout << q.front(); q.pop(); cout << q.front(); q.pop(); }

A. ab

B. abc ✓

C. a

D. error

Answer: Option B — abc

Q1150. What will be the output of the following C++ code? #include <list> #include <string> #include <iostream> using namespace std ; typedef list<string> LISTSTR; int main() { LISTSTR :: iterator i; LISTSTR test; test.insert(test.end(), "one"); test.insert(test.end(), "two"); LISTSTR test2(test); LISTSTR test3(3, "three"); LISTSTR test4(++test3.begin(), test3.end()); cout << "test:"; for (i = test.begin(); i != test.end(); ++i) cout << " " << *i << endl; cout << "test:"; for (i = test2.begin(); i != test2.end(); ++i) cout << " " << *i << endl; cout << "test:"; for (i = test3.begin(); i != test3.end(); ++i) cout << " " << *i << endl; cout << "test:"; for (i = test4.begin(); i != test4.end(); ++i) cout << " " << *i << endl; }

A. test

B. test one

C. test two

D. test: one two test: one two test: three three three test: three three ✓

Answer: Option D — test: one two test: one two test: three three three test: three three

---

Q1151. What is the use of <exception> header

A. Contains the standard exception files ✓

B. Contains the standard library files

C. It is used to arise an exception in the program

D. Reduce the memory size

Answer: Option A — Contains the standard exception files

Q1152. Which header file is used for Iterators?

A. <iterator> ✓

B. <algorithm>

C. <iter>

D. <loopIter>

Answer: Option A — <iterator>

Q1153. What will be the output of the following C++ code? #include <iostream> #include <exception> using namespace std; class myexc: public exception { virtual const char* what() const throw() { return "My exception"; } } myex; int main () { try { throw myex; } catch (exception& e) { cout << e.what() << endl; } return 0; }

A. My

B. My exception ✓

C. No exception

D. exception

Answer: Option B — My exception

Q1154. What is the use of make_heap in the heap operation?

A. Rearrange a heap

B. Deform a heap

C. Form a heap ✓

D. Delete a heap

Answer: Option C — Form a heap

Q1155. What will be the output of the following C++ code? #include <stdio.h> #include <math.h> int main () { printf ("The value of -3.1416 is %lf\n", fabs (-3.1416)); return 0; }

A. 3.1416

B. -3.1416

C. -3.141600

D. 3.141600 ✓

Answer: Option D — 3.141600

Q1156. What are the predefined exceptions in c++?

A. Memory allocation errors

B. I/O errors

C. Both Memory allocation errors & I/O errors ✓

D. static errors

Answer: Option C — Both Memory allocation errors & I/O errors

Q1157. Pick out the correct library in the following choices.

A. Search
B. Generate
C. Numeric
D. All of the mentioned ✓
Answer: Option D — All of the mentioned

Q1158. In how many categories, containers are divided?

A. 1
B. 2
C. 3
D. 4 ✓
Answer: Option D — 4

Q1159. Which header file is used to declare the standard exception?

A. #include<exception> ✓
B. #include<except>
C. #include<error>
D. #include<exce>
Answer: Option A — #include<exception>

Q1160. Pick out the wrong header file about strings.

A. <string>
B. <regex>
C. <ios> ✓
D. <reg>
Answer: Option C — <ios>

Q1161. What is the type of the first item in the heap?

A. Bigger than others ✓
B. Lower than others
C. Mean value of the heap
D. Equal to others
Answer: Option A — Bigger than others

Q1162. How many components STL has?

A. 1
B. 2
C. 3
D. 4 ✓
Answer: Option D — 4

Q1163. What will be the output of the following C++ code? #include <iostream> #include <exception> using namespace std; int main () { try { int* myarray= new int[1000]; cout << "Allocated"; } catch (exception& e) { cout << "Standard exception: " << e.what() << endl; } return 0; }

A. Allocated
B. Standard exception:
C. bad_alloc
D. Depends on memory ✓

Answer: Option D — Depends on memory

Q1164. What is meant by standard c++ library?

A. It is the collection of class definitions for standard data structures and a collection of algorithms ✓

B. It is a header file

C. Collection of algorithms

D. Step by step process

Answer: Option A — It is the collection of class definitions for standard data structures and a collection of algorithms

Q1165. How many Container Adaptors are provided by C++?

A. 2

B. 3 ✓

C. 4

D. 5

Answer: Option B — 3

Q1166. Which of the following mathematical function is overloaded in <complex> and <valarray>?

A. cos

B. tan ✓

C. sin

D. mod

Answer: Option B — tan

Q1167. Pick out the correct syntax of the header file that can be used with C++.

A. #include <float>

B. #include <float.h> ✓

C. Both #include <float> & #include <float.h>

D. #include <flot.h>

Answer: Option B — #include <float.h>

Q1168. What will be the output of the following C++ code? #include <stdio.h> #include <stdlib.h> int compareints (const void * a, const void * b) { return (*(int*)a - *(int*)b); } int values[] = { 50, 20, 60, 40, 10, 30 }; int main () { int * p; int key = 40; qsort(values, 6, sizeof (int), compareints); p = (int*) bsearch (&key, values, 6, sizeof (int), compareints); if (p != NULL) printf ("%d\n",*p); return 0; }

A. 10

B. 20

C. 40 ✓

D. 30

Answer: Option C — 40

Q1169. With which does the trigonometric functions work with angles in c++?

A. Degrees

B. Radians ✓

C. Both Degrees & Radians

D. Celsius

Answer: Option B — Radians

Q1170. What are the Associative Containers?

A. Containers that implements data structures which can be accessed sequentially

B. Containers that implements sorted data structures for fast search in O(logn) ✓

C. Containers that implements unsorted(hashed) data structures for quick search in O(1)

D. Containers that implements data structures which can be accessed non-sequentially

Answer: Option B — Containers that implements sorted data structures for fast search in O(logn)

Q1171. How many types of Iterators are provided by C++?

A. 2

B. 3

C. 4

D. 5 ✓

Answer: Option D — 5

Q1172. What are the containers?

A. Containers store objects and data ✓

B. Containers stores all the algorithms

C. Containers contain overloaded functions

D. Containers contain set of Iterators

Answer: Option A — Containers store objects and data

Q1173. What is the header file used for declaring the standard library algorithms?

A. container

B. algorithm ✓

C. library

D. iterator

Answer: Option B — algorithm

Q1174. What will be the output of the following C++ code? #include <iostream> #include <algorithm> #include <vector> using namespace std; int main () { int myints[] = {10, 20, 30, 30, 20, 10, 10, 20}; int mycount = count (myints, myints + 8, 10); cout << "10 appears " << mycount << " times.\n"; vector<int> myvector (myints, myints+8); mycount = count (myvector.begin(), myvector.end(), 20); cout << "20 appears " << mycount << " times.\n"; return 0; }

A. 3 3 ✓

B. 3 1

C. 8

D. 9

Answer: Option A — 3 3

Q1175. What will be the output of the following C++ code? #include <stdio.h> #include <stdlib.h> int main () { div_t divresult; divresult = div (38, 5); printf ("%d\n", divresult.rem); return 0; }

A. 7

B. 3 ✓

C. 4

D. 9

Answer: Option B — 3

Q1176. What will be the output of the following C++ code? #include <iostream> #include <algorithm> #include <vector> using namespace std; int main () { vector<int> myvector (4); fill (myvector.begin(), myvector.begin() + 2, 3); fill (myvector.begin() + 1, myvector.end() - 1, 4); for (vector<int> :: iterator it = myvector.begin(); it != myvector.end(); ++it) cout << ' ' << *it; return 0; }

A. 3 4

B. 3 4 4

C. 3 4 & 3 4 4

D. 3 4 4 0 ✓

Answer: Option D — 3 4 4 0

Q1177. What are Iterators?

A. Iterators are used to iterate over C-like arrays

B. Iterators are used to iterate over pointers

C. Iterators are used to point memory addresses of STL containers ✓

D. Iterators are used to iterate over functions

Answer: Option C — Iterators are used to point memory addresses of STL containers

Q1178. What will be the output of the following C++ code? #include <typeinfo> #include <iostream> using namespace std; class Test { public: Test(); virtual ~Test(); }; int main() { Test *ptrvar = NULL; try { cout << typeid(*ptrvar).name() << endl; } catch (bad_typeid) { cout << "The object is null" << endl; } return 0; }

A. No exception arises

B. The object is null ✓

C. Error

D. The object is

Answer: Option B — The object is null

Q1179. What will be the output of the following C++ code? #include <iostream> #include <algorithm> using namespace std; int main () { int myints[] = {10, 20, 30, 30, 20, 10, 10, 20}; int* pbegin = myints; int* pend = myints + sizeof(myints) / sizeof(int); pend = remove (pbegin, pend, 20); for (int* p = pbegin; p != pend; ++p) cout << ' ' << *p; return 0; }

A. 10, 20, 30, 30, 20, 10, 10, 20

B. 10, 30, 30, 10, 10 ✓

C. 10, 20, 20, 10, 10, 10, 20

D. 10, 20, 20, 10, 30, 10, 15

Answer: Option B — 10, 30, 30, 10, 10

Q1180. What is meant by vector in the container library contains?

A. It is a sequence container that encapsulates dynamic size arrays ✓

B. It is a sequence container that encapsulates static size arrays

C. It manages the memory

D. It manages the length and size

Answer: Option A — It is a sequence container that encapsulates dynamic size arrays

---

Q1181. How many macros are used by mathematical functions in the header file <cerrno>?

A. 1

B. 2

C. 3 ✓

D. 4

Answer: Option C — 3

Q1182. What will be the output of the following C++ code? #include <iostream> #include <algorithm> #include <vector> using namespace std; int main () { int first[] = {5, 10, 15, 20, 25}; int second[] = {50, 40, 30, 20, 10}; vector<int> v(10); vector<int> :: iterator it; sort (first, first + 5); sort (second, second + 5); it = set_union (first, first + 5, second, second + 5, v.begin()); cout << int(it - v.begin()); return 0; }

A. 6

B. 7

C. 8 ✓

D. 9

Answer: Option C — 8

Q1183. What are the Sequence Containers?

A. Containers that implements data structures which can be accessed sequentially ✓

B. Containers that implements sorted data structures for fast search in O(logn)

C. Containers that implements unsorted(hashed) data structures for quick search in O(1)

D. Containers that implements data structures which can be accessed non-sequentially

Answer: Option A — Containers that implements data structures which can be accessed sequentially

Q1184. Which of the following is best to include under try block?

A. static values

B. const values

C. dynamic allocations ✓

D. default values

Answer: Option C — dynamic allocations

Q1185. How many Associative Containers are provided by C++?

A. 2

B. 3

C. 4 ✓

D. 5

Answer: Option C — 4

Q1186. How many elements does a floating point number is composed of?

A. 1

B. 2

C. 3

D. 4 ✓

Answer: Option D — 4

Q1187. What will be the output of the following C++ code? #include <stdio.h> #include <stdlib.h> int main () { int n, m; n = abs(23); m = abs(-11); printf ("%d", n); printf ("%d", m); return 0; }

A. 23-11

B. 1123

C. 2311 ✓

D. 4325

Answer: Option C — 2311

Q1188. What is the Standard Template Library?

A. Set of C++ template classes to provide common programming data structures and functions ✓

B. Set of C++ classes

C. Set of Template functions used for easy data structures implementation

D. Set of Template data structures only

Answer: Option A — Set of C++ template classes to provide common programming data structures and functions

Q1189. Which header file is required for manipulation of math functions in c++?

A. cmath ✓

B. maths

C. math

D. dmath

Answer: Option A — cmath

Q1190. What will be the output of the following C++ code? #include <stdio.h> #include <math.h> int main () { double param, result; param = 5.5; result = log (param); printf ("%lf", result); return 0; }

A. 5.5

B. 3.14

C. 1.704 ✓

D. 2.4656

Answer: Option C — 1.704

Q1191. Pick out the correct method in the c++ standard library algorithm.

A. mismatch ✓

B. maximum

C. minimum

D. maxmatch

Answer: Option A — mismatch

Multi Threading In C++

Q1192. What is multi-threading in C++?

A. A process that allows parallel execution of tasks

B. A process that allows a single thread to execute

C. A process that allows multiple processes to execute concurrently

D. A process that allows multiple threads to execute concurrently ✓

Answer: Option D — A process that allows multiple threads to execute concurrently

Q1193. Which header file is used for multi-threading in C++?

A. <mutex>

B. <condition_variable>

C. <thread> ✓

D. <atomic>

Answer: Option C — <thread>

Q1194. What is the maximum number of threads that can run concurrently in C++?

A. 2

B. Depends on the hardware and operating system ✓

C. 4

D. 8

Answer: Option B — Depends on the hardware and operating system

Q1195. What is the purpose of the 'std::thread' class in C++ multi-threading?

A. To define thread-specific data

B. To manage thread priorities

C. To create and manage threads ✓

D. To synchronize access to shared resources

Answer: Option C — To create and manage threads

Q1196. Which of the following is a common synchronization primitive in C++ multi-threading?

A. Semaphore

B. Barrier

C. Spinlock

D. Mutex ✓

Answer: Option D — Mutex

Q1197. What is a race condition in multi-threading?

A. A condition where the outcome of a program depends on the order of thread execution

B. A condition where multiple threads try to access shared resources simultaneously ✓

C. A condition where threads wait indefinitely for a signal

D. A condition where threads execute in parallel

Answer: Option B — A condition where multiple threads try to access shared resources simultaneously

Q1198. Which function is used to join a thread in C++ multi-threading?

A. join() ✓

B. wait()

C. sync()

D. connect()

Answer: Option A — join()

Q1199. What is the purpose of the 'std::mutex' class in C++ multi-threading?

A. To manage thread priorities

B. To synchronize thread execution

C. To notify waiting threads

D. To provide mutual exclusion ✓

Answer: Option D — To provide mutual exclusion

Q1200. What is deadlock in multi-threading?

A. A situation where a thread blocks other threads from executing

B. A situation where threads execute out of order

C. A situation where two or more threads are waiting indefinitely for each other to release resources ✓

D. A situation where threads race to access shared resources

Answer: Option C — A situation where two or more threads are waiting indefinitely for each other to release resources

---

Q1201. Which of the following is used to avoid race conditions in C++ multi-threading?

A. Mutex

B. Semaphore

C. Condition variable

D. All of the above ✓

Answer: Option D — All of the above

Q1202. What is the purpose of the 'std::condition_variable' class in C++ multi-threading?

A. To notify waiting threads

B. To synchronize the execution of threads ✓

C. To manage thread priorities

D. None of the above

Answer: Option B — To synchronize the execution of threads

Q1203. Which of the following is true about thread safety in C++ multi-threading?

A. Thread safety ensures that threads do not race

B. Thread safety ensures that threads do not deadlock

C. Thread safety ensures that data accessed by multiple threads is synchronized ✓

D. Thread safety guarantees that threads execute in parallel

Answer: Option C — Thread safety ensures that data accessed by multiple threads is synchronized

Q1204. What is the purpose of the 'std::atomic' class in C++ multi-threading?

A. To manage thread priorities

B. To synchronize thread execution

C. To notify waiting threads

D. To perform atomic operations on shared variables ✓

Answer: Option D — To perform atomic operations on shared variables

Q1205. Which of the following is used to implement a barrier in C++ multi-threading?

A. std::barrier ✓

B. std::mutex

C. std::condition_variable

D. std::atomic

Answer: Option A — std::barrier

Q1206. What is the purpose of the 'std::async' function in C++ multi-threading?

A. To manage thread priorities

B. To synchronize access to shared resources

C. To join two threads asynchronously

D. To create a new thread and execute a function asynchronously ✓

Answer: Option D — To create a new thread and execute a function asynchronously

Q1207. Which of the following is a thread management library in C++?

A. pthread

B. std::thread ✓

C. thread.h

D. concurrent.h

Answer: Option B — std::thread

Q1208. What is the purpose of the 'std::lock_guard' class in C++ multi-threading?

A. To block the execution of a thread

B. To signal waiting threads

C. To acquire and release a mutex automatically ✓

D. To synchronize thread execution

Answer: Option C — To acquire and release a mutex automatically

Q1209. What is the purpose of the 'std::unique_lock' class in C++ multi-threading?

A. To manage thread priorities

B. To provide more flexibility than std::lock_guard ✓

C. To handle exceptions in threads

D. None of the above

Answer: Option B — To provide more flexibility than std::lock_guard

Q1210. What is the purpose of the 'std::condition_variable_any' class in C++ multi-threading?

A. To provide mutual exclusion

B. To synchronize the execution of threads

C. To wait for a condition to be satisfied by any type of mutex ✓

D. To signal waiting threads

Answer: Option C — To wait for a condition to be satisfied by any type of mutex

---

Q1211. What is the purpose of the 'std::packaged_task' class in C++ multi-threading?

A. To execute a task asynchronously

B. To provide mutual exclusion

C. To manage thread priorities

D. To package a task and its associated state ✓

Answer: Option D — To package a task and its associated state

Q1212. What is the purpose of the 'std::future' class in C++ multi-threading?

A. To represent the result of an asynchronous operation ✓

B. To execute a task asynchronously

C. To synchronize the execution of threads

D. To manage thread priorities

Answer: Option A — To represent the result of an asynchronous operation

Q1213. What is the difference between std::async and std::thread in C++ multi-threading?

A. std::async returns a std::future object representing the result of the asynchronous operation, while std::thread does not return anything

B. std::thread allows you to specify the execution policy, while std::async does not

C. std::async creates a new thread and executes a function asynchronously, while std::thread simply creates a new thread ✓

D. std::thread provides more control over thread creation and management compared to std::async

Answer: Option C — std::async creates a new thread and executes a function asynchronously, while std::thread simply creates a new thread

Q1214. What is the purpose of the 'std::promise' class in C++ multi-threading?

A. To signal waiting threads

B. To manage thread priorities

C. To acquire and release a mutex automatically

D. To store a value or an exception that will be made available asynchronously ✓

Answer: Option D — To store a value or an exception that will be made available asynchronously

Q1215. What is the difference between std::condition_variable and std::condition_variable_any in C++ multi-threading?

A. std::condition_variable is more efficient than std::condition_variable_any

B. std::condition_variable_any can work with any mutex type, while std::condition_variable can only work with std::mutex

C. std::condition_variable can only work with std::mutex, while std::condition_variable_any can work with any mutex type ✓

D. std::condition_variable_any is more efficient than std::condition_variable

Answer: Option C — std::condition_variable can only work with std::mutex, while std::condition_variable_any can work with any mutex type

---

Q1216. What is the purpose of the 'std::thread::hardware_concurrency()' function in C++ multi-threading?

A. To return the number of threads created by the program

B. To return the number of cores in the system

C. To return the number of hardware concurrency units supported by the implementation ✓

D. To return the number of threads currently running

Answer: Option C — To return the number of hardware concurrency units supported by the implementation

Q1217. What is the purpose of the 'std::recursive_mutex' class in C++ multi-threading?

A. To provide mutual exclusion

B. To allow a thread to lock the same mutex multiple times without causing a deadlock ✓

C. To manage thread priorities

D. None of the above

Answer: Option B — To allow a thread to lock the same mutex multiple times without causing a deadlock

Q1218. What is the purpose of the 'std::atomic_flag' class in C++ multi-threading?

A. To manage thread priorities

B. To synchronize the execution of threads

C. To provide atomic access to a boolean flag ✓

D. To provide mutual exclusion

Answer: Option C — To provide atomic access to a boolean flag

Q1219. What is the purpose of the 'std::latch' class in C++ multi-threading?

A. To signal waiting threads

B. To provide mutual exclusion

C. To manage thread priorities

D. To synchronize the execution of threads until a specified count is reached ✓

Answer: Option D — To synchronize the execution of threads until a specified count is reached

Q1220. What is the purpose of the 'std::barrier' class in C++ multi-threading?

A. To synchronize the execution of threads at a specified point ✓

B. To execute a task asynchronously

C. To signal waiting threads

D. To manage thread priorities

Answer: Option A — To synchronize the execution of threads at a specified point

---

Q1221. What is the purpose of the 'std::semaphore' class in C++ multi-threading?

A. To manage thread priorities

B. To synchronize the execution of threads

C. To provide mutual exclusion

D. To control access to a shared resource by multiple threads ✓

Answer: Option D — To control access to a shared resource by multiple threads

Q1222. Which of the following is not a synchronization mechanism in C++ multi-threading?

A. Semaphore

B. Mutex

C. Spinlock

D. Barrier ✓

Answer: Option D — Barrier

Q1223. What does the term "thread-safe" mean in the context of multi-threading?

A. The code is guaranteed to be error-free

B. The code can be safely executed by multiple threads concurrently ✓

C. The code can only be executed by a certain thread

D. None of the above

Answer: Option B — The code can be safely executed by multiple threads concurrently

Q1224. What is the primary purpose of a mutex in C++ multi-threading?

A. To manage thread priorities

B. To synchronize the execution of threads

C. To provide mutual exclusion ✓

D. To execute a task asynchronously

Answer: Option C — To provide mutual exclusion

Q1225. Which of the following is true about a condition variable in C++ multi-threading?

A. It allows multiple threads to share the same memory location

B. It provides mutual exclusion

C. It provides atomic access to shared variables

D. It allows one thread to notify other threads about a condition ✓

Answer: Option D — It allows one thread to notify other threads about a condition

---

Q1226. What is a deadlock in the context of multi-threading?

A. A situation where threads execute out of order

B. A situation where threads wait indefinitely for a signal

C. A situation where two or more threads are waiting indefinitely for each other to release resources ✓

D. A situation where a thread blocks other threads from executing

Answer: Option C — A situation where two or more threads are waiting indefinitely for each other to release resources

Q1227. Which C++11 feature introduced support for multi-threading?

A. std::mutex

B. std::atomic

C. std::condition_variable

D. std::thread ✓

Answer: Option D — std::thread

Q1228. What is the difference between std::mutex and std::recursive_mutex in C++ multi-threading?

A. std::mutex can be locked multiple times by the same thread, while std::recursive_mutex cannot

B. std::recursive_mutex can be locked multiple times by the same thread, while std::mutex cannot ✓

C. std::mutex cannot be locked multiple times by the same thread, while std::recursive_mutex can

D. They both provide the same functionality

Answer: Option B — std::recursive_mutex can be locked multiple times by the same thread, while std::mutex cannot

Q1229. Which of the following is not a potential issue in multi-threading?

A. Deadlock

B. Race condition

C. Starvation

D. Deterministic execution ✓

Answer: Option D — Deterministic execution

Q1230. What is the purpose of the 'std::this_thread::sleep_for' function in C++ multi-threading?

A. To block the execution of a thread

B. To synchronize the execution of threads

C. To terminate the current thread

D. To make the current thread sleep for a specified duration ✓

Answer: Option D — To make the current thread sleep for a specified duration

---

Q1231. What is the primary difference between a std::mutex and a std::spinlock in C++ multi-threading?

A. std::spinlock is a blocking synchronization primitive, while std::mutex is a non-blocking synchronization primitive

B. They both provide the same functionality

C. std::mutex is a blocking synchronization primitive, while std::spinlock is a non-blocking synchronization primitive ✓

D. std::spinlock is more efficient than std::mutex

Answer: Option C — std::mutex is a blocking synchronization primitive, while std::spinlock is a non-blocking synchronization primitive

Q1232. Which of the following is a potential benefit of multi-threading in C++?

A. Increased reliability

B. Reduced memory usage

C. Improved performance ✓

D. Decreased complexity

Answer: Option C — Improved performance

Q1233. What is the primary goal of using multi-threading in C++ programs?

A. To increase program size

B. To decrease program speed

C. To decrease program size

D. To improve program responsiveness ✓

Answer: Option D — To improve program responsiveness

C++ Miscellaneous

Q1234. What does the 'sizeof' operator return in C++?

A. Size of a data type in bits

B. Size of a data type in bytes

C. Size of a variable in bytes ✓

D. Size of a variable in bits

Answer: Option C — Size of a variable in bytes

Q1235. Which of the following is not a fundamental data type in C++?

A. Char

B. Float

C. String ✓

D. Double

Answer: Option C — String

Q1236. What is the purpose of the 'static' keyword in C++?

A. To declare a variable with dynamic storage duration

B. To declare a constant

C. To declare a variable with external linkage

D. To declare a variable with static storage duration ✓

Answer: Option D — To declare a variable with static storage duration

Q1237. What is the difference between '++i' and 'i++' in C++?

A. None of the above

B. They both have the same effect

C. '++i' increments the value of 'i' before returning it, while 'i++' increments the value of 'i' after returning it ✓

D. '++i' increments the value of 'i' after returning it, while 'i++' increments the value of 'i' before returning it

Answer: Option C — '++i' increments the value of 'i' before returning it, while 'i++' increments the value of 'i' after returning it

Q1238. What does the 'const' qualifier indicate in C++?

A. That the variable is a constant

B. That the variable has dynamic storage duration

C. That the variable has external linkage

D. That the value of the variable cannot be changed ✓

Answer: Option D — That the value of the variable cannot be changed

Q1239. What is the purpose of the 'nullptr' keyword in C++?

A. To represent a null pointer ✓

B. To represent a null reference

C. To represent a null character

D. To represent a null value

Answer: Option A — To represent a null pointer

Q1240. What is the difference between 'delete' and 'delete[]' in C++?

A. None of the above

B. 'delete' is used to deallocate memory allocated for an array of objects, while 'delete[]' is used to deallocate memory allocated for a single object

C. They both have the same effect

D. 'delete' is used to deallocate memory allocated for a single object, while 'delete[]' is used to deallocate memory allocated for an array of objects ✓

Answer: Option D — 'delete' is used to deallocate memory allocated for a single object, while 'delete[]' is used to deallocate memory allocated for an array of objects

---

Q1241. What is the purpose of the 'inline' keyword in C++?

A. To declare a variable as inline

B. To define a function as inline

C. To suggest the compiler to inline the function call ✓

D. To declare a function as static

Answer: Option C — To suggest the compiler to inline the function call

Q1242. What is the purpose of the 'volatile' qualifier in C++?

A. To indicate that a variable should be optimized by the compiler

B. To indicate that a variable may be modified by external factors ✓

C. To indicate that a variable should not be optimized by the compiler

D. None of the above

Answer: Option B — To indicate that a variable may be modified by external factors

Q1243. Which of the following is a valid way to initialize a constant variable in C++?

A. Using the 'volatile' keyword

B. Using the 'static' keyword

C. Using the 'const' keyword ✓

D. Using the 'constexpr' keyword

Answer: Option C — Using the 'const' keyword

Q1244. What is the purpose of the 'using' directive in C++?

A. To define a new namespace

B. To declare a variable

C. To declare a function

D. To introduce a name from a namespace into the current scope ✓

Answer: Option D — To introduce a name from a namespace into the current scope

Q1245. Which of the following is not a valid C++ data type?

A. long long

B. long double

C. unsigned float ✓

D. wchar_t

Answer: Option C — unsigned float

Q1246. What is the purpose of the typeid operator in C++?

A. To return the type of an expression ✓

B. To return the memory address of a type

C. To compare two types

D. To perform type casting

Answer: Option A — To return the type of an expression

Q1247. What does the mutable keyword in C++ mean when applied to a class member function?

A. It specifies that the member function cannot be overridden

B. It specifies that the member function does not modify the object's data members

C. It allows the member function to modify the object's data members declared as static

D. It allows the member function to modify the object's data members declared as const ✓

Answer: Option D — It allows the member function to modify the object's data members declared as const

Q1248. Which of the following correctly defines a lambda function in C++?

A. lambda(int x, int y) { return x + y; };

B. function lambda(int x, int y) { return x + y; };

C. auto lambda = [](int x, int y) { return x + y; }; ✓

D. auto lambda(int x, int y) -> int { return x + y; };

Answer: Option C — auto lambda = [](int x, int y) { return x + y; };

Q1249. What is the purpose of the static_cast operator in C++?

A. To perform implicit type conversions

B. To perform explicit type conversions ✓

C. To access static class members

D. None of the above

Answer: Option B — To perform explicit type conversions

Q1250. Which of the following is not a valid way to initialize an array in C++?

A. int arr[5] = {1, 2, 3, 4, 5};

B. int arr[] = {1, 2, 3, 4, 5};

C. int arr[5] {1, 2, 3, 4, 5};

D. int arr[5]; arr = {1, 2, 3, 4, 5}; ✓

Answer: Option D — int arr[5]; arr = {1, 2, 3, 4, 5};

Q1251. What does the nullptr keyword represent in C++?

A. A null pointer value ✓

B. A reference to a null pointer

C. An empty string

D. An integer value

Answer: Option A — A null pointer value

Q1252. What is the purpose of the std::move function in C++?

A. To access static class members

B. To allocate memory dynamically

C. To perform explicit type conversions

D. To cast an object to an rvalue reference ✓

Answer: Option D — To cast an object to an rvalue reference

Q1253. What is the purpose of the std::initializer_list class in C++?

A. To store a list of initializers

B. To initialize class objects with a list of values at runtime

C. To initialize containers with a list of values at compile time ✓

D. To allocate memory dynamically

Answer: Option C — To initialize containers with a list of values at compile time

Q1254. What is the purpose of the volatile keyword in C++?

A. To specify that a variable's value cannot be changed once initialized

B. To indicate that a variable may be changed unexpectedly by hardware or another thread ✓

C. To specify that a function cannot be overridden

D. None of the above

Answer: Option B — To indicate that a variable may be changed unexpectedly by hardware or another thread

Q1255. Which of the following is not a valid C++ standard library container?

A. queue

B. deque

C. ArrayList ✓

D. vector

Answer: Option C — ArrayList

Q1256. What is the purpose of the const keyword in C++?

A. To indicate that a variable may be changed unexpectedly by hardware or another thread

B. To indicate that a variable cannot be modified

C. To specify that a function cannot be overridden

D. To specify that a variable's value cannot be changed once initialized ✓

Answer: Option D — To specify that a variable's value cannot be changed once initialized

Q1257. What does RAII stand for in C++?

A. Resource Acquisition Is Initialization ✓

B. Resource Allocation Is Implementation

C. Resource Allocation Is Initialization

D. Resource Acquisition Is Implementation

Answer: Option A — Resource Acquisition Is Initialization

Q1258. Which class is used to design the base class?

A. abstract class ✓

B. derived class

C. base class

D. derived & base class

Answer: Option A — abstract class

Q1259. What will be the output of the following C++ code? #include <iostream> #include <limits> using namespace std; int main( ) { cout << numeric_limits<float> :: is_exact; cout << numeric_limits<double> :: is_exact; cout << numeric_limits<long int> :: is_exact; cout << numeric_limits<unsigned char> :: is_exact; }

A. 0010

B. 0012

C. 1100

D. 0011 ✓

Answer: Option D — 0011

Q1260. Where does the abstract class is used?

A. base class only ✓

B. derived class

C. both derived & base class

D. virtual class

Answer: Option A — base class only

Q1261. Which is the best design choice for using pointer to member function?

A. Interface ✓

B. Class

C. Structure

D. Block

Answer: Option A — Interface

Q1262. What is an abstract class in C++?

A. Class specifically used as a base class with atleast one virtual functions

B. Class specifically used as a base class with atleast one pure virtual functions ✓

C. Class from which any class is derived

D. Any Class in C++ is an abstract class

Answer: Option B — Class specifically used as a base class with atleast one pure virtual functions

Q1263. Which function is used to check whether a character is punctuation mark?

A. isxdigit()

B. isprint()

C. iscntrl()

D. ispunct() ✓

Answer: Option D — ispunct()

Q1264. Which function is used to calculate the norm of a complex number?

A. abs()

B. norm() ✓

C. mod()

D. square_sum()

Answer: Option B — norm()

Q1265. What will be the output of the following C++ code? #include <iostream> #include <cctype> using namespace std; int main(int argc, char const *argv[]) { char arr[20] = "\'H3ll0\'"; for(int i=0;i<8;i++) { cout<<(bool)ispunct(arr[i]); } }

A. 10000010 ✓

B. 111111111110

C. 111000111110

D. 111110000000

Answer: Option A — 10000010

Q1266. What is the syntax of printing the first element of an array Arr using get() function?

A. Arr.get(0)

B. get<0>(Arr) ✓

C. Arr.get[0]

D. get<0>[Arr]

Answer: Option B — get<0>(Arr)

Q1267. What will be the output of the following C++ code? #include <iostream> #include <vector> #include <forward_list> using namespace std; int main() { forward_list<int> fl1 = {1,7,8,9,10}; forward_list<int> fl2 = {2,3,4,5,6}; fl1.splice_after(fl1.begin(), fl2); for (int&c : fl1) cout << c << " "; cout<<endl; return 0; }

A. 1 2 3 4 5

B. 1 2 3 4 5 6 7 8 9 10 ✓

C. 1 7 8 9 10

D. 2 3 4 5 6

Answer: Option B — 1 2 3 4 5 6 7 8 9 10

Q1268. Which of the following is correct about map and multimap?

A. Map can have same keys whereas multimap cannot

B. Implementation of maps and multimap are different

C. Multimap can have same keys whereas the map cannot ✓

D. Average search time of map is greater than multimap

Answer: Option C — Multimap can have same keys whereas the map cannot

Q1269. Which of the following is correct about Input Iterators?

A. They cannot be decremented

B. Cannot be used in multi-pass algorithms

C. Can only be incremented

D. All of the mentioned ✓

Answer: Option D — All of the mentioned

Q1270. What will be the output of the following C++ code? #include <iostream> #include <functional> #include <numeric> using namespace std; int myop (int x, int y) { return x + y; } int main () { int val[] = {1, 2, 3, 5}; int result[7]; adjacent_difference (val, val + 7, result); for (int i = 0; i < 4; i++) cout << result[i] <<' '; return 0; }

A. 1 1 1 2 ✓

B. 1 2 3 1

C. 1 2 3 5

D. 1 2 5 6

Answer: Option A — 1 1 1 2

Q1271. Pick the incorrect statement.

A. Vectors have a dynamic size

B. Vectors are placed in contiguous storage

C. Insertion in vectors always takes constant time ✓

D. Vectors insert the element at the end

Answer: Option C — Insertion in vectors always takes constant time

Q1272. Given below classes which of the following are the possible row entries in vtable of D1 class? class Base { public: virtual void function1() {}; virtual void function2() {}; }; class D1: public Base { public: virtual void function1() {}; }; class D2: public Base { public: virtual void function2() {}; };

A. Base::function1() and Base::function2()

B. D1::function1() and Base::function2() ✓

C. Base::function1() and D1::function2()

D. D1::function1() and D1::function2()

Answer: Option B — D1::function1() and Base::function2()

Q1273. What will be the capacity of the vector after 10 is pushed into the vector in the following C++ code? #include <iostream> #include <vector> using namespace std; int main() { vector<int> v; for (int i = 1; i <= 5; i++) v.push_back(i); cout<<v.capacity()<<endl; v.shrink_to_fit(); cout<<v.capacity()<<endl; v.push_back(10); return 0; }

A. 8

B. 10 ✓

C. 5

D. 6

Answer: Option B — 10

Q1274. Which header file is used for generating random numbers?

A. <gen_dist>

B. <distribution>

C. <generator>

D. <random> ✓

Answer: Option D — <random>

Q1275. What will be the output of the following C++ code? #include<iostream> using namespace std; class X { int m; public: X() : m(10) { } X(int mm): m(mm) { } int getm() { return m; } }; class Y : public X { int n; public: Y(int nn) : n(nn) {} int getn() { return n; } }; int main() { Y yobj(100); cout << yobj.getm() << " " << yobj.getn() << endl; }

A. 10 100 ✓

B. 100 10

C. 10 10

D. 100 100

Answer: Option A — 10 100

Q1276. What will be the output of the following C++ code? #include <iostream> #include <bitset> using namespace std; int main() { bitset<8> b1(95); bitset<8> b2(45); cout<<~b1<<endl; cout<<(b1|b2)<<endl; cout<<(b1&b2)<<endl; }

A. 10100000 01111111 00001101 ✓

B. 11111111 01111111 00001101

C. 10100000 00001111 00001101

D. 10100000 01111111 00000000

Answer: Option A — 10100000 01111111 00001101

Q1277. Which function is used to check whether a character is a number?

A. isalpha()

B. isalnum()

C. isdigit() ✓

D. isblank()

Answer: Option C — isdigit()

Q1278. How many kinds of parameters are there in C++?

A. 1

B. 2

C. 3 ✓

D. 5

Answer: Option C — 3

Q1279. What will be the output of the following C++ code? #include <iostream> #include <functional> #include <numeric> using namespace std; int myfunction (int x, int y) { return x + 2 * y; } struct myclass { int operator()(int x, int y) { return x + 3 * y; } } myobject; int main () { int init = 100; int numbers[] = {10, 20, 30}; cout << accumulate(numbers, numbers + 3, init); cout << endl; }

A. 100

B. 140

C. 160 ✓

D. 180

Answer: Option C — 160

Q1280. Which is used to get the input during runtime?

A. cout

B. cin ✓

C. coi

D. cinout

Answer: Option B — cin

Q1281. What is meant by pure virtual function?

A. Function which does not have definition of its own ✓

B. Function which does have definition of its own

C. Function which does not have any return type

D. Function which does not have any return type & own definition

Answer: Option A — Function which does not have definition of its own

Q1282. What will be the output of the following C++ code? #include <iostream> #include <complex> using namespace std; int main() { complex <double> cn(3.0, 5.0); cout<<"Complex number is: "<<real(cn)<<" + "<<imag(cn)<<"i"<<endl; return 0; }

A. Complex number is: 3 + 5i ✓

B. Complex number is: 5 + 3i

C. Complex number is: 9 + 25i

D. Complex number is: 3 – 5i

Answer: Option A — Complex number is: 3 + 5i

Q1283. What will be the output of the following C++ code? #include <iostream> using namespace std; class poly { protected: int width, height; public: void set_values(int a, int b) { width = a; height = b; } }; class Coutput { public: void output(int i); }; void Coutput::output(int i) { cout << i; } class rect:public poly, public Coutput { public: int area() { return(width * height); } }; class tri:public poly, public Coutput { public: int area() { return(width * height / 2); } }; int main() { rect rect; tri trgl; rect.set_values(3, 4); trgl.set_values(4, 5); rect.output(rect.area()); trgl.output(trgl.area()); return 0; }

A. 1212

B. 1210 ✓

C. 1010

D. 1250

Answer: Option B — 1210

Q1284. Which of the following is correct about Input Iterators?

A. Input iterators can be used with all relational operators

B. Input iterators can work with arithmetic operators

C. No value can be assigned to the location pointed by Input Iterator ✓

D. Input iterators can work with sequence operators

Answer: Option C — No value can be assigned to the location pointed by Input Iterator

Q1285. How many types of sequence operations are provided by the C++ algorithm STL?

A. 1

B. 2 ✓

C. 3

D. 4

Answer: Option B — 2

Q1286. What will be the output of the following C++ code? #include <iostream> #include <memory> #include <string> using namespace std; int main () { string numbers[] = {"steve", "jobs"}; pair <string*, ptrdiff_t> result = get_temporary_buffer<string>(2); if (result.second>0) { uninitialized_copy (numbers, numbers + result.second, result.first); for (int i = 0; i < result.second; i++) cout << result.first[i] << " "; return_temporary_buffer(result.first); } return 0; }

A. steve

B. jobs

C. jobs steve

D. steve jobs ✓

Answer: Option D — steve jobs

Q1287. What kind of iteration does forward_list provide in C++?

A. Uni-directional ✓

B. Bi-directional

C. Multi-directional

D. Bi-directional & Multi-directional

Answer: Option A — Uni-directional

Q1288. Which of the following header file is needed to use vectors in your program?

A. <array>

B. <vector> ✓

C. <containers>

D. <stdio>

Answer: Option B — <vector>

Q1289. What will be the output of the following C++ code? #include <string> #include <iostream> using namespace std; void string_permutation(string& orig, string& perm) { if (orig.empty()) { cout<<perm<<endl; return; } for (int i = 0; i < orig.size(); ++i) { string orig2 = orig; orig2.erase(i, 1); string perm2 = perm; perm2 += orig.at(i); string_permutation(orig2, perm2); } } int main() { string orig = "ter"; string perm; string_permutation(orig, perm); return 0; }

A. ter

B. ert

C. ret

D. returns all the combination of ter ✓

Answer: Option D — returns all the combination of ter

Q1290. What type of class template is list?

A. Class-based

B. Node-based ✓

C. Method-based

D. size-based

Answer: Option B — Node-based

Q1291. Pick out the correct statement about vector.

A. vector<int> values (5) ✓

B. vector values (5)

C. vector<int> (5)

D. vector<5>

Answer: Option A — vector<int> values (5)

Q1292. How do define the user-defined exceptions?

A. inheriting and overriding exception class functionality ✓

B. overriding class functionality

C. inheriting class functionality

D. delting and adding class member

Answer: Option A — inheriting and overriding exception class functionality

Q1293. What will be the output of the following C++ code? #include <iostream> #include <vector> using namespace std; int main () { vector<int> first; first.assign (7,100); vector<int>::iterator it; it=first.begin()+1; int myints[] = {1776,7,4}; cout << int (first.size()) << '\n'; return 0; }

A. 10

B. 9

C. 8

D. 7 ✓

Answer: Option D — 7

Q1294. What will be the output of the following C++ code? #include <iostream> #include <Valarray> using namespace std; int main() { Valarray<int> varr = { 1, 2, 3, 4, 5 }; for (int &x: varr) cout << x << " "; cout<<endl; varr = varr.shift(2); for (int &x: varr) cout << x << " "; return 0; }

A. 1 2 3 4 5 3 4 5 0 0 ✓

B. 1 2 3 4 5 0 0 3 4 5

C. 1 2 3 4 5 1 2 3 4 5

D. 1 2 3 4 5 3 4 5 1 2

Answer: Option A — 1 2 3 4 5 3 4 5 0 0

Q1295. What function will be called when we have an uncaught exception?

A. catch

B. throw

C. terminate ✓

D. try

Answer: Option C — terminate

Q1296. What will be the output of the following C++ code? #include <iostream> #include <vector> #include <forward_list> using namespace std; int main() { forward_list<int> fl1 = {1,2,3,4,5}; for (int&c : fl1) cout << c << " "; cout<<endl; fl1.remove_if([](int x){ return x > 3;}); for (int&c : fl1) cout << c << " "; cout<<endl; return 0; }

A. 1 2 3 4 5 1 2 3 4 5

B. 1 2 3 4 5 1 2

C. 1 2 3 4 5 1 2 3 ✓

D. 4 5 1 2 3 4 5

Answer: Option C — 1 2 3 4 5 1 2 3

Q1297. What will be the output of the following C++ code? #include <iostream> #include <valarray> using namespace std; int main() { valarray<int> varr = { 10, 2, 20, 1, 30 }; cout<<"Sum of array: "<<varr.sum(); return 0; }

A. Sum of array: 20

B. Sum of array: 53

C. Sum of array: 12

D. Sum of array: 63 ✓

Answer: Option D — Sum of array: 63

Q1298. What will be the output of the following C++ code? #include <iostream> using namespace std; class Base { public: Base ( ) { cout << "1" << endl; } ~Base ( ) { cout << "2" << endl; } }; class Derived : public Base { public: Derived ( ) { cout << "3" << endl; } ~Derived ( ) { cout << "4" << endl; } }; int main( ) { Derived x; }

A. 1234

B. 4321

C. 1423

D. 1342 ✓

Answer: Option D — 1342

Q1299. What will be the output of the following C++ code? #include <iostream> #include <bitset> using namespace std; int main() { bitset<8> b1(95); bitset<8> b2 = b1 << 3; cout<<b2; }

A. 01110001

B. 11111000 ✓

C. 01111111

D. 00001101

Answer: Option B — 11111000

Q1300. What is the main feature of locale in C++?

A. Sustainability

B. Portability ✓

C. Reliability

D. Sustainability & Reliability

Answer: Option B — Portability

Q1301. Elements in STL heap are removed in . . . . . .

A. decreasing order ✓

B. increasing order

C. alternate i.e. once max element then min element

D. input order

Answer: Option A — decreasing order

Q1302. What will be the output of the following C++ code? #include <iostream> #include <algorithm> #include <vector> using namespace std; int main () { vector<int> myvector; for (int i = 1; i < 5; ++i) myvector.push_back(i); rotate(myvector.begin(), myvector.begin() + 3, myvector.end( )); for (vector<int> :: iterator it = myvector.begin(); it != myvector.end(); ++it) cout << ' ' << *it; return 0; }

A. 1 2 3 4

B. 4 3 2 1

C. 3 4 2 1

D. 4 1 2 3 ✓

Answer: Option D — 4 1 2 3

Q1303. What are command line arguments?

A. Arguments passed to main() function ✓

B. Arguments passed to any function

C. Arguments passed to class functions

D. Arguments passed to structure functions

Answer: Option A — Arguments passed to main() function

Q1304. How many parameters are there in getline function?

A. 1

B. 2

C. 2 or 3 ✓

D. 3

Answer: Option C — 2 or 3

Q1305. Which of the following relationships is uni-directional?

A. Aggregation

B. Association

C. Composition

D. Both Aggregation and Composition ✓

Answer: Option D — Both Aggregation and Composition

Q1306. What will be the output of the following C++ code? #include <iostream> using namespace std; int main() { int a = 10; if (a < 15) { time: cout << a; goto time; } break; return 0; }

A. 1010

B. 10

C. infinitely print 10

D. compile time error ✓

Answer: Option D — compile time error

Q1307. Which is used to create a pure virtual function?

A. $

B. =0 ✓

C. &

D. !

Answer: Option B — =0

Q1308. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main () { int num = 3; string str_bad = "wrong number used"; try { if (num == 1) { throw 5; } if (num == 2) { throw 1.1f; } if (num != 1 || num != 2) { throw str_bad; } } catch (int a) { cout << "Exception is: " << a << endl; } catch (float b) { cout << "Exception is: " << b << endl; } catch (...) { cout << str_bad << endl; } return 0; }

A. Exception is 5

B. Exception is 1.1f

C. Wrong number used ✓

D. Exception is 1.6g

Answer: Option C — Wrong number used

Q1309. Which of the following function is used to insert an element at the end of a vector?

A. push_back() ✓

B. pop_back()

C. front()

D. end()

Answer: Option A — push_back()

Q1310. What are the tuples in C++?

A. Objects that can hold more than one element of different types ✓

B. Objects that can hold a single element of complex type

C. Objects that can hold more than one element of the same types

D. Objects that can hold a single element of fundamental type

Answer: Option A — Objects that can hold more than one element of different types

---

Q1311. How many ways of reusing are there in the class hierarchy?

A. 1

B. 2 ✓

C. 3

D. 4

Answer: Option B — 2

Q1312. Which header file is used for the numeric limits in C++?

A. <iostream>

B. <limits> ✓

C. <number>

D. <digit>

Answer: Option B — <limits>

Q1313. What will be the output of the following C++ code? #include <iostream> #include <string> #include <tuple> using namespace std; int main() { tuple <int, char, string> tp1; tuple <int, char, string> tp2; tp1 = make_tuple(6, 'a', "Hello"); tp2 = make_tuple(9, 'z', "World"); cout<<"("<<get<0>(tp1)<<", "<<get<1>(tp1)<<", "<<get<2>(tp1)<<")"<<endl; cout<<"("<<get<0>(tp2)<<", "<<get<1>(tp2)<<", "<<get<2>(tp2)<<")"<<endl; tp1.swap(tp2); cout<<"("<<get<0>(tp1)<<", "<<get<1>(tp1)<<", "<<get<2>(tp1)<<")"<<endl; cout<<"("<<get<0>(tp2)<<", "<<get<1>(tp2)<<", "<<get<2>(tp2)<<")"<<endl; return 0; }

A. (6, a, Hello) (9, z, World) (9, z, World) (6, a, Hello) ✓

B. (6, a, Hello) (9, z, World) (6, a, Hello) (9, z, World)

C. (9, z, World) (6, a, Hello) (9, z, World) (6, a, Hello)

D. (6, a, Hello) (6, a, Hello) (9, z, World) (9, z, World)

Answer: Option A — (6, a, Hello) (9, z, World) (9, z, World) (6, a, Hello)

Q1314. What will be the output of the following C++ code? #include <iostream> #include <vector> #include <algorithm> using namespace std; void show(const vector<int>& vi) { for (size_t i = 0; i < vi.size(); ++i) cout << vi[i]; cout << endl; } int main() { vector<int> vi; vi.push_back(3); vi.push_back(5); vi.push_back(5); sort(vi.begin(), vi.end()); show(vi); while(next_permutation(vi.begin(), vi.end())) show(vi); return 0; }

A. 355

B. 535

C. 553

D. 355 535 553 ✓

Answer: Option D — 355 535 553

Q1315. In how many different ways any-container can be constructed?

A. 1

B. 2

C. 3 ✓

D. 4

Answer: Option C — 3

Q1316. What does this template function indicates? ========== template<class T, class U> U func(T a, U b) { cout<<a<<"\t"<<b; } ==========

A. A function taking a single generic parameter and returning a generic type which may be different from argument type ✓

B. A function taking a single generic parameter and returning a generic type which must be different from argument type

C. A function taking a single generic parameter and returning a generic type which must have the same type as argument type

D. A function taking a single generic parameter and returning a specific non-void type

Answer: Option A — A function taking a single generic parameter and returning a generic type which may be different from argument type

Q1317. What will be the output of the following C++ code? #include <iostream> #include <algorithm> #include <vector> using namespace std; int main () { vector<int> v = {4,2,10,5,1,8}; sort(v.begin(), v.end()); if (binary_search(v.begin(), v.end(), 4)) cout << "found.\n"; else cout << "not found.\n"; return 0; }

A. found. ✓

B. not found.

C. Error

D. Segmentation fault

Answer: Option A — found.

Q1318. Identify the correct statement about throw(type).

A. A function can throw any type of exceptions

B. A function can throw an exception of certain type only ✓

C. A function can't throw any type of exception

D. A function can catch all types of exceptions

Answer: Option B — A function can throw an exception of certain type only

Q1319. Which operator is used as catch-all handler?

A. ellipses operator ✓

B. ternary operator

C. string operator

D. unary operator

Answer: Option A — ellipses operator

Q1320. What will be the output of the following C++ code? #include <iostream> #include <string> using namespace std; int main() { string s = "a long string"; s.insert(s.size() / 2, " * "); cout << s << endl; return 0; }

A. a long* string

B. a long st*ring

C. Depends on compiler ✓

D. a long string*

Answer: Option C — Depends on compiler

Q1321. What will be the output of the following C++ code? #include <iostream> #include <vector> using namespace std; int main() { vector<int> v; for (int i = 1; i <= 5; i++) v.push_back(i); for(int i=0;i<v.size();i++) cout<<v[i]<<" "; cout<<endl; v.assign(3, 8); for(int i=0;i<v.size();i++) cout<<v[i]<<" "; cout<<endl; return 0; }

A. 1 2 3 4 5 8 8 8 ✓

B. 1 2 3 4 5 8 8 8 8 8

C. 1 2 3 4 5 8 8 8 4 5

D. 1 2 3 4 5

Answer: Option A — 1 2 3 4 5 8 8 8

Q1322. What will be the output of the following C++ code? #include <iostream> #include <vector> using namespace std; int main () { vector<int> myvector; myvector.push_back(78); myvector.push_back(16); myvector.front() += myvector.back(); cout << myvector.front() << '\n'; return 0; }

A. 78

B. 16

C. 94 ✓

D. 86

Answer: Option C — 94

Q1323. What is the default operation of adjacent_difference function in numeric library?

A. Difference ✓

B. Addition

C. Multiplication

D. Subtraction

Answer: Option A — Difference

Q1324. What will be the output of the following C++ code? #include <iostream> #include <algorithm> #include <vector> using namespace std; bool IsOdd (int i) { return (i%2)==1; } int main () { vector<int> v = {4,2,10,5,1,8}; for(int i=0;i<v.size();i++) cout<<v[i]<<" "; cout<<endl; sort(v.begin(), v.end()); for(int i=0;i<v.size();i++) cout<<v[i]<<" "; return 0; }

A. 4 2 10 5 1 8 1 2 4 5 8 10 ✓

B. 1 2 4 5 8 10 4 2 10 5 1 8

C. 4 2 10 5 1 8 4 2 10 5 1 8

D. 1 2 4 5 8 10 1 2 4 5 8 10

Answer: Option A — 4 2 10 5 1 8 1 2 4 5 8 10

Q1325. How many types of Association can be there between classes?

A. 1

B. 2

C. 3

D. 4 ✓

Answer: Option D — 4

Q1326. What will be the output of the following C++ code? #include <iostream> using namespace std; template<typename T> inline T square(T x) { T result; result = x * x; return result; }; int main() { int i, ii; float x, xx; double y, yy; i = 2; x = 2.2; y = 2.2; ii = square(i); cout << i << " " << ii << endl; yy = square(y); cout << y << " " << yy << endl; }

A. 2 4 2.2 4.84 ✓

B. 2 4

C. error

D. 3 6

Answer: Option A — 2 4 2.2 4.84

Q1327. What type of access does deque and vector provide?

A. Linear access

B. Parallel access

C. Random access ✓

D. Memory access

Answer: Option C — Random access

Q1328. What are Iterators?

A. STL component used to point a memory address of a container ✓

B. STL component used for vectors

C. STL component used to call functions efficiently

D. STL component used to define template classes

Answer: Option A — STL component used to point a memory address of a container

Q1329. Indexing of bitset variables starts from . . . . . .

A. leftmost bit

B. rightmost bit ✓

C. same as in an array

D. front

Answer: Option B — rightmost bit

Q1330. What will be the output of the following C++ code? #include <iostream> using namespace std; class Base { protected: int a; public: Base() { a = 34; } Base(int i) { a = i; } virtual ~Base() { if (a < 0) throw a; } virtual int getA() { if (a < 0) { throw a; } } }; int main() { try { Base b(-25); cout << endl << b.getA(); } catch (int) { cout << endl << "Illegal initialization"; } }

A. Illegal initialization

B. Terminate called after throwing an instance of 'int' ✓

C. Illegal initialization & terminate called after throwing an instance

D. initialization

Answer: Option B — Terminate called after throwing an instance of 'int'

Q1331. By using which function does the buffer are automatically flushed?

A. fopen

B. copy

C. compare

D. fclose ✓

Answer: Option D — fclose

Q1332. Which function is used to access the last element of an array class?

A. end()

B. start()

C. back() ✓

D. last()

Answer: Option C — back()

Q1333. How many constructors can present in a class?

A. 1

B. 2

C. 3

D. multiple ✓

Answer: Option D — multiple

Combined from various sources by SFI DCA CUSAT